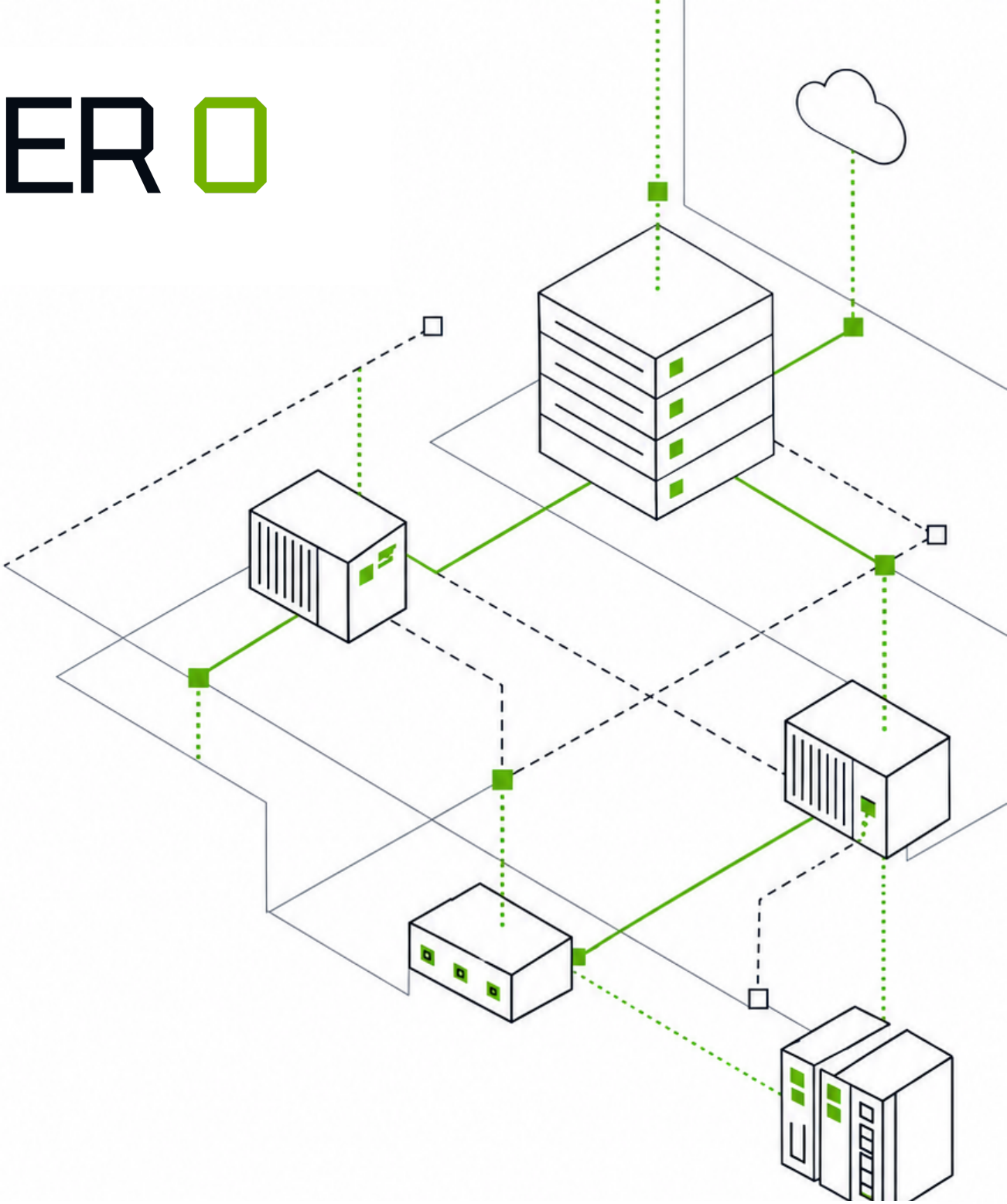
TIER 0



Tier0 Technical Guidance Manual

Table of Contents

GET STARTED

1. Choosing the Best Product
2. Installation
3. Try the Demo Factory in Tier0

USING TIER0

1. Factory Data Foundation
2. Preparing Data Foundation
3. Manage Data from Center to Edge
4. Working with Factory Data
5. Building Apps
6. Building Digital Twins
7. Analyzing Data
8. Operating on Tier0 with Agents

BEST PRACTICE

1. Modeling Factory Data
2. Connecting Industrial Protocols
3. Security and Resilience
4. Building Analytics Apps
5. Building Shopfloor Workflow with Agent
6. Displaying Data with Digital Twin

REFERENCE

1. API Reference
2. CLI Commands Reference
3. Embedded Builder Skills
4. User Management
5. Tier0 Cloud Network and Data Security Overview
6. ISA-95 Equipment Hierarchy
7. Standard Deployment Plan
8. High Availability Deployment Plan
9. SLA and High Availability Boundaries
10. Standard Port List
11. Operations Runbook

APPENDIX

1. Complete OpenAPI Reference

Get Started

Choosing the Best Product

Edge, Cloud, or Enterprise — positioning, plans, and a capability matrix for picking your Tier0 edition.

Check here for detailed features of each product and make the choice that fits you the best.

Next

- Build Apps on UNS — Build and deploy Tier0 app on Enterprise
- Installation

GET STARTED

Installation

Install Tier0 by edition.

Tier0 ships in three editions:

- **Cloud** - Managed SaaS, no infrastructure required.
- **Enterprise** - Private deployment for enterprise environments.
- **Edge** - Open-source, runs locally with Docker.

See Choosing the Best Version to find the edition that best fits your requirements.

Installing Tier0

TAB: CLOUD

Apply for a trial account before exploring Tier0 Cloud.

1. Register your account at Register Tier0.
2. Use the verification code sent to your email to verify and then log in with the registered account.
3. Create a workspace and start your trial.

TAB: ENTERPRISE

Requirements

- Single-Node Deployment
  - Minimum: 4C / 8G / 100G
  - Recommended: 8C / 16G / 500G
- Standard Deployment
  - Minimum: 4 VMs
  - Required baseline per VM types:
    - App: 8C / 32G / 100G

        - Database: 4C / 8G / 500G
        - Witness: 2C / 4G / 40G
- High Availability Deployment
    - Minimum: 6 VMs
    - Required baseline per VM types:
        - App (Swarm): 8C / 32G / 100G x 3
        - Database: 4C / 8G / 500G x 2
        - Witness: 2C / 4G / 40G

> **Hardware requirements may vary**
> Enterprise hardware requirements vary by deployment scenarios.

> **Note**
> Detailed deployment information is available in the **Standard Deployment Plan** and the **High Availability Deployment Plan**.

Operations

1. Get the package from your vendor, and verify it.

```
sha256sum -c tier0-enterprise-offline-standard-YYYYmmddTHHMMSS.tar.gz.sha256
```

2. Decompress the package.

```
tar -xzf tier0-enterprise-offline-standard-YYYYmmddTHHMMSS.tar.gz
cd tier0-enterprise-offline-standard-YYYYmmddTHHMMSS
```

3. Edit `install.env` based on the deployment details, then start the deployment.

```
vi install.env
```

```
bash install.sh
```

Common Maintenance Operations

- Check the deployment environment without installing Enterprise.

```
./install.sh --check-only
```

- Clear all nodes after a failed deployment.

```
./full-cleanup.sh --env install.env --yes
```

> ⚠ **What difference it makes**
> This command wipes all Tier0 runtime data and database backups while keeping the VMs, operating system, and installed Docker software.

- A deployment report is generated after installation.

```
runtime/install-reports/<timestamp>/deployment-report.md
```

> ⚠ **What you should do when FAIL appears in the report**
> Do not deliver the project if the report contains any **FAIL** item. Contact support first.

Maintenance References

- Standard Port List
- Maintenance and Troubleshooting Guide
- HA Boundaries Instructions

TAB: EDGE

Requirements

- Minimum 4-core CPU, 8 GB RAM, 100 GB disk
- Ubuntu 24.04 or Windows 10/11 with Docker and Git

Operations

1. Clone the Tier0-Edge repository from GitHub to your server and access the project.

```
git clone https://github.com/FREEZONEX/Tier0-Edge
cd Tier0-Edge/deploy
```

2. Configure the parameters listed in the table below in `.env.default` according to your environment, and save the changes.

```
cp .env.default .env
vi .env
:wq!
```

Parameters to be configured

Field	Type	Description	Example
VOLUMES_PATH	string	Storage path for project data	`/srv/tier0/data` on Linux; `D:/tier0/data` or `/d/tier0/data` in Windows Bash
ENTRANCE_DOMAIN/ENTRANCE_PORT	string	Tier0 access endpoint	`192.168.1.10` / `8088`

Field	Type	Description	Example
		(IP/domain + port)	
LANGUAGE	enum	zh-CN / en-US	`en-US`
ADMIN_INIT_PASSWORD	string	Initial password for the admin user	A strong private password
COMPOSE_PROJECT_NAME/ PORT_OFFSET	string	Isolate multiple instances running on the same host	`edge_lab` / `100`

3 Run the installation script and check the installation status.

```
bash bin/install.sh
bash bin/compose.sh ps # Check the status of the containers
bash bin/compose.sh logs -f backend # Check the logs of the backend container
curl -fsS http://127.0.0.1:8088/readyz # Check the readiness of the backend service
```

> ⚠ **Caution**
> Do not edit `deploy/.env.runtime`; it contains generated secrets and resolved runtime values.

4 Access Tier0 with the set endpoint and the default admin account:

- Username: `tier0`
- Password: The value of `ADMIN_INIT_PASSWORD` in `.env` or `tier0` if not set.

Next

- Try the Demo Factory - A preloaded plant to explore

- **Choosing the Best Version** - Compare each version and select the one best fits you

GET STARTED

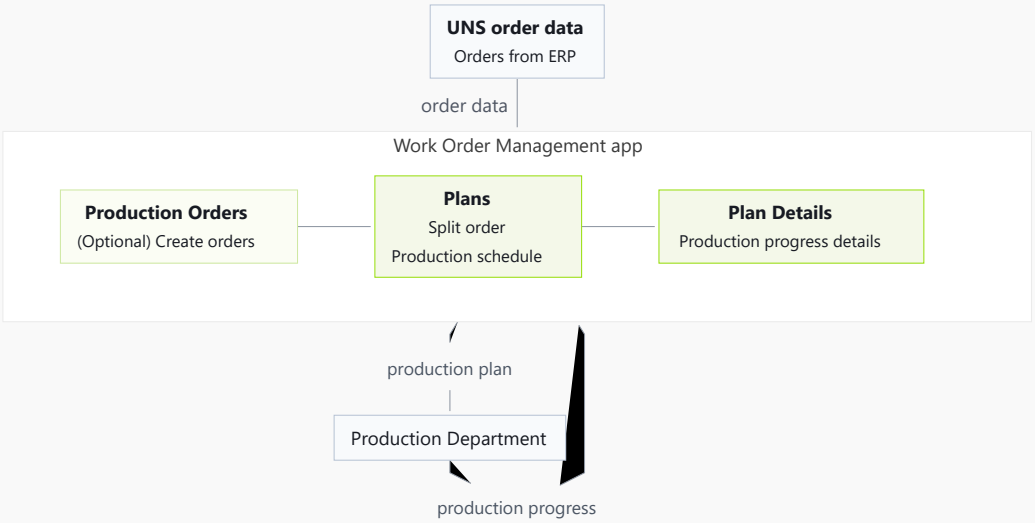
Try the Demo Factory in Tier0

Explore a preloaded, working plant in your Cloud trial workspace — namespace, flows, notebooks, and apps.

A demo factory that already works on Tier0 shows you how the factory data is modeled, managed and used in a smart and smooth way.

Business Background

The factory in Tier0 receives orders from ERP, splits them, schedules production, sends the schedule to production, and receives production progress data as work proceeds.



Process

1. Go to **UNS** and check the details of orders that came from ERP.
   - `DemoFactory/ERP/ProductionOrders/State/UpsertProductionOrder` : Orders.

- `DemoFactory/ERP/ProductionOrders/State/OrderList`: Current order list snapshot.

> **How is data collected into UNS?**
> Go to **Flows** > **Source Flow** > **DemoFactory-Flow** to check the data collection details.

2. (optional) Go to **Launchpad**, access the **Work Order Management** application, and create orders on the **Production Orders** page.
3. In **Work Order Management**, split orders and schedule production plans for the split work orders on the **Plans** page.
4. Send the plan to production, and check the plan details on the following topics on **UNS**.

   - `DemoFactory/ERP/WorkOrderPlan/Metric/SplitCount`: The number of work orders after splitting.
   - `DemoFactory/ERP/WorkOrderPlan/State/PlanStatus`: Current plan status.
   - `DemoFactory/ERP/WorkOrderPlan/State/WorkOrderList`: The work order list after the production plan is scheduled.

> **Note**
> The application directly sends the plans and work orders to **UNS**.

5. Check the production progress data on the **Plan Details** page in the application.

> **Where do the details come from?**
> The production progress data is sent from production department and collected by **DemoFactory-Flow** in **Source Flow** to **UNS**, and displayed on the **Plan Details** page.
>
> - `DemoFactory/Site_01/Production/Line_01/WorkOrderExecution/State/CurrentWorkOrder`: The order in process.
> - `DemoFactory/Site_01/Production/Line_01/WorkOrderExecution/State/WorkOrderStatus`: The execution status of the current order.
> - `DemoFactory/Site_01/Production/Line_01/WorkOrderExecution/Metric/Target_Qty`: The target production amount of the current order.

- `DemoFactory/Site_01/Production/Line_01/WorkOrderExecution/Metric/Produced_Qty`: The quantity already produced for the current order.
- `DemoFactory/Site_01/Production/Line_01/WorkOrderExecution/Metric/Defect_Qty`: The defect quantity for the current order.
- `DemoFactory/Site_01/Production/Line_01/WorkOrderExecution/Metric/Completion_Rate`: The completion rate of the current order.

Next

- Choosing the Best Version - Compare each version and select the one best fits you
- UNS Concepts - Understand data modeling in Unified Namespace

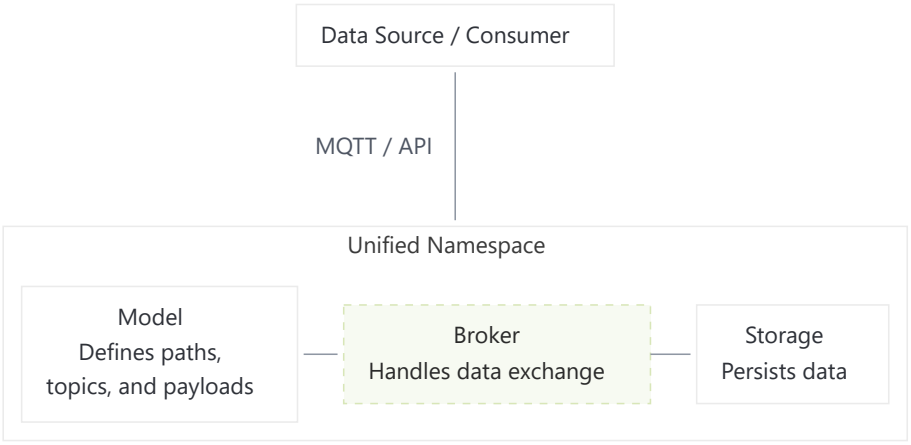
Factory Data Foundation

The Unified Namespace model: Metric, State, Action, and how the UNS stores data.

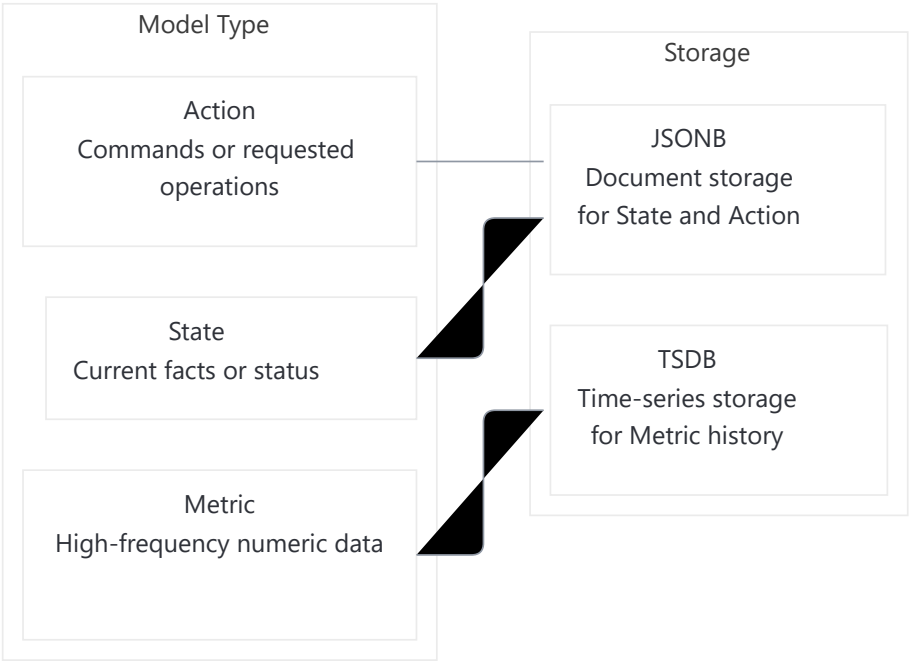
In Tier0, factory data is organized into clear, tree-structured models based on UNS (Unified Namespace) methodology, forming a unified data foundation.

Data Foundation Architecture

- The broker receives and sends data through model topics.



- Models are categorized into three types, **Metric**, **State**, and **Action**.
- Data is stored in different databases based on model type.
  - **Action/State**: JSONB (PostgreSQL)
  - **Metric**: Real-time data (TSDB)



Broker

The UNS broker is the platform component that receives and sends data through MQTT or OpenAPI based on data models.

- **By MQTT**: The complete path of each node in the model tree works as the MQTT topic address to pass the message payload.
- **By OpenAPI**: Perform CRUD commands on data models through HTTP requests.

> **API Reference**
> For API details, see **API Reference**.

Model

Which guide should I follow when building a model?
Choose the modeling approach based on what the data represents.

- **Physical Data**: Real-time data from physical machines. It is organized based on the **ISA-95** standard and relies heavily on **location hierarchy**.
- **Business Data**: Data related to production requirements, schedules, tasks, and dispatching. It is usually organized by **business domain and object**.

Metric, State, Action

Industrial data changes at different frequencies and serves different purposes, so Tier0 separates UNS models into three fixed types.

Type	Object	Example
`METRIC`	Designed for high-frequency data that is collected continuously and needs time-series storage.	Machine temperature/pressure/voltage
`STATE`	Designed for data that changes less often and describes the current condition or fact of an object.	Machine operational status
`ACTION`	Designed for lower-frequency command data that records what an external system or operator should execute.	Start batch, stop machine

Path/Topic, Payload

Each UNS model contains path, topic, and payload. Together, they form a **tree** structure that carries business context and transmits industrial data.

- **Path/Topic**: The path represents data ownership, and the topic carries the message payload. Together, they define the ***model address***.
- **Payload**: The message payload held by the topic. It is JSON in a **key-value** format and defines the ***model data***. For example, a model in UNS looks like the tree displayed below:

TAB: PHYSICAL DATA

```
Factory_A
└── Site_01
    └── SMT_Line_1
        └── Metric
            └── Machine_001
```

- **Path**: `Factory_A/Site_01/SMT_Line_1/Metric`
- **Topic**: `/Machine_001`.
- **Payload**

```
"temperature": 85,
"vibration": 2.8,
```

TAB: BUSINESS DATA

```
Factory_A
└── ERP
    └── ProductionOrders
        └── State
            └── OrderList
```

- **Path**: `Factory_A/ERP/ProductionOrders/State`
- **Topic**: `/OrderList`.
- **Payload**

```
"orders": "0123569876",
"updated_at": 2026-7-13,
```

To build a model
See How to Build a Data Model.

Storage

Tier0 uses PostgreSQL to store data. Different data types change at different rates and have different storage requirements.

Data Type	Storage	Description
`METRIC`	TSDB	Optimized for high-frequency measurements such as temperature and pressure. Each topic includes default fields: `timestamp` and `quality`.
`STATE`	PostgreSQL	Stores machine or system states in a flexible `JSONB` format.
`ACTION`	PostgreSQL	Stores commands or actions in a flexible `JSONB` format.

Next

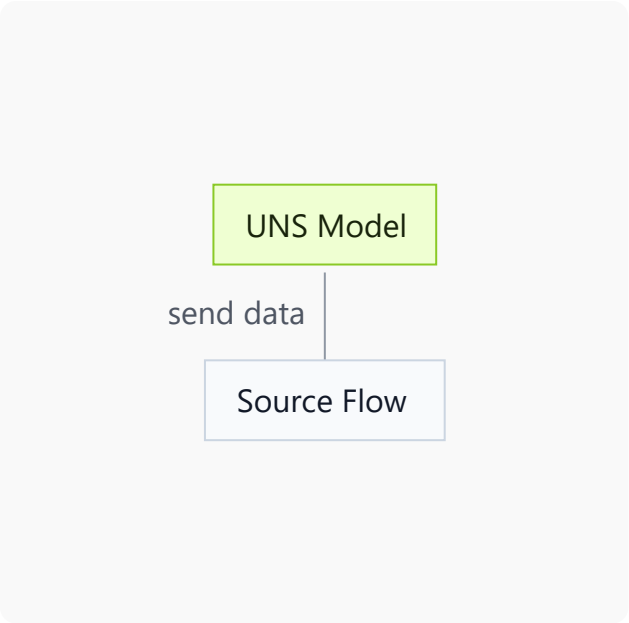
- Connect Data to UNS - How to model your plant to UNS and connect data to the model.
- Working with UNS Data - How to operate on your UNS model through MQTT and APIs.

USING TIER0

Preparing Data Foundation

Model your namespace and bring data in with Source Flows. Available in all editions.

First build a model in **UNS**, then connect data through **Source Flow** to the model, forming a data foundation for subsequent use.



Build a Data Model

Define the data hierarchy as a tree map according to:

- The physical location the data comes from (ISA-95);
- The business logic the data conforms to.

Build Models Manually

> **Use drafts to verify models first**
> Create or import models in **Draft** to verify whether the models conform to model principles and deploy them after validity.

1. Design the model structure, and add paths by order before adding the topic.
2. Set **Topic Type** among **Metric**, **State** and **Action** based on data features.

3. Select **Mock Data** to send simulated data to the model, and **Enable History** to store the data to database.

> **Note**
> A flow will be created in **Flows** to send simulated data to the model.

4. Add data fields to the topic with the correct data types by **Auto Parsing** or **Pre-defined**.

   - **Auto Parsing**: Convert fields automatically from JSON text in batches.

     ```
     {
       "temp":85.5
     }
     ```

     > **Caution**
     > Can only parse single layer JSON, and for multi-layer JSON, only the first layer will be parsed.

   - **Pre-defined**: Manually set fields for the topic one by one.

Import Models

> **Tip**
> Use LLMs like ChatGPT to help import models.

1. Send the template JSON from **Import** to AI, and use a similar prompt.

   ```
   Generate a UNS model used for xx in xx plant, including xx equipment and
   data sources based on the template.
   ```

2. Import the generated result in UNS.

Connect Data to UNS

Data Flow Features

- Based on Node-RED.
- Every flow ends with an `mqtt out` node. It works as an MQTT client to publish data to the UNS broker.
- The UNS broker is embedded in the `mqtt out` and `mqtt in` nodes with the same name as the flow.
- When a UNS model is used as the topic, data goes directly to the corresponding model in **UNS**.

Steps

1. Use nodes based on the data source type to build a flow, and end it with an `mqtt out` node.

   > **If you need nodes that are not there**
   > Install nodes from Node-RED community for your requirements. Official nodes often start with `node-red-contrib`.

2. Make sure the **Server** of the `mqtt out` node is set to the UNS broker, and topic is a model from **UNS**.

Flow Version Management

Save versions of the data flow, so you can either roll back to an earlier version or start a new flow with a history version.

> **Caution**
> Version rollback is not supported for Edge edition.

Next

- Build Apps on UNS — Build industrial applications with UNS data.
- Analyze UNS Data — Analyze UNS data with Marimo Notebook and Python.

USING TIER0

Manage Data from Center to Edge

Seamlessly manage data on connected edge nodes from a central node.

In a real factory, data is often collected by edge devices and managed from a central node to reduce network bandwidth usage and improve data security. Tier0 provides a fleet management solution to manage data on edge nodes from a central node.

Connecting Edge Nodes to the Center

1. Create fleet nodes on the central node first.

   > **Note**
   > The number of edge nodes that can be connected depends on the license.

2. Use the token generated by the central node to connect each edge node.

Syncing Data between Edge and Center

- UNS data can be selected and synced only from edge nodes to the central node. Once synced, both history and real-time data of the selected topics will be available on the central node.
- From the central node, you can assign users or user groups to edge nodes for node management.

> **Note**
> Set application access permission to user groups on the central node first, so that users in the group can access allowed applications on edge nodes.

Next

- Modeling Factory Data — Learn how to model factory data in UNS.
- Preparing Data Foundation — Model your namespace and connect source data.

USING TIER0

Working with Factory Data

Operate on the UNS data over MQTT and the API — with the API reference.

Data in Tier0 unified namespace can be altered and managed through MQTT and APIs with authorization.

MQTT

MQTT clients can connect to the UNS broker with credentials generated on Tier0.

> **Note**
> You can use MQTT clients such as **MQTTX**, **MQTT Explorer**, and **Node-RED** to connect to the broker and publish/subscribe messages.

Generating MQTT Credentials

On the **Edge** tab in Tier0, create a new credential and make a note of it.

> **Exceptions on Tier0 Edge version**
> The Tier0 Edge version does not require credentials for external connections.

Connecting MQTT Clients

> **Tip**
> - The UNS broker can be connected through both TCP and WSS with the following address:
>   - mqtt://host:1883
>   - mqtts://host:8883
>   - ws://host:8083/mqtt
>   - wss://host:8084/mqtt
> - TCP without TLS is used as an example in the following connections.

TAB: MQTTX

1. Install and open MQTTX.
2. Add a connection with the information of the UNS broker and the generated credentials.

```
{
  "Host": "mqtt.tier0.dev",
  "Port": 1883,
  "Client ID": "<generated client ID>",
  "Username": "<generated username>",
  "Password": "<generated password>"
}
```

TAB: MQTT EXPLORER

1. Install and open MQTT Explorer.
2. Add a new connection with the UNS broker information and the generated credentials.

```
{
  "Protocol": "mqtt://",
  "Host": "mqtt.tier0.dev",
  "Port": 1883,
  "Username": "<generated username>",
  "Password": "<generated password>"
}
```

3. Change the client ID to the credential ID in **ADVANCED** before connecting.

TAB: NODE-RED

- Use **Event Flow** - Data processing module in Tier0 based on Node-RED.
    1. Create an **Event Flow** in **Flows**, and start it with an `mqtt in` node.
    2. Select the UNS broker as **Server** (same name as the flow) and set **Topic** to a UNS model in the node.

- Use **Independent Node-RED Instance**

    1. Install and open the Node-RED instance.
    2. Add a server in the `mqtt in` node with the following information.

```
{
  {/* under Connection */}
  "Server": "mqtt://mqtt.tier0.dev",
  "Port": 1883,
  "Client ID": "<generated clientId>",
  {/* under Security */}
  "Username": "<generated username>",
  "Password": "<generated password>"
}
```

Testing Connections

1. Make sure the client status is normal.
2. Copy a UNS model topic and publish a message to it through the client.

API

Use the **REST API** to operate on UNS data.

> **How to use the API**
> The OpenAPI base path is `/openapi/v1`, For example `https://your-workspace.tier0.app/openapi/v1`, and pass your API key in the `x-api-key` header.

Getting API Key

API keys are from **Settings** > **API Keys**. Depend on different account type, allowed key types are different.

- **Service Key**: Can only be created by **Admin** and **Owner**.

- **Personal Key**: Can be created by anyone, and the permission scope is the same as that of the account.

> ⚠ **Key permisson vs Account permission**
> Account permission trumps key permission. When the key permission is set to **Full Access**, data unavailable to the account still cannot be accessed with this key.

Examples

- `/uns/create`

  Create UNS path or topic. Use `namespace` to define the node tree.

  TAB: JAVASCRIPT

```
const BASE_URL = process.env.TIER0_BASE_URL;
const API_KEY = process.env.TIER0_API_KEY;

const response = await fetch(`${BASE_URL}/uns/create`, {
  method: 'POST',
  headers: {
    'content-type': 'application/json',
    'x-api-key': API_KEY,
  },
  body: JSON.stringify({
    namespace: [
      {
        name: 'DemoFactory',
        type: 'PATH',
        children: [
          {
            name: 'Site_01',
            type: 'PATH',
            children: [
              {
                name: 'Production',
                type: 'PATH',
                children: [
                  {
                    name: 'Produced_Qty',
                    type: 'TOPIC',
                    topicType: 'METRIC',
                    displayName: 'Produced Quantity',
```

```
                    description: 'Produced quantity from the production
line.',
                    enableHistory: true,
                    fields: [
                      { name: 'value', type: 'DOUBLE', unit: 'pcs' },
                    ],
                  },
                ],
              },
            ],
          },
        ],
      },
    ],
  }),
});

const result = await response.json();
console.log(result.data.results);
```

TAB: PYTHON

```
import os
import requests

base_url = os.environ["TIER0_BASE_URL"]
api_key = os.environ["TIER0_API_KEY"]

response = requests.post(
    f"{base_url}/uns/create",
    headers={"x-api-key": api_key},
    json={
        "namespace": [
            {
                "name": "DemoFactory",
                "type": "PATH",
                "children": [
                    {
                        "name": "Site_01",
                        "type": "PATH",
                        "children": [
                            {
                                "name": "Production",
                                "type": "PATH",
                                "children": [
                                    {
                                        "name": "Produced_Qty",
```

```
                                        "type": "TOPIC",
                                        "topicType": "METRIC",
                                        "displayName": "Produced
Quantity",
                                        "description": "Produced quantity
from the production line.",
                                        "enableHistory": True,
                                        "fields": [
                                            {"name": "value", "type":
"DOUBLE", "unit": "pcs"},
                                        ],
                                    },
                                ],
                            },
                        ],
                    },
                ],
            },
        ],
    },
    timeout=30,
)
response.raise_for_status()

result = response.json()
print(result["data"]["results"])
```

TAB: CURL

```
curl -X POST "$TIER0_BASE_URL/uns/create" \
  -H "content-type: application/json" \
  -H "x-api-key: $TIER0_API_KEY" \
  -d '{
    "namespace": [
      {
        "name": "DemoFactory",
        "type": "PATH",
        "children": [
          {
            "name": "Site_01",
            "type": "PATH",
            "children": [
              {
                "name": "Production",
                "type": "PATH",
                "children": [
                  {
```

```
                    "name": "Produced_Qty",
                    "type": "TOPIC",
                    "topicType": "METRIC",
                    "displayName": "Produced Quantity",
                    "description": "Produced quantity from the production
line.",
                    "enableHistory": true,
                    "fields": [
                      { "name": "value", "type": "DOUBLE", "unit": "pcs"
}
                    ]
                  }
                ]
              }
            ]
          }
        ]
      }
    ]
  }'
```

- `/uns/read`

  Read the latest value and metadata for one or more UNS topics.

TAB: JAVASCRIPT

```
const BASE_URL = process.env.TIER0_BASE_URL;
const API_KEY = process.env.TIER0_API_KEY;

const response = await fetch(`${BASE_URL}/uns/read`, {
  method: 'POST',
  headers: {
    'content-type': 'application/json',
    'x-api-key': API_KEY,
  },
  body: JSON.stringify({
    topics: [
      'DemoFactory/Site_01/Production/Produced_Qty',
    ],
    include_leaf_value: true,
    include_metadata: true,
  }),
});

const result = await response.json();
```

```
console.log(result.data.results);
```

TAB: PYTHON

```
import os
import requests

base_url = os.environ["TIER0_BASE_URL"]
api_key = os.environ["TIER0_API_KEY"]

response = requests.post(
    f"{base_url}/uns/read",
    headers={"x-api-key": api_key},
    json={
        "topics": [
            "DemoFactory/Site_01/Production/Produced_Qty",
        ],
        "include_leaf_value": True,
        "include_metadata": True,
    },
    timeout=30,
)
response.raise_for_status()

result = response.json()
print(result["data"]["results"])
```

TAB: CURL

```
curl -X POST "$TIER0_BASE_URL/uns/read" \
  -H "content-type: application/json" \
  -H "x-api-key: $TIER0_API_KEY" \
  -d '{
    "topics": [
      "DemoFactory/Site_01/Production/Produced_Qty"
    ],
    "include_leaf_value": true,
    "include_metadata": true
  }'
```

- `/uns/update`

  Update a UNS node by `path`. Use `updateMask` to specify which fields should change.

TAB: JAVASCRIPT

```
const BASE_URL = process.env.TIER0_BASE_URL;
const API_KEY = process.env.TIER0_API_KEY;

const response = await fetch(`${BASE_URL}/uns/update`, {
  method: 'POST',
  headers: {
    'content-type': 'application/json',
    'x-api-key': API_KEY,
  },
  body: JSON.stringify({
    path: 'DemoFactory/Site_01/Production/Produced_Qty',
    displayName: 'Produced Quantity',
    description: 'Updated production output quantity.',
    updateMask: ['displayName', 'description'],
  }),
});

const result = await response.json();
console.log(result.data);
```

TAB: PYTHON

```
import os
import requests

base_url = os.environ["TIER0_BASE_URL"]
api_key = os.environ["TIER0_API_KEY"]

response = requests.post(
    f"{base_url}/uns/update",
    headers={"x-api-key": api_key},
    json={
        "path": "DemoFactory/Site_01/Production/Produced_Qty",
        "displayName": "Produced Quantity",
        "description": "Updated production output quantity.",
        "updateMask": ["displayName", "description"],
    },
```

```
    timeout=30,
)
response.raise_for_status()

result = response.json()
print(result["data"])
```

TAB: CURL

```
curl -X POST "$TIER0_BASE_URL/uns/update" \
  -H "content-type: application/json" \
  -H "x-api-key: $TIER0_API_KEY" \
  -d '{
    "path": "DemoFactory/Site_01/Production/Produced_Qty",
    "displayName": "Produced Quantity",
    "description": "Updated production output quantity.",
    "updateMask": ["displayName", "description"]
  }'
```

- `/uns/delete`

  Delete one or more UNS models. Use `hard_delete: false` to move models to the recycle bin.

TAB: JAVASCRIPT

```
const BASE_URL = process.env.TIER0_BASE_URL;
const API_KEY = process.env.TIER0_API_KEY;

const response = await fetch(`${BASE_URL}/uns/delete`, {
  method: 'POST',
  headers: {
    'content-type': 'application/json',
    'x-api-key': API_KEY,
  },
  body: JSON.stringify({
    topics: [
      'DemoFactory/Site_01/Production/Produced_Qty',
    ],
    hard_delete: false,
  }),
```

```
});

const result = await response.json();
console.log(result.data);
```

TAB: PYTHON

```
import os
import requests

base_url = os.environ["TIER0_BASE_URL"]
api_key = os.environ["TIER0_API_KEY"]

response = requests.post(
    f"{base_url}/uns/delete",
    headers={"x-api-key": api_key},
    json={
        "topics": [
            "DemoFactory/Site_01/Production/Produced_Qty",
        ],
        "hard_delete": False,
    },
    timeout=30,
)
response.raise_for_status()

result = response.json()
print(result["data"])
```

TAB: CURL

```
curl -X POST "$TIER0_BASE_URL/uns/delete" \
  -H "content-type: application/json" \
  -H "x-api-key: $TIER0_API_KEY" \
  -d '{
    "topics": [
      "DemoFactory/Site_01/Production/Produced_Qty"
    ],
    "hard_delete": false
  }'
```

API reference

- Tier0 OpenAPI Reference

> ⚠ **API key limitations on Tier0 Edge**
> API key obtained from Tier0 Edge is only valid for UNS and Flows APIs.

Next

- Build Apps on UNS — Build industrial applications with UNS data.
- Analyze Data — Analyze UNS data with Marimo Notebook and Python.

USING TIER0

Building Apps

A container is an app — deploy full containers on Tier0 Edge and Enterprise; build logic with Event Flow.

Build customized industrial applications through AI conversations. Applications can use factory data and include built-in capabilities such as role-based access control, version management, and external deployment.

Building a Tier0 App

Create a project before building an application.

- Project is used to categorize applications and manage application access control.
- Applications in one project can work as related functional modules of a complex system.
- Data in each project is independent from other projects.

Writing Prompt

Make sure your prompt covers ***at least*** the following aspects for the agent to generate an application that better fits your requirements.

- **Application Purpose**: What is the purpose of the application?

```
Create an equipment maintenance application for factory technicians.
```

- **Application Data**: What data does the application use and where is it?

TAB: PLATFORM PLAN

Applications can consume data from and publish data to UNS by stating the UNS topic in the prompt.

```
The application should use equipment status, alarm records, and
maintenance history from the xxx UNS model, and publish updates to
```

```
the xxx UNS topic.
```

TAB: BUILDER PLAN

The application does not connect to UNS. Data generated during application usage is stored in the database.

- **Application Function**: What does the application do?

```
The application should allow users to monitor equipment conditions,
submit maintenance requests, and track repair progress.
```

> ⚠ **When you are on Builder**
> The **Builder** plan does not currently support UNS, but simulated data is available. For more specific data integration needs, please contact our team at **sales@freezonex.io**.

> 🚀 **To build applications faster**
> - Templates of common industrial applications are provided in **Library**. You can use a template and change features on top to build an application much faster.
> - Use the **embedded Builder skills** to help build applications.
> - Use `/clear` to clear context and `/compact` to compress context when the conversation is too long.

Access Control

Role-based access control is applied to both projects and applications.

- Project access levels
  - **Owner**: Full access to the project.
  - **Can Edit**: Full access to the project except deleting it.
  - **Can Launch**: Can only see the project in **Launchpad** and use allowed applications inside.
- Application roles

- Automatically generated by agent based on business logic of the application when not explicitly defined. Existing roles from other applications may be reused when their responsibilities align.
- Can only be assigned to project members.
- The application is open to all project members if the prompt explicitly specifies that no roles are required.

> **If you want to assign your own roles**
> State the roles and permission scope in the prompt, and the agent will compare them with existing roles generated from other applications. If there is a conflict, the agent will confirm with you.

> **Application role assignment is a MUST for role-controlled applications**
> To access a role-controlled application in **Launchpad**, you must have application roles assigned. Even a workspace/project owner cannot access the application without an application role.

Version Management

Applications exist on Tier0 in 3 shapes.

- **Draft**: The latest state of the application. Changes on this version will not affect other versions if not deployed.
- **Snapshot**: Saves the application at the time of taking the snapshot. Deploying a snapshot version takes down the current **Active** version.
- **Active**: The current online version of the application.

> **Version switch**
> - Deploy a draft, the active application updates to a newer version.
> - Deploy a snapshot, the active application rolls back to an earlier version.

Export to External

Once the application is ready, you can export it from the **Preview** page for external third-party environments or Tier0 Enterprise.

- **Download Source Code**: Export source code of the application for secondary development.
- **Export Bundle**: Specifically exported for local import on **Tier0 Enterprise**.

> **Download the source code of a snapshot**
> Download a snapshot after saving it to get the source code of that snapshot version.

Next

- Analyze Data — Analyze UNS data with Marimo Notebook and Python.
- Best practice: connecting industrial protocols - Examples of common data source connections.

USING TIER0

Building Digital Twins

Bind live UNS data to 3D models and assemble a complete factory scene for digital-twin visualization.

Factory data connected to Tier0 can be visualized as digital twin with life-like motions and environment. The process goes from mapping data to 3D models to combining models together to build a complete factory scene.

> ⓘ **Looking for 2D dashboards?**
> This page covers **3D digital-twin visualization** — mapping UNS data to 3D models and composing factory scenes. For **2D analytics dashboards** (charts, tables, KPI panels) built with Notebook, see **Building Analytics Apps**.

Mapping Data to 3D Models

1. Select nodes to map with UNS data for real-time motions after uploading a model in **Model**.
2. Add instances with different UNS data onto the model to simulate different machines.
3. Bind data to the model, and use **Preview** to check whether it moves and rotates as the data changes.
   - **Position**: The model moves along the mapped direction of the coordinate.
   - **Rotation**: The model rotates around the coordinate original point in the mapped direction.
4. Use **Data Tagging** to add tags for static information display.

Visualizing Data

1. Add instances that should exist in the scene from **Instance Library**.
2. Use tools at the top to move, scale, and rotate models to fit the scene, and to change the viewing perspective and coordinate position.
3. Model position can be confirmed by **Object** at the right corner, which displays the bound UNS data.

4 Viewport and lighting are for environment details.

External Share

Only instances can be shared either by model or by scene.

- **Share Model**: Generates a Tier0 platform link where the model with live UNS data is displayed.
- **Share Scene**: Generates a Tier0 platform link where the model and its scene, with live UNS data and environment settings, are displayed.

USING TIER0

Analyzing Data

Query live namespace data in Tier0 Notebook and build interactive analyses and machine-learning apps. Available in Cloud and Enterprise.

Marimo Notebook is used in Tier0 for advanced data analysis with **Python** scripts based on the factory data modeled and connected to UNS.

> **Know before you start**
> Data connected to UNS models with **Enable History** selected is available as an existing data source in **Notebook**.

1. UNS data is under **VARIABLES** > **PostgreSQL** > **tenant_id** > **uns**.
2. Use **@** to include data sources when you **GENERATE WITH AI**, and fuzzy match is available for data sources searching.
3. Save versions of your analysis for future rollbacks.

> **AI assist in Notebook**
> - AI assist is available in Tier0 Cloud.
> - You need to provide key for AI assist in Tier0 Enterprise.

Next

- Build Apps on UNS — Build industrial applications with UNS data.
- UNS Concepts — Understand data modeling in Unified Namespace.

USING TIER0

Operating on Tier0 with Agents

Use AI agents to operate Tier0 through Skills and CLI commands.

Tier0 provides Skills and CLI commands that allow AI agents to operate on the platform.

Work on Tier0 with your preferred AI agent.

Claude Code Codex Gemini CLI Cursor GitHub Copilot

Installing Skills and CLI

Choose the installer that matches your environment.

With Node.js

If Node.js 16 or later is available, use `npx` to install the CLI and skills.

```
npx -y @tier0/cli@latest install
```

This runs the full installer and does two things:

- Installs the `tier0` CLI binary into `~/.tier0/bin/`.
- Installs the embedded Tier0 Skill baseline and synchronizes it to detected agents such as Codex, Claude Code, and more.

Without Node.js

1 Install the `tier0` CLI binary.

TAB: CURL

```
curl -fsSL https://raw.githubusercontent.com/FREEZONEX/Tier0-cli/main/install.sh | bash
```

TAB: IWR

```
iwr https://raw.githubusercontent.com/FREEZONEX/Tier0-cli/main/install.ps1 | iex
```

2 Download the latest Tier0 Skills and synchronize them to detected agents.

```
tier0 skills update
```

3 Verify the installed Skill version and health.

```
tier0 skills status
```

Managing Skills

- Check the installed Skill version and health:

```
tier0 skills status
```

- Repair a missing or damaged embedded Skill baseline:

```
tier0 skills install
```

- Update to the latest Tier0 Skill:

```
tier0 skills update
```

How Skills and CLI Work Together

Tier0 CLI provides executable commands, and Skills guide agents in choosing and safely using these commands.

> **Note**
> Tier0 Skills are designed to be automatically triggered by prompts and guide agents what to do and which commands to use.

Example

- Prompt: Check the current temperature of `Plant/Line1/Metric/Temperature`.
- Skill
  - `tier0-uns`
  - It tells the agent that a current value should be read from a full topic path, and selects the appropriate UNS read operation.
- CLI: `tier0 uns read Plant/Line1/Metric/Temperature --json`

Skills

Skill	Purpose
`tier0`	Routes Tier0 tasks to the appropriate skill and provides shared setup, authentication, and safety rules.
`tier0-uns`	Guides agents through UNS operations, including browse, read, write, history, search, create, update, delete, and restore.
`tier0-flow`	Guides agents through SourceFlow and EventFlow management, including

Skill	Purpose
	inspection, export, creation, update, deployment, and deletion.
`tier0-mqtt`	Guides agents through MQTT credential management, publishing, and bounded real-time subscriptions.
`tier0-files`	Guides agents through Tier0 file operations, including upload, download, access URL generation, and deletion.

> **Use agent to read the skill repo**
> Ask your agent to read the **Tier0 Skill repo** for details.

CLI Operations

Setting Operating Instance

> **Note**
> Cloud uses `https://tier0.dev` by default and needs no base URL configuration.

Private deployments must configure the base URL before login. Once it's configured, all the following commands will be executed on that instance.

```
tier0 config --base-url https://your-tier0.example.com
tier0 login # interactive browser login
```

Authentication

TAB: WITH AN API KEY

1. Set the API key you got from the operating Tier0 instance.

```
tier0 config --api-key <api-key>
```

2. Verify the API key identity and permissions.

```
tier0 auth whoami --json
```

3. Perform other operations with the `tier0` CLI.

TAB: WITHOUT AN API KEY

1. Get the browser authentication link.

```
tier0 login --no-wait --json
```

2. Go through authentication on the browser by accessing the returned `verification_url`.
3. Poll for the login result with the returned `setup_code`.

```
tier0 login --setup-code <setup_code>
```

> ⚠ **Important**
> Run `config` to set the operating instance before configuring an API key or starting browser login; otherwise the CLI may operate on the wrong instance.

Previewing Operations

Use `--dry-run` to validate and preview supported write operations without sending a request to Tier0.

Supported Operations

- UNS: `write/create/update/delete/restore`
- Flow: `create/update/delete/deploy`
- API: `tier0 api`
- MQTT: `auth create/auth delete/publish`
- Files: `delete`

Examples

```
tier0 uns write --topic demo --value '{"value":1}' --dry-run
tier0 flow deploy --id 1 --flows-file flows.json --dry-run --json
tier0 api /openapi/v1/uns/write --body-file body.json --dry-run --json
tier0 mqtt auth create --name my-client --save my-client --dry-run --json
tier0 assets delete --file-path workspace/path/to/file.csv --dry-run --json
```

> ⓘ **Note**
> The `--json` option outputs the request in JSON format.

> ⚠ **Request body must contain valid JSON**
> The CLI no longer guesses at or rewrites malformed JSON if no valid JSON is provided.

Returned Previews

> ⓘ **Note**
> No headers and API keys are revealed in the dry-run output.

```
{
  "ok": true,
  "dry_run": true,
  "data": {
    "api": [
      {"method": "POST", "url": "https://tier0.dev/openapi/v1/uns/write", "body":
{}}
    ]
```

```
  }
}
```

Handling Errors

With `--json`, command failures return a JSON object with the error message and code.

```
{"ok":false,"error":
{"type":"validation","subtype":"invalid_argument","param":"--qos","message":"-
-qos must be 0, 1, or 2"}}
```

Uninstalling Skills and CLI

- Uninstallation Commands

```
# Remove the Tier0 CLI while keeping local configuration and skills.
npx @tier0/cli@latest uninstall

# Remove the Tier0 CLI and delete local configuration, credentials and cache, but
keep skills.
npx @tier0/cli@latest uninstall --purge

# Remove skills
npx @tier0/cli@latest uninstall --remove-skills

# Remove all of the above
npx @tier0/cli@latest uninstall --purge --remove-skills
```

- Global NPM Uninstallation

```
npm uninstall -g @tier0/cli
```

- Manual Cleanup

```

```

```
rm -rf ~/.tier0/bin/tier0 ~/.tier0/skills
npx -y --package=skills -- skills remove tier0 -y -g
```

CLI Reference

Tier0 CLI Reference lists all CLI commands.

Next

- Working with UNS Data — How to operate on your UNS model through MQTT and APIs.

Best Practice

Modeling Factory Data

The Demo UNS design: avoiding topic explosion and payload structure best practices.

Data in your factory, either machine data sending from sensors, or order data exiting in ERP/MES, can all be modeled into understandable tree structure in Tier0.

How the Demo UNS is Structured

What needs to be done first

Before modeling data, answer the following questions.

- What data will be consumed from the UNS?
- What data will be published back to the UNS?

Physical Data

Industrial data from machines and equipment with physical locations is modeled based on the ISA-95 standard.

Demo Model

TAB: ISA-95 HIERARCHY

```
Enterprise (Level 4)
└── Site (Level 3/4)
    └── Area (Level 3)
        └── Work Center / Process Cell (Level 2/3)
            └── Equipment / Unit (Level 1/2)
                └── Control Module / Device (Level 0/1)
```

TAB: TREE STRUCTURE

```
DemoFactory
└── Site_01
    └── Production
        └── Line_01
            └── WorkOrderExecution
                ├── Metric
                │   ├── Target_Qty
                │   ├── Produced_Qty
                │   ├── Defect_Qty
                │   └── Completion_Rate
                └── State
                    ├── CurrentWorkOrder
                    └── WorkOrderStatus
```

TAB: JSON


```
{
  "namespace": [
    {
      "name": "DemoFactory",
      "alias": "DemoFactory_9e50bbcd833d",
      "type": "PATH",
      "children": [
        {
          "name": "Site_01",
          "alias": "DemoFactory__6d1fcad14e86",
          "type": "PATH",
          "children": [
            {
              "name": "Production",
              "alias": "__Site_01_Pro_91523d2ad304",
              "type": "PATH",
              "children": [
                {
                  "name": "Line_01",
                  "alias": "__Production__acd5841c1ada",
                  "type": "PATH",
                  "children": [
                    {
                      "name": "WorkOrderExecution",
                      "alias": "__Line_01_Wor_54904ff56e91",
                      "type": "PATH",
                      "children": [
                        {
                          "name": "Metric",
                          "alias": "metric_233fd84305c4",
```

```
                        "type": "PATH",
                        "children": [
                          {
                            "name": "Target_Qty",
                            "alias": "Target_Qty_8646b092e543",
                            "type": "TOPIC",
                            "topicType": "METRIC",
                            "description": "Target quantity for the current
Work Order.",
                            "fields": [
                              {
                                "name": "production_order_id",
                                "type": "STRING"
                              },
                              {
                                "name": "plan_id",
                                "type": "STRING"
                              },
                              {
                                "name": "plan_number",
                                "type": "STRING"
                              },
                              {
                                "name": "work_order_id",
                                "type": "STRING"
                              },
                              {
                                "name": "value",
                                "type": "DOUBLE"
                              },
                              {
                                "name": "unit",
                                "type": "STRING"
                              },
                              {
                                "name": "timestamp",
                                "type": "DATETIME"
                              }
                            ],
                            "enableHistory": "TRUE",
                            "mockData": "FALSE"
                          },
                          {
                            "name": "Produced_Qty",
                            "alias": "Produced_Qty_1f8cd3e11f77",
                            "type": "TOPIC",
                            "topicType": "METRIC",
                            "description": "Produced quantity for the
current Work Order.",
                            "fields": [
                              {
```

```
            "name": "production_order_id",
            "type": "STRING"
          },
          {
            "name": "plan_id",
            "type": "STRING"
          },
          {
            "name": "plan_number",
            "type": "STRING"
          },
          {
            "name": "work_order_id",
            "type": "STRING"
          },
          {
            "name": "value",
            "type": "DOUBLE"
          },
          {
            "name": "unit",
            "type": "STRING"
          },
          {
            "name": "timestamp",
            "type": "DATETIME"
          }
        ],
        "enableHistory": "TRUE",
        "mockData": "FALSE"
      },
      {
        "name": "Defect_Qty",
        "alias": "Defect_Qty_b48fb3563ec1",
        "type": "TOPIC",
        "topicType": "METRIC",
        "description": "Defect quantity for the current
Work Order.",
        "fields": [
          {
            "name": "production_order_id",
            "type": "STRING"
          },
          {
            "name": "plan_id",
            "type": "STRING"
          },
          {
            "name": "plan_number",
            "type": "STRING"
          },
```

```
                {
                  "name": "work_order_id",
                  "type": "STRING"
                },
                {
                  "name": "value",
                  "type": "DOUBLE"
                },
                {
                  "name": "unit",
                  "type": "STRING"
                },
                {
                  "name": "timestamp",
                  "type": "DATETIME"
                }
              ],
              "enableHistory": "TRUE",
              "mockData": "FALSE"
            },
            {
              "name": "Completion_Rate",
              "alias": "Completion_R_7294e38e1530",
              "type": "TOPIC",
              "topicType": "METRIC",
              "description": "Completion percentage for the
current Work Order.",
              "fields": [
                {
                  "name": "production_order_id",
                  "type": "STRING"
                },
                {
                  "name": "plan_id",
                  "type": "STRING"
                },
                {
                  "name": "plan_number",
                  "type": "STRING"
                },
                {
                  "name": "work_order_id",
                  "type": "STRING"
                },
                {
                  "name": "value",
                  "type": "DOUBLE"
                },
                {
                  "name": "unit",
                  "type": "STRING"
```

```
                    },
                    {
                      "name": "timestamp",
                      "type": "DATETIME"
                    }
                  ],
                  "enableHistory": "TRUE",
                  "mockData": "FALSE"
                }
              ]
            },
            {
              "name": "State",
              "alias": "state_539a0f331fe0",
              "type": "PATH",
              "children": [
                {
                  "name": "CurrentWorkOrder",
                  "alias": "CurrentWorkO_a638320784dd",
                  "type": "TOPIC",
                  "topicType": "STATE",
                  "description": "Current active Work Order on
Line_01.",
                  "fields": [
                    {
                      "name": "production_order_id",
                      "type": "STRING"
                    },
                    {
                      "name": "plan_id",
                      "type": "STRING"
                    },
                    {
                      "name": "plan_number",
                      "type": "STRING"
                    },
                    {
                      "name": "work_order_id",
                      "type": "STRING"
                    },
                    {
                      "name": "target_line",
                      "type": "STRING"
                    },
                    {
                      "name": "workstation",
                      "type": "STRING"
                    },
                    {
                      "name": "product_code",
                      "type": "STRING"
```

```
        },
        {
          "name": "target_qty",
          "type": "DOUBLE"
        },
        {
          "name": "unit",
          "type": "STRING"
        },
        {
          "name": "started_at",
          "type": "DATETIME"
        },
        {
          "name": "updated_at",
          "type": "DATETIME"
        }
      ],
      "enableHistory": "TRUE",
      "mockData": "FALSE"
    },
    {
      "name": "WorkOrderStatus",
      "alias": "WorkOrderSta_e24e3bf113d1",
      "type": "TOPIC",
      "topicType": "STATE",
      "description": "Execution status of the current
Work Order.",
      "fields": [
        {
          "name": "production_order_id",
          "type": "STRING"
        },
        {
          "name": "plan_id",
          "type": "STRING"
        },
        {
          "name": "plan_number",
          "type": "STRING"
        },
        {
          "name": "work_order_id",
          "type": "STRING"
        },
        {
          "name": "status",
          "type": "STRING"
        },
        {
          "name": "reason",
```

```
                    "type": "STRING"
                  },
                  {
                    "name": "updated_at",
                    "type": "DATETIME"
                  }
                ],
                "enableHistory": "TRUE",
                "mockData": "FALSE"
              }
            ]
          }
        ]
      }
    ]
  }
]
}
```

Business Data

For business planning and process objects, such as production orders and work order plans in an ERP, model data according to its business context.

> **How to recognize business data**
> Data that does not have a physical location.

Business Data Modeling Guide

```
Factory / Business Domain / Business Object / Topic Type / Topic
```

- **Business Domain**: The business process or system the data belongs to.

```
ERP
MES
WMS
QMS
EAM
LIMS
Planning
Inventory
Maintenance
Quality
Logistics
...
```

- **Business Object**: The core managed object in the business domain.

```
ProductionOrders
WorkOrderPlan
MaterialInventory
QualityInspection
MaintenanceOrders
PurchaseOrders
Shipments
...
```

- **Topic Type**: **Metric**, **Action**, and **State**.

Topic Type	Used for	Examples
STATE	Current status or current facts	Work order status, order status, equipment status
METRIC	Continuously changing numeric values	Temperature, speed, output, OEE, energy consumption
ACTION	Commands for external executors	Start equipment, stop equipment, reset alarm, request work order dispatch

Demo Model

TAB: TREE STRUCTURE

```
DemoFactory
└── ERP
    ├── ProductionOrders
    │   └── State
    │       ├── UpsertProductionOrder
    │       └── OrderList
    └── WorkOrderPlan
        ├── Metric
        │   └── SplitCount
        └── State
            ├── PlanStatus
            └── WorkOrderList
```

TAB: JSON


```
{
  "namespace": [
    {
      "name": "DemoFactory",
      "alias": "DemoFactory_f925c1ec5041",
      "type": "PATH",
      "children": [
        {
          "name": "ERP",
          "alias": "DemoFactory__e504805b1973",
          "type": "PATH",
          "children": [
            {
              "name": "ProductionOrders",
              "alias": "__ERP_Product_1e7984ac3a1d",
              "type": "PATH",
              "children": [
                {
                  "name": "State",
                  "alias": "state_c2616c8698fa",
                  "type": "PATH",
                  "children": [
                    {
                      "name": "UpsertProductionOrder",
                      "alias": "UpsertProduc_d5f166da35c6",
                      "type": "TOPIC",
```

```
                    "topicType": "STATE",
                    "description": "Event for creating or updating a
Production Order from WOM or ERP simulation.",
                    "fields": [
                      {
                        "name": "event_id",
                        "type": "STRING"
                      },
                      {
                        "name": "event_type",
                        "type": "STRING"
                      },
                      {
                        "name": "source",
                        "type": "STRING"
                      },
                      {
                        "name": "production_order_id",
                        "type": "STRING"
                      },
                      {
                        "name": "order_number",
                        "type": "STRING"
                      },
                      {
                        "name": "product_code",
                        "type": "STRING"
                      },
                      {
                        "name": "planned_qty",
                        "type": "DOUBLE"
                      },
                      {
                        "name": "unit",
                        "type": "STRING"
                      },
                      {
                        "name": "status",
                        "type": "STRING"
                      },
                      {
                        "name": "due_date",
                        "type": "DATETIME"
                      },
                      {
                        "name": "updated_at",
                        "type": "DATETIME"
                      }
                    ],
                    "enableHistory": "TRUE",
                    "mockData": "FALSE"
```

```
                    },
                    {
                      "name": "OrderList",
                      "alias": "OrderList_11fbca51a2bd",
                      "type": "TOPIC",
                      "topicType": "STATE",
                      "description": "Retained compatibility snapshot of
current Production Orders.",
                      "fields": [
                        {
                          "name": "orders",
                          "type": "STRING"
                        },
                        {
                          "name": "updated_at",
                          "type": "DATETIME"
                        }
                      ],
                      "enableHistory": "TRUE",
                      "mockData": "FALSE"
                    }
                  ]
                }
              ]
            },
            {
              "name": "WorkOrderPlan",
              "alias": "__ERP_WorkOrd_19f04960daf5",
              "type": "PATH",
              "children": [
                {
                  "name": "Metric",
                  "alias": "metric_17258b809a4f",
                  "type": "PATH",
                  "children": [
                    {
                      "name": "SplitCount",
                      "alias": "SplitCount_032e86f5fbfc",
                      "type": "TOPIC",
                      "topicType": "METRIC",
                      "description": "Number of executable Work Orders
generated from a Work Order Plan.",
                      "fields": [
                        {
                          "name": "production_order_id",
                          "type": "STRING"
                        },
                        {
                          "name": "plan_id",
                          "type": "STRING"
                        },
```

```
                    {
                      "name": "plan_number",
                      "type": "STRING"
                    },
                    {
                      "name": "split_count",
                      "type": "INTEGER"
                    },
                    {
                      "name": "updated_at",
                      "type": "DATETIME"
                    }
                  ],
                  "enableHistory": "TRUE",
                  "mockData": "FALSE"
                }
              ]
            },
            {
              "name": "State",
              "alias": "state_884cee548bf7",
              "type": "PATH",
              "children": [
                {
                  "name": "PlanStatus",
                  "alias": "PlanStatus_2e21c1327c49",
                  "type": "TOPIC",
                  "topicType": "STATE",
                  "description": "Current lifecycle status of a Work Order
Plan.",
                  "fields": [
                    {
                      "name": "production_order_id",
                      "type": "STRING"
                    },
                    {
                      "name": "plan_id",
                      "type": "STRING"
                    },
                    {
                      "name": "plan_number",
                      "type": "STRING"
                    },
                    {
                      "name": "status",
                      "type": "STRING"
                    },
                    {
                      "name": "updated_at",
                      "type": "DATETIME"
                    }
```

```
                    ],
                    "enableHistory": "TRUE",
                    "mockData": "FALSE"
                  },
                  {
                    "name": "WorkOrderList",
                    "alias": "WorkOrderLis_887b92f26fc2",
                    "type": "TOPIC",
                    "topicType": "STATE",
                    "description": "Split result list generated from a sent
Work Order Plan.",
                    "fields": [
                      {
                        "name": "production_order_id",
                        "type": "STRING"
                      },
                      {
                        "name": "plan_id",
                        "type": "STRING"
                      },
                      {
                        "name": "plan_number",
                        "type": "STRING"
                      },
                      {
                        "name": "split_count",
                        "type": "INTEGER"
                      },
                      {
                        "name": "lines",
                        "type": "STRING"
                      },
                      {
                        "name": "updated_at",
                        "type": "DATETIME"
                      }
                    ],
                    "enableHistory": "TRUE",
                    "mockData": "FALSE"
                  }
                ]
              }
            ]
          }
        ]
      }
    ]
}
```

Payload Structure

Data to be consumed and written back must both be considered when structuring the topic payload.

- **Action**

  Action payloads automatically include `_timestamp`, which records when the data is stored in the Tier0 database.

  TAB: START_JOB

  ```
  {
    "light_turn_on": true,
    "height_adjust": 12.5,
    "_timestamp": "2026-07-10T10:00:00Z"
  }
  ```

  TAB: STOP_JOB

  ```
  {
    "water_pump_off": true,
    "window_close": true,
    "_timestamp": "2026-07-10T10:00:00Z"
  }
  ```

- **Metric**

  Metric payloads automatically include `_timestamp`, which records when the data is stored in the Tier0 database, and `_quality`, which represents data quality.

  TAB: ERP / SPLITCOUNT

  ```
  {
    "production_order_id": "PO-20260710-001",
    "plan_id": "PLAN-001",
  ```

```
  "plan_number": "WOP-20260710-001",
  "split_count": 3,
  "updated_at": "2026-07-10T10:00:00Z",
  "_timestamp": "2026-07-10T10:00:00Z",
  "_quality": "Good"
}
```


TAB: SITE_01 / TARGET_QTY


```
{
  "production_order_id": "PO-20260710-001",
  "plan_id": "PLAN-001",
  "plan_number": "WOP-20260710-001",
  "work_order_id": "WO-001",
  "value": 1200,
  "unit": "pcs",
  "timestamp": "2026-07-10T10:00:00Z",
  "_timestamp": "2026-07-10T10:00:00Z",
  "_quality": "Good"
}
```


- **State**

  State payloads automatically include `_timestamp`, which records when the data is stored in the Tier0 database.

TAB: ERP / UPSERTPRODUCTIONORDER


```
{
  "event_id": "EVT-001",
  "event_type": "UPSERT",
  "source": "ERP",
  "production_order_id": "PO-20260710-001",
  "order_number": "PO-001",
  "product_code": "SKU-1001",
  "planned_qty": 1200,
  "unit": "pcs",
  "status": "Released",
  "due_date": "2026-07-12T00:00:00Z",
  "updated_at": "2026-07-10T10:00:00Z",
  "_timestamp": "2026-07-10T10:00:00Z"
```

```
}
```

TAB: SITE_01 / WORKORDERSTATUS

```
{
  "production_order_id": "PO-20260710-001",
  "plan_id": "PLAN-001",
  "plan_number": "WOP-20260710-001",
  "work_order_id": "WO-001",
  "status": "Running",
  "reason": "In production",
  "updated_at": "2026-07-10T10:00:00Z",
  "_timestamp": "2026-07-10T10:00:00Z"
}
```

How to Avoid Topic Explosion

Tier0 uses three fixed topic types, `Metric`, `Action`, and `State`, to avoid topic explosion.

- **Use `Metric` for high-frequency real-time data**

  Put high-frequency measurements and telemetry under `Metric` instead of expanding the namespace for every signal detail.

TAB: BAD

```
Factory_A
└── SMT_Line_1
    └── Machine_001
        ├── Temperature_Value
        ├── Temperature_Unit
        ├── Temperature_Status
        └── Speed_Value
```

TAB: BETTER

```
Factory_A
└── SMT_Line_1
    └── Metric
        └── Machine_001
```

Topic payload:

```
{
  "temperature": 80,
  "temperature_unit": "Celsius",
  "temperature_status": "Normal",
  "speed": 300
}
```

- **Use `State` for low-frequency status data**

  Put low-frequency status, snapshots, and current business state under `State`. Avoid creating a new topic for every changing entity such as jobs, batches, or orders.

  TAB: BAD

  ```
  Factory_A
  └── SMT_Line_1
      └── State
          ├── Job_001
          ├── Job_002
          └── Job_003
  ```

  TAB: BETTER

  ```
  Factory_A
  └── SMT_Line_1
      └── State
          └── Machine_001
  ```

  Topic payload:

```
{
  "current_job": "Job_003",
  "product": "Product_A"
}
```

- **Use `Action` for low-frequency action data**

  Put commands, requests, and operator actions under `Action`. Keep action inputs as fields on a topic rather than creating one topic per parameter.

TAB: BAD

```
Factory_A
└── SMT_Line_1
    └── Machine_001
        ├── Start_Command
        ├── Target_Height
        └── Light_On
```

TAB: BETTER

```
Factory_A
└── SMT_Line_1
    └── Action
        └── StartJob
```

Topic payload:

```
{
  "target_height": 12.5,
  "light_on": true,
  "requested_by": "operator"
}
```

BEST PRACTICE

Connecting Industrial Protocols

Connect OPC UA, Modbus, and other typical protocol devices into the UNS with Node-RED — with real flow JSON references.

This section uses common data source types, including OPC UA, Modbus, and APIs, as examples to demonstrate how to connect data sources to **UNS** through **Source Flow**.

OPC UA

Configuring Data Flow

1. In a new **Source Flow**, add `inject`, `OPC UA Client` and `mqtt out` nodes.
2. Connect them in order and configure their details.

- `inject` Add a property entry and set the properties to the following items:

```
{
  "msg.payload": "[
                    {
                        "name": "D103_CHLORIDE",
                        "nodeId": "ns=2;i=3",
                        "datatype": "Float"
                    },
                    {
                        "name": "D103_PH",
                        "nodeId": "ns=2;i=4",
                        "datatype": "Float"
                    },
                    ...
                ]",
  "msg.topic": "multiple"
}
```

- `OPC UA Client`

```
"Endpoint": "<OPC UA server address and port>", //e.g.
opc.tcp://127.0.0.1:4850
"Action": "SUBSCRIBE",
```

```
"Interval": "<data collecting interval>" //e.g. 10 seconds
```

- `mqtt out`

```
"Server": "<embedded UNS broker>", //same name as the flow
"Topic": "<UNS model>",
```

Flow Reference

Show OPC UA flow JSON


```
[
    {
        "id": "fn_mapper_01",
        "type": "function",
        "z": "b002a07d1163c3de",
        "name": "ISA95 Mapping (Air Compressor)",
        "func": "// OPC UA NodeId → ISA-95 MQTT Topic Mapping\n\nconst mapping
= {\n    \"ns=2;i=2\": {\n        topic:
\"global_plant/Smart_Manufacturing_Group/Suzhou_Plant/Utility_Area/Compressed_
Air_System/Air_Compressor_Station/Air_Compressor_01/Metric/outlet_temperature\
",\n        field: \"temperature\"\n    },\n    \"ns=2;i=3\": {\n
topic:
\"global_plant/Smart_Manufacturing_Group/Suzhou_Plant/Utility_Area/Compressed_
Air_System/Air_Compressor_Station/Air_Compressor_01/Metric/discharge_pressure\
",\n        field: \"pressure\"\n    },\n    \"ns=2;i=4\": {\n        topic:
\"global_plant/Smart_Manufacturing_Group/Suzhou_Plant/Utility_Area/Compressed_
Air_System/Air_Compressor_Station/Air_Compressor_01/Metric/vibration_rms\",\n
field: \"vibration\"\n    }\n};\n\nlet value = msg.payload;\n\n// OPC UA
structure normalize\nif (value && typeof value === \"object\") {\n    if
(value.value?.value !== undefined) {\n        value = value.value.value;\n
} else if (value.value !== undefined) {\n        value = value.value;\n
}\n}\n\nconst m = mapping[msg.topic];\nif (!m) return null;\n\nmsg.topic =
m.topic;\n\nmsg.payload = {\n    [m.field]: Number(value)\n};\n\nreturn msg;",
        "outputs": 1,
        "timeout": "",
        "noerr": 0,
        "initialize": "",
        "finalize": "",
        "libs": [],
        "x": 340,
        "y": 320,
        "wires": [
            [
```

```
                "mqtt_out_01"
            ]
        ]
    },
    {
        "id": "mqtt_out_01",
        "type": "mqtt out",
        "z": "b002a07d1163c3de",
        "name": "MQTT Air Compressor Metrics",
        "topic": "",
        "qos": "0",
        "retain": "false",
        "respTopic": "",
        "contentType": "",
        "userProps": "",
        "correl": "",
        "expiry": "",
        "broker": "broker-djnwpg00y370",
        "x": 570,
        "y": 200,
        "wires": []
    },
    {
        "id": "1f242deefa9fdba9",
        "type": "OpcUa-Client",
        "z": "b002a07d1163c3de",
        "endpoint": "298ef5d50e9dc411",
        "action": "subscribe",
        "deadbandtype": "a",
        "deadbandvalue": 1,
        "time": 10,
        "timeUnit": "s",
        "certificate": "n",
        "localfile": "",
        "localkeyfile": "",
        "securitymode": "None",
        "securitypolicy": "None",
        "useTransport": false,
        "maxChunkCount": 1,
        "maxMessageSize": 8192,
        "receiveBufferSize": 8192,
        "sendBufferSize": 8192,
        "setstatusandtime": false,
        "keepsessionalive": false,
        "name": "",
        "x": 220,
        "y": 220,
        "wires": [
            [
                "fn_mapper_01"
            ],
```

```
            [],
            []
        ]
    },
    {
        "id": "7be5f021ed1f10ef",
        "type": "inject",
        "z": "b002a07d1163c3de",
        "name": "",
        "props": [
            {
                "p": "payload"
            },
            {
                "p": "topic",
                "vt": "str"
            }
        ],
        "repeat": "",
        "crontab": "",
        "once": false,
        "onceDelay": 0.1,
        "topic": "multiple",
        "payload": "[{\"nodeId\":\"ns=2;i=2\",\"datatype\":\"Float\"},
{\"nodeId\":\"ns=2;i=3\",\"datatype\":\"Float\"},
{\"nodeId\":\"ns=2;i=4\",\"datatype\":\"Float\"}]",
        "payloadType": "json",
        "x": 110,
        "y": 160,
        "wires": [
            [
                "1f242deefa9fdba9"
            ]
        ]
    },
    {
        "id": "broker-djnwpg00y370",
        "type": "mqtt-broker",
        "z": "b002a07d1163c3de",
        "name": "emqx:1883",
        "broker": "emqx",
        "port": "1883",
        "clientid": "355424069147520",
        "usetls": false,
        "protocolVersion": "4",
        "keepalive": "60",
        "cleansession": true,
        "birthTopic": "",
        "birthQos": "0",
        "birthPayload": "",
        "closeTopic": "",
```

```
        "closeQos": "0",
        "closePayload": "",
        "willTopic": "",
        "willQos": "0",
        "willPayload": ""
    },
    {
        "id": "298ef5d50e9dc411",
        "type": "OpcUa-Endpoint",
        "endpoint": "opc.tcp://172.31.151.237:4841",
        "secpol": "None",
        "secmode": "None",
        "none": true,
        "login": false,
        "usercert": false,
        "usercertificate": "",
        "userprivatekey": ""
    },
    {
        "id": "a003de8af8e62c75",
        "type": "global-config",
        "env": [],
        "modules": {
            "node-red-contrib-opcua": "0.2.339"
        }
    }
]
```


Modbus

Configuring Data Flow

In the flow, add `modbus-read` and `mqtt out` nodes and configure the required information.

- `modbus-read`
  - **Settings**
    - **FC**: Function code that specifies the Modbus action, e.g., FC 3 = Read Holding Registers.
    - **Address**: The starting register address to read from (usually zero-based).
    - **Quantity**: The number of consecutive registers to read.
    - **Poll Rate**: How often the node polls the Modbus device (e.g., every 10 seconds).
    - **Server**: Reference to a configured Modbus server (IP, port, protocol, etc.).

    - **Server**
        - **Host**: Modbus server IP.
        - **Port**: Modbus server port.
- `mqtt out`

```
"Server": "<embedded UNS broker>", //same name as the flow
"Topic": "<UNS model>",
```

Flow Reference

<details>
<summary>Show Modbus flow JSON</summary>

```
[
    {
        "id": "d3becb19e0d762f4",
        "type": "modbus-read",
        "z": "b002a07d1163c3de",
        "name": "status",
        "topic": "",
        "showStatusActivities": false,
        "logIOActivities": false,
        "showErrors": false,
        "showWarnings": true,
        "unitid": "1",
        "dataType": "HoldingRegister",
        "adr": "0",
        "quantity": "1",
        "rate": "10",
        "rateUnit": "s",
        "delayOnStart": false,
        "startDelayTime": "",
        "server": "c665b74952f6cf5a",
        "useIOFile": false,
        "ioFile": "",
        "useIOForPayload": false,
        "emptyMsgOnFail": false,
        "x": 130,
        "y": 500,
        "wires": [
            [
                "2987638967f632b4",
                "09b8538e09d0766a"
```

</details>

```
            ],
            []
        ]
    },
    {
        "id": "b068c119928c029a",
        "type": "mqtt out",
        "z": "b002a07d1163c3de",
        "name": "status",
        "topic":
"global_plant/Smart_Manufacturing_Group/Suzhou_Plant/Utility_Area/Compressed_A
ir_System/Air_Compressor_Station/Air_Compressor_01/State/compressor_status",
        "qos": "",
        "retain": "",
        "respTopic": "",
        "contentType": "",
        "userProps": "",
        "correl": "",
        "expiry": "",
        "broker": "broker-djnwpg00y370",
        "x": 510,
        "y": 540,
        "wires": []
    },
    {
        "id": "2987638967f632b4",
        "type": "debug",
        "z": "b002a07d1163c3de",
        "name": "debug 3",
        "active": true,
        "tosidebar": true,
        "console": false,
        "tostatus": false,
        "complete": "false",
        "statusVal": "",
        "statusType": "auto",
        "x": 260,
        "y": 600,
        "wires": []
    },
    {
        "id": "09b8538e09d0766a",
        "type": "function",
        "z": "b002a07d1163c3de",
        "name": "function 1",
        "func": "const value = msg.payload;\n\n// 1️⃣ verify\nif (value ===
undefined || value === null) {\n    node.warn(\"Empty Modbus payload\");\n
return null;\n}\n\n// 2️⃣ convert to number\nconst status_code =
Number(value);\n\n// 3️⃣ mapping to status\nconst statusMap = {\n    0:
\"maintenance\",\n    1: \"running\",\n    2: \"standby\",\n    3:
\"debug\",\n    4: \"fault\"\n};\n\n// 4️⃣ filtering\nif (!(status_code in
```

```
statusMap)) {\n    node.warn(\"Invalid status_code: \" + status_code);\n
return null;\n}\n\n// 5️⃣ contextualized output\nmsg.payload = {\n
status_code: status_code,\n    status_label: statusMap[status_code],\n
_valid: true,\n    _ts: new Date().toISOString()\n};\n\n// 6️⃣ optional  set
topic\nmsg.topic = \"v1/aircompressor01/state\";\n\nreturn msg;",
        "outputs": 1,
        "timeout": 0,
        "noerr": 0,
        "initialize": "",
        "finalize": "",
        "libs": [],
        "x": 320,
        "y": 500,
        "wires": [
            [
                "b068c119928c029a"
            ]
        ]
    },
    {
        "id": "c665b74952f6cf5a",
        "type": "modbus-client",
        "name": "modbus",
        "clienttype": "tcp",
        "bufferCommands": true,
        "stateLogEnabled": false,
        "queueLogEnabled": false,
        "failureLogEnabled": true,
        "tcpHost": "172.31.151.237",
        "tcpPort": "5020",
        "tcpType": "DEFAULT",
        "serialPort": "/dev/ttyUSB",
        "serialType": "RTU-BUFFERD",
        "serialBaudrate": 9600,
        "serialDatabits": 8,
        "serialStopbits": 1,
        "serialParity": "none",
        "serialConnectionDelay": 100,
        "serialAsciiResponseStartDelimiter": "0x3A",
        "unit_id": 1,
        "commandDelay": 1,
        "clientTimeout": 1000,
        "reconnectOnTimeout": true,
        "reconnectTimeout": 2000,
        "parallelUnitIdsAllowed": true,
        "showErrors": false,
        "showWarnings": true,
        "showLogs": true
    },
    {
        "id": "broker-djnwpg00y370",
```

```
        "type": "mqtt-broker",
        "z": "b002a07d1163c3de",
        "name": "emqx:1883",
        "broker": "emqx",
        "port": "1883",
        "clientid": "355424069147520",
        "usetls": false,
        "protocolVersion": "4",
        "keepalive": "60",
        "cleansession": true,
        "birthTopic": "",
        "birthQos": "0",
        "birthPayload": "",
        "closeTopic": "",
        "closeQos": "0",
        "closePayload": "",
        "willTopic": "",
        "willQos": "0",
        "willPayload": ""
    },
    {
        "id": "5cc203d0788c324d",
        "type": "global-config",
        "env": [],
        "modules": {
            "node-red-contrib-modbus": "5.43.0"
        }
    }
]
```

API

Configuring Data Flow

In the flow, add `inject`, `http request` and `mqtt out` nodes and configure the required information.

- `http request`

```
"Method": "<API method>",
"URL": "<API URL>",
"Return": "a parsed JSON object",
"Headers": "<API headers>"
```

- `mqtt out`

```
"Server": "<embedded UNS broker>", //same name as the flow
"Topic": "<UNS model>",
```

Flow Reference

Show API flow JSON

```
[
    {
        "id": "46273fcbb98afa11",
        "type": "inject",
        "z": "932355c6886aaf27",
        "name": "",
        "props": [
            {
                "p": "payload"
            },
            {
                "p": "topic",
                "vt": "str"
            }
        ],
        "repeat": "",
        "crontab": "",
        "once": false,
        "onceDelay": 0.1,
        "topic": "",
        "payload": "",
        "payloadType": "date",
        "x": 170,
        "y": 380,
        "wires": [
            [
                "3aca156076ad6dd1"
            ]
        ]
    },
    {
        "id": "3aca156076ad6dd1",
        "type": "http request",
        "z": "932355c6886aaf27",
        "name": "",
        "method": "GET",
        "ret": "obj",
```

```
        "paytoqs": "ignore",
        "url": "http://127.0.0.1:1880/api/wms/picking-zone-a/status",
        "tls": "",
        "persist": false,
        "proxy": "",
        "insecureHTTPParser": false,
        "authType": "",
        "senderr": false,
        "headers": [],
        "x": 380,
        "y": 380,
        "wires": [
            [
                "83fd65cf177c2ddf"
            ]
        ]
    },
    {
        "id": "83fd65cf177c2ddf",
        "type": "mqtt out",
        "z": "932355c6886aaf27",
        "name": "",
        "topic": "v1/Suzhou_Plant/Assembly-Area-1/Line-01/Station-
Screwing/Metric/torque",
        "qos": "",
        "retain": "",
        "respTopic": "",
        "contentType": "",
        "userProps": "",
        "correl": "",
        "expiry": "",
        "broker": "broker-djm245hxt9d9",
        "x": 780,
        "y": 440,
        "wires": []
    },
    {
        "id": "broker-djm245hxt9d9",
        "type": "mqtt-broker",
        "name": "emqx:1883",
        "broker": "emqx",
        "port": "1883",
        "clientid": "355039328257392",
        "usetls": false,
        "protocolVersion": "4",
        "keepalive": "60",
        "cleansession": true,
        "birthTopic": "",
        "birthQos": "0",
        "birthPayload": "",
        "closeTopic": "",
```

```
        "closeQos": "0",
        "closePayload": "",
        "willTopic": "",
        "willQos": "0",
        "willPayload": ""
    }
]
```

Other things you might want to know

- A `function` node is often used to clean the data before publishing to **UNS**.
- A `debug` node can be used to check the data on the spot.
- For node details, see **Node-RED flows**.

BEST PRACTICE

Security and Resilience

Overview of identity, data access, deployment availability, and recovery controls in Tier0.

Tier0 ensures data security through the following aspects:

- **Security**: Controls who can access the platform, applications, and data.
- **Resilience**: Covers service continuity and data recovery after failures.

Security Scope by Edition

Edition	Security Coverage
Cloud	Platform access control and authenticated access to MQTT and APIs.
Enterprise	Platform and data access control, plus deployment availability, backup, and recovery for private deployments.
Edge (open source)	Local data connection and modeling. Its built-in MQTT broker allows unauthenticated subscriptions.

See Choosing the Best Product for edition positioning.

> **Tier0 Cloud Security Overview**
> For network boundaries, tenant isolation, data protection, and security operations in the production Cloud service, see **Tier0 Cloud Network and Data Security Overview**. This overview applies only to Tier0 Cloud.

Security

Identity and Access Control

Area	Tier0 Mechanism	Details
Platform access	• Cloud controls access through workspace membership. • Enterprise controls access through user accounts.	User Management
Agent access	Agents authenticate with an API key and use the Tier0 CLI to access and operate the platform.	CLI authentication
Project and application access	Tier0 uses RBAC to control access to applications.	Access control

> **Note**
> Workspace membership does not automatically create an account. You still need to register and sign in to your own accounts. Account registration requires two-factor verification.

Data Access Control

Access	Tier0 Mechanism	Details
MQTT	• Clients authenticate with credentials generated by Tier0. • TLS and secure WebSocket encrypt data in transit.	MQTT credentials and connections
REST API	Accessing data requires a key generated by Tier0.	API Reference

Resilience

Deployment Availability

Enterprise currently provides two deployment plans for center nodes:

Plan	Availability Boundary	Details
Standard deployment	• Provides database primary/standby failover. • The Enterprise application and EMQX remain single points of failure.	Standard Deployment Plan
High availability deployment	• Adds active/standby Enterprise nodes and defined failover behavior for the database, cache, object storage, and managed runtime files. • Does not promise zero interruption or zero data loss.	High Availability Deployment Plan

> **Note**
> Detailed availability limitations are documented in **SLA and High Availability Boundaries**.

Backup and Recovery

Enterprise provides database backup and recovery. After recovery, check deployment status, run basic functionality checks, and confirm that users can sign in to the platform.

- For backups, see Backup and Restore.
- For recovery verification requirements, see Drill Requirements.

Response time

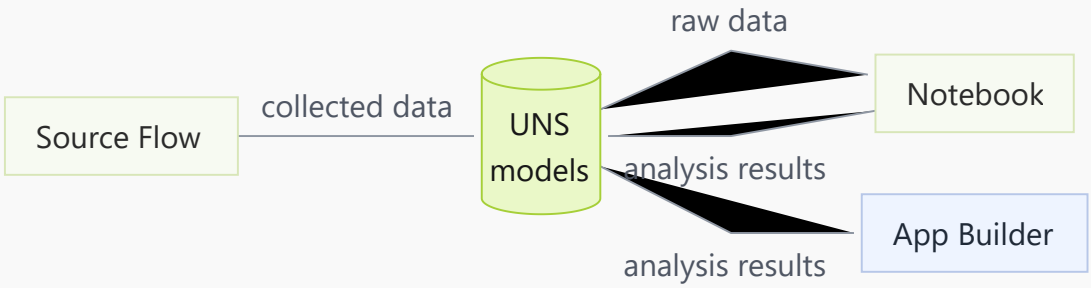
The typical response time is 3 business days.

BEST PRACTICE

Building Analytics Apps

Building an analytics app with Notebook using the Aramco Bowtie case (sanitized data).

Tier0 uses Marimo Notebook for advanced data analysis with Python. This guide uses a Bowtie application as an example.



Example Background

Refinery corrosion is a complex process influenced by multiple operating factors. This example demonstrates a real-time corrosion risk assessment workflow that combines process data to estimate the likelihood of corrosion and support proactive maintenance.

Getting Raw Data

1. Import the data model in **UNS**.

Show UNS model JSON

```
{
  "notes":
"type:PATH|TOPIC,topicType:STATE|ACTION|METRIC,schema.type:INTEGER|STRING
|FLOAT|DOUBLE|BOOLEAN|LONG|DATETIME",
  "namespace": [
    {
      "name": "Refinery",
      "type": "PATH",
```

```
      "children": [
        {
          "name": "CDU_Plant",
          "type": "PATH",
          "children": [
            {
              "name": "State",
              "type": "PATH",
              "topicType": "STATE",
              "children": [
                {
                  "name": "Corrosion_Risk",
                  "type": "TOPIC",
                  "topicType": "STATE",
                  "description": "Current corrosion risk state inferred
by Bayesian Network",
                  "enableHistory": "TRUE",
                  "schema": [
                    {
                      "name": "risk_state",
                      "type": "STRING"
                    },
                    {
                      "name": "previous_state",
                      "type": "STRING"
                    },
                    {
                      "name": "model_confidence",
                      "type": "FLOAT"
                    },
                    {
                      "name": "timestamp",
                      "type": "DATETIME"
                    }
                  ]
                }
              ]
            },
            {
              "name": "Metric",
              "type": "PATH",
              "topicType": "METRIC",
              "children": [
                {
                  "name": "Corrosion_Monitoring",
                  "type": "TOPIC",
                  "topicType": "METRIC",
                  "description": "Real-time process measurements for CDU
atmospheric overhead corrosion monitoring",
                  "enableHistory": "TRUE",
                  "mockData": "FALSE",
```

```
      "schema": [
        {
          "name": "d103_chloride",
          "type": "FLOAT"
        },
        {
          "name": "d103_ph",
          "type": "FLOAT"
        },
        {
          "name": "wash_water_flow",
          "type": "FLOAT"
        },
        {
          "name": "desalter_salt_ptb",
          "type": "FLOAT"
        },
        {
          "name": "desalter_bsw",
          "type": "FLOAT"
        },
        {
          "name": "wash_water_rate",
          "type": "FLOAT"
        },
        {
          "name": "rrd_ph",
          "type": "FLOAT"
        },
        {
          "name": "rrd_chloride",
          "type": "FLOAT"
        },
        {
          "name": "rrd_total_iron",
          "type": "FLOAT"
        },
        {
          "name": "sap_evidence",
          "type": "STRING"
        },
        {
          "name": "injection_quill_fail",
          "type": "BOOLEAN"
        },
        {
          "name": "timestamp",
          "type": "DATETIME"
        }
      ]
    },
```

```
                {
                  "name": "Corrosion_Analysis",
                  "type": "TOPIC",
                  "topicType": "METRIC",
                  "description": "Posterior probabilities generated by
the Bayesian Network",
                  "enableHistory": "TRUE",
                  "mockData": "FALSE",
                  "schema": [
                    {
                      "name": "p_normal",
                      "type": "DOUBLE"
                    },
                    {
                      "name": "p_developing",
                      "type": "DOUBLE"
                    },
                    {
                      "name": "p_confirmed",
                      "type": "DOUBLE"
                    },
                    {
                      "name": "total_corrosion_risk",
                      "type": "DOUBLE"
                    },
                    {
                      "name": "lopc_probability",
                      "type": "DOUBLE"
                    },
                    {
                      "name": "shutdown_probability",
                      "type": "DOUBLE"
                    },
                    {
                      "name": "escalation_probability",
                      "type": "DOUBLE"
                    },
                    {
                      "name": "timestamp",
                      "type": "DATETIME"
                    }
                  ]
                }
              ]
            }
          ]
        }
      ]
    }
  ]
```

```
}
```

2 Go to **Flows**, use **Source Flow** to connect raw data and publish it to **UNS**.

Show Source Flow JSON


```
[
    {
        "id": "6173469ed5064ccf",
        "type": "tab",
        "label": "corrosion",
        "disabled": false,
        "info": ""
    },
    {
        "id": "ad20b016baf7247f",
        "type": "OpcUa-Client",
        "z": "6173469ed5064ccf",
        "endpoint": "66bd118c33eb12f8",
        "action": "subscribe",
        "deadbandtype": "a",
        "deadbandvalue": 1,
        "time": 10,
        "timeUnit": "s",
        "certificate": "n",
        "localfile": "",
        "localkeyfile": "",
        "securitymode": "None",
        "securitypolicy": "None",
        "useTransport": false,
        "maxChunkCount": 1,
        "maxMessageSize": 8192,
        "receiveBufferSize": 8192,
        "sendBufferSize": 8192,
        "setstatusandtime": false,
        "keepsessionalive": false,
        "name": "",
        "x": 280,
        "y": 440,
        "wires": [
            [
                "ac9195bb4863dbdd",
                "5a3d61b2ae40dcb6"
            ],
            [],
            []
        ]
```

```
        },
        {
            "id": "b6746ed066ba0e1a",
            "type": "inject",
            "z": "6173469ed5064ccf",
            "name": "",
            "props": [
                {
                    "p": "payload"
                },
                {
                    "p": "topic",
                    "vt": "str"
                }
            ],
            "repeat": "",
            "crontab": "",
            "once": false,
            "onceDelay": 0.1,
            "topic": "multiple",
            "payload": "[   {     \"name\": \"D103_CHLORIDE\",
\"nodeId\": \"ns=2;i=3\",     \"datatype\": \"Float\"   },   {
\"name\": \"D103_PH\",     \"nodeId\": \"ns=2;i=4\",     \"datatype\":
\"Float\"   },   {     \"name\": \"WASH_WATER_FLOW\",     \"nodeId\":
\"ns=2;i=5\",     \"datatype\": \"Float\"   },   {     \"name\":
\"DESALTER_SALT_PTB\",     \"nodeId\": \"ns=2;i=6\",     \"datatype\":
\"Float\"   },   {     \"name\": \"DESALTER_BSW\",     \"nodeId\":
\"ns=2;i=7\",     \"datatype\": \"Float\"   },   {     \"name\":
\"WASH_WATER_RATE\",     \"nodeId\": \"ns=2;i=8\",     \"datatype\":
\"Float\"   },   {     \"name\": \"RRD_PH\",     \"nodeId\":
\"ns=2;i=9\",     \"datatype\": \"Float\"   },   {     \"name\":
\"RRD_CHLORIDE\",     \"nodeId\": \"ns=2;i=10\",     \"datatype\":
\"Float\"   },   {     \"name\": \"RRD_TOTAL_IRON\",     \"nodeId\":
\"ns=2;i=11\",     \"datatype\": \"Float\"   },   {     \"name\":
\"SAP_EVIDENCE\",     \"nodeId\": \"ns=2;i=12\",     \"datatype\":
\"UInt16\"   },   {     \"name\": \"INJECTION_QUILL_FAIL\",
\"nodeId\": \"ns=2;i=13\",     \"datatype\": \"Boolean\"   } ]",
            "payloadType": "json",
            "x": 90,
            "y": 440,
            "wires": [
                [
                    "ad20b016baf7247f"
                ]
            ]
        },
        {
            "id": "ac9195bb4863dbdd",
            "type": "debug",
            "z": "6173469ed5064ccf",
            "name": "debug 2",
```

```
            "active": true,
            "tosidebar": true,
            "console": false,
            "tostatus": false,
            "complete": "false",
            "statusVal": "",
            "statusType": "auto",
            "x": 490,
            "y": 480,
            "wires": []
        },
        {
            "id": "5a3d61b2ae40dcb6",
            "type": "function",
            "z": "6173469ed5064ccf",
            "name": "function 1",
            "func": "const TOPIC =
\"Refinery/CDU_Plant/Metric/Corrosion_Monitoring\";\n\nconst nodeMap =
{\n    \"ns=2;i=3\": { field: \"d103_chloride\", type: \"float\" },\n
\"ns=2;i=4\": { field: \"d103_ph\", type: \"float\" },\n    \"ns=2;i=5\":
{ field: \"wash_water_flow\", type: \"float\" },\n    \"ns=2;i=6\": {
field: \"desalter_salt_ptb\", type: \"float\" },\n    \"ns=2;i=7\": {
field: \"desalter_bsw\", type: \"float\" },\n    \"ns=2;i=8\": { field:
\"wash_water_rate\", type: \"float\" },\n    \"ns=2;i=9\": { field:
\"rrd_ph\", type: \"float\" },\n    \"ns=2;i=10\": { field:
\"rrd_chloride\", type: \"float\" },\n    \"ns=2;i=11\": { field:
\"rrd_total_iron\", type: \"float\" },\n    \"ns=2;i=12\": { field:
\"sap_evidence\", type: \"sap\" },\n    \"ns=2;i=13\": { field:
\"injection_quill_fail\", type: \"boolean\" }\n};\n\nconst sapMap = {\n
0: \"none\",\n    1: \"work_order\",\n    2:
\"inspection_finding\"\n};\n\nfunction normalizeNodeId(v) {\n    if (!v)
return \"\";\n    if (typeof v === \"string\") return v;\n    if
(v.toString) return v.toString();\n    return \"\";\n}\n\nfunction
extractValue(payload) {\n    // OPC UA DataValue: payload.value.value\n
if (payload && payload.value && payload.value.value !== undefined) {\n
return payload.value.value;\n    }\n\n    // Some OPC UA nodes output
Variant directly.\n    if (payload && payload.value !== undefined) {\n
return payload.value;\n    }\n\n    return payload;\n}\n\nfunction
extractNodeId(msg) {\n    return normalizeNodeId(\n        msg.nodeId
||\n        msg.topic ||\n        msg.payload?.nodeId ||\n
msg.payload?.nodeid\n    );\n}\n\nconst nodeId =
extractNodeId(msg);\nconst spec = nodeMap[nodeId];\n\nif (!spec) {\n
node.warn(`Unknown OPC UA nodeId: ${nodeId}`);\n    return
null;\n}\n\nlet value = extractValue(msg.payload);\n\nif (spec.type ===
\"float\") {\n    value = Number(value);\n} else if (spec.type ===
\"boolean\") {\n    value = Boolean(value);\n} else if (spec.type ===
\"sap\") {\n    value = sapMap[Number(value)] || \"unknown\";\n}\n\nlet
latest = context.get(\"latest\") || {};\nlatest[spec.field] =
value;\nlatest.timestamp = new
Date().toISOString();\ncontext.set(\"latest\", latest);\n\n// Publish
only after all 11 fields have been received at least once.\nconst
```

```
requiredFields = Object.values(nodeMap).map(x => x.field);\nconst ready =
requiredFields.every(field => latest[field] !== undefined);\n\nif
(!ready) {\n    node.status({\n        fill: \"yellow\",\n        shape:
\"ring\",\n        text: `waiting ${Object.keys(latest).length - 1}/11`\n
});\n    return null;\n}\n\nmsg.topic = TOPIC;\nmsg.payload = {\n
d103_chloride: latest.d103_chloride,\n    d103_ph: latest.d103_ph,\n
wash_water_flow: latest.wash_water_flow,\n    desalter_salt_ptb:
latest.desalter_salt_ptb,\n    desalter_bsw: latest.desalter_bsw,\n
wash_water_rate: latest.wash_water_rate,\n    rrd_ph: latest.rrd_ph,\n
rrd_chloride: latest.rrd_chloride,\n    rrd_total_iron:
latest.rrd_total_iron,\n    sap_evidence: latest.sap_evidence,\n
injection_quill_fail: latest.injection_quill_fail,\n    timestamp:
latest.timestamp\n};\n\nnode.status({\n    fill: \"green\",\n    shape:
\"dot\",\n    text: \"published corrosion monitoring\"\n});\n\nreturn
msg;",
          "outputs": 1,
          "timeout": 0,
          "noerr": 0,
          "initialize": "",
          "finalize": "",
          "libs": [],
          "x": 370,
          "y": 580,
          "wires": [
              [
                  "aa15be33a84fb812",
                  "4fdd68dda3ee5c9e"
              ]
          ]
      },
      {
          "id": "aa15be33a84fb812",
          "type": "mqtt out",
          "z": "6173469ed5064ccf",
          "name": "",
          "topic": "",
          "qos": "",
          "retain": "",
          "respTopic": "",
          "contentType": "",
          "userProps": "",
          "correl": "",
          "expiry": "",
          "broker": "broker-djx75cwgiywq",
          "x": 550,
          "y": 580,
          "wires": []
      },
      {
          "id": "4fdd68dda3ee5c9e",
          "type": "debug",
```

```
        "z": "6173469ed5064ccf",
        "name": "debug 3",
        "active": true,
        "tosidebar": true,
        "console": false,
        "tostatus": false,
        "complete": "false",
        "statusVal": "",
        "statusType": "auto",
        "x": 450,
        "y": 680,
        "wires": []
    },
    {
        "id": "66bd118c33eb12f8",
        "type": "OpcUa-Endpoint",
        "endpoint": "opc.tcp://172.31.151.237:4850",
        "secpol": "None",
        "secmode": "None",
        "none": true,
        "login": false,
        "usercert": false,
        "usercertificate": "",
        "userprivatekey": ""
    },
    {
        "id": "broker-djx75cwgiywq",
        "type": "mqtt-broker",
        "z": "6173469ed5064ccf",
        "name": "corrosion",
        "broker": "emqx",
        "port": "1883",
        "clientid": "357356352615696",
        "usetls": false,
        "protocolVersion": "4",
        "keepalive": "60",
        "cleansession": true,
        "birthTopic": "",
        "birthQos": "0",
        "birthPayload": "",
        "closeTopic": "",
        "closeQos": "0",
        "closePayload": "",
        "willTopic": "",
        "willQos": "0",
        "willPayload": ""
    },
    {
        "id": "c5201f1ad482b182",
        "type": "global-config",
        "env": [],
```

```
            "modules": {
                "node-red-contrib-opcua": "0.2.339"
            }
        }
    ]
```

Building an Analytics App in Notebook

1. Add the following cells in the notebook to analyze data from **UNS**.
    - Cell 1 - Import required dependencies and configure the MQTT broker, topics, and runtime settings.

Show code

```
@app.cell
def _():
    """Imports and runtime configuration."""
    import json
    import math
    import os
    import threading
    import time
    from datetime import datetime, timezone

    import marimo as mo
    import pandas as pd
    import paho.mqtt.client as mqtt

    MQTT_BROKER_HOST = os.getenv("TIER0_MQTT_HOST", "emqx")
    MQTT_BROKER_PORT = int(os.getenv("TIER0_MQTT_PORT", "1883"))
    MQTT_USERNAME = os.getenv("TIER0_MQTT_USERNAME",
"357890055321856")
    MQTT_PASSWORD = os.getenv("TIER0_MQTT_PASSWORD",
"caa6b34242002a3dff4948dd73ce8a")
    MQTT_CLIENT_ID = os.getenv(
        "TIER0_MQTT_CLIENT_ID",
        "357890055321856_bn",
    )

    SOURCE_TOPIC = "Refinery/CDU_Plant/Metric/Corrosion_Monitoring"
    ANALYSIS_TOPIC = "Refinery/CDU_Plant/Metric/Corrosion_Analysis"
    RISK_TOPIC = "Refinery/CDU_Plant/State/Corrosion_Risk"
```

```
    SOURCE_QOS = int(os.getenv("TIER0_SOURCE_QOS", "1"))
    OUTPUT_QOS = int(os.getenv("TIER0_OUTPUT_QOS", "1"))
    AUTOSTART = os.getenv("TIER0_BN_AUTOSTART", "true").lower() in {
        "1", "true", "yes", "on"
    }
    ROOT_CONFIDENCE = float(os.getenv("TIER0_BN_ROOT_CONFIDENCE",
"0.85"))

    def utc_now_iso() -> str:
        return (
            datetime.now(timezone.utc)
            .isoformat(timespec="milliseconds")
            .replace("+00:00", "Z")
        )

    mo.md(
        """
        # CDU Corrosion Bayesian Analysis

        This notebook subscribes to a complete corrosion-monitoring
snapshot,
        performs one Bayesian inference for each new source message,
and publishes
        the analysis results back to the UNS.
        """
    )
    return (
        ANALYSIS_TOPIC,
        AUTOSTART,
        MQTT_BROKER_HOST,
        MQTT_BROKER_PORT,
        MQTT_CLIENT_ID,
        MQTT_PASSWORD,
        MQTT_USERNAME,
        OUTPUT_QOS,
        RISK_TOPIC,
        ROOT_CONFIDENCE,
        SOURCE_QOS,
        SOURCE_TOPIC,
        datetime,
        json,
        math,
        mo,
        mqtt,
        os,
        pd,
        threading,
        time,
        timezone,
        utc_now_iso,
```

```
    )
```

> **For external connections to UNS broker**
> Refer to **Connecting MQTT Clients**

- Cell 2 - Define the Bayesian Network topology, state spaces, CPT generators, and probability rules.

Show code

```
@app.cell
def _(math, pd):
    """Static Bayesian-network definition and CPT generators."""
    import itertools

    from pgmpy.factors.discrete import TabularCPD
    from pgmpy.inference import VariableElimination
    from pgmpy.models import DiscreteBayesianNetwork

    STATE_SPACES = {
        "STATE_3": ["LOW", "NORMAL", "HIGH"],
        "STATE_YN": ["NO", "YES"],
        "PB2_3STATE": ["FAIL", "IL_OK", "SL_OK"],
        "TOP_EVENT_STATE": ["NORMAL", "DEVELOPING", "CONFIRMED"],
    }

    NODE_STATE_SPACE = {
        "T1_HighChlorideLoad": "STATE_3",
        "T2_ChemProtectionFail": "STATE_3",
        "T3_WashWaterLow": "STATE_3",
        "PB1_Desalter_OK": "STATE_YN",
        "PB2_WashQty_State": "PB2_3STATE",
        "PB3_Neutralization_OK": "STATE_YN",
        "MB1_ESD_OK": "STATE_YN",
        "MB2_TSV_OK": "STATE_YN",
        "TE_CorrosionState": "TOP_EVENT_STATE",
        "EV_TOTIRON_High": "STATE_YN",
        "EV_SAP_Positive": "STATE_YN",
        "EV_QuillFailure": "STATE_YN",
        "C1_LOPC": "STATE_YN",
        "C2_UnplannedShutdown": "STATE_YN",
        "C3_Escalation": "STATE_YN",
    }

    EDGES = [
```

```
        ("T1_HighChlorideLoad", "TE_CorrosionState"),
        ("T2_ChemProtectionFail", "TE_CorrosionState"),
        ("T3_WashWaterLow", "TE_CorrosionState"),
        ("PB1_Desalter_OK", "TE_CorrosionState"),
        ("PB2_WashQty_State", "TE_CorrosionState"),
        ("PB3_Neutralization_OK", "TE_CorrosionState"),
        ("TE_CorrosionState", "EV_TOTIRON_High"),
        ("TE_CorrosionState", "EV_SAP_Positive"),
        ("TE_CorrosionState", "EV_QuillFailure"),
        ("TE_CorrosionState", "C1_LOPC"),
        ("MB1_ESD_OK", "C1_LOPC"),
        ("MB2_TSV_OK", "C1_LOPC"),
        ("TE_CorrosionState", "C2_UnplannedShutdown"),
        ("MB1_ESD_OK", "C2_UnplannedShutdown"),
        ("C1_LOPC", "C3_Escalation"),
        ("MB1_ESD_OK", "C3_Escalation"),
        ("MB2_TSV_OK", "C3_Escalation"),
    ]

    def states_for(node_id: str) -> list[str]:
        return STATE_SPACES[NODE_STATE_SPACE[node_id]]

    def softmax(values: list[float]) -> list[float]:
        maximum = max(values)
        exps = [math.exp(value - maximum) for value in values]
        total = sum(exps)
        return [value / total for value in exps]

    THREAT_SCORE = {"LOW": 0.0, "NORMAL": 1.0, "HIGH": 2.0}
    PB1_OK_BONUS = {"NO": 0.8, "YES": -0.6}
    PB3_OK_BONUS = {"NO": 1.0, "YES": -0.7}
    PB2_STATE_BONUS = {"FAIL": 1.2, "IL_OK": 0.3, "SL_OK": -0.6}

    def top_event_distribution(t1, t2, t3, pb1, pb2, pb3) ->
dict[str, float]:
        score = (
            THREAT_SCORE[t1]
            + THREAT_SCORE[t2]
            + THREAT_SCORE[t3]
            + PB1_OK_BONUS[pb1]
            + PB2_STATE_BONUS[pb2]
            + PB3_OK_BONUS[pb3]
        )
        probabilities = softmax(
            [
                2.2 - score,
                -0.2 + 0.6 * score,
                -1.2 + 0.8 * score,
            ]
        )
```

```
        return dict(zip(STATE_SPACES["TOP_EVENT_STATE"],
probabilities))

    TE_PARENTS = [
        "T1_HighChlorideLoad",
        "T2_ChemProtectionFail",
        "T3_WashWaterLow",
        "PB1_Desalter_OK",
        "PB2_WashQty_State",
        "PB3_Neutralization_OK",
    ]

    TE_PARENT_SPACES = [states_for(parent) for parent in TE_PARENTS]
    te_rows = []
    for combination in itertools.product(*TE_PARENT_SPACES):
        distribution = top_event_distribution(*combination)
        te_rows.append(
            [
                *combination,
                distribution["NORMAL"],
                distribution["DEVELOPING"],
                distribution["CONFIRMED"],
            ]
        )
    TE_CPT_DF = pd.DataFrame(
        te_rows,
        columns=TE_PARENTS
        + ["P_NORMAL", "P_DEVELOPING", "P_CONFIRMED"],
    )

    def evidence_yes_probability(te_state: str, kind: str) -> float:
        mappings = {
            "TOTIRON": {"NORMAL": 0.15, "DEVELOPING": 0.65,
"CONFIRMED": 0.85},
            "SAP": {"NORMAL": 0.08, "DEVELOPING": 0.35, "CONFIRMED":
0.75},
            "QUILL": {"NORMAL": 0.03, "DEVELOPING": 0.15,
"CONFIRMED": 0.55},
        }
        return mappings[kind][te_state]

    def lopc_yes_probability(te: str, mb1: str, mb2: str) -> float:
        base = {"NORMAL": 0.02, "DEVELOPING": 0.08, "CONFIRMED":
0.25}[te]
        return min(0.98, base + (0.10 if mb1 == "NO" else 0) + (0.18
if mb2 == "NO" else 0))

    def shutdown_yes_probability(te: str, mb1: str) -> float:
        base = {"NORMAL": 0.05, "DEVELOPING": 0.35, "CONFIRMED":
0.60}[te]
        return min(0.98, base + (0.10 if mb1 == "NO" else 0))
```

```
    def escalation_yes_probability(lopc: str, mb1: str, mb2: str) ->
float:
        if lopc == "NO":
            return 0.01
        return min(0.98, 0.10 + (0.18 if mb1 == "NO" else 0) + (0.35
if mb2 == "NO" else 0))

    def peaked_prior(node_id: str, center_state: str, confidence:
float) -> list[float]:
        states = states_for(node_id)
        confidence = min(max(float(confidence), 0.0), 1.0)
        if center_state not in states:
            return [1.0 / len(states)] * len(states)
        if len(states) == 1:
            return [1.0]
        remainder = (1.0 - confidence) / (len(states) - 1)
        probabilities = [remainder] * len(states)
        probabilities[states.index(center_state)] = confidence
        return probabilities

    def binary_cpd(child: str, parent: str, yes_probability_fn):
        parent_states = states_for(parent)
        yes_values = [float(yes_probability_fn(state)) for state in
parent_states]
        return TabularCPD(
            variable=child,
            variable_card=2,
            values=[[1.0 - value for value in yes_values],
yes_values],
            evidence=[parent],
            evidence_card=[len(parent_states)],
            state_names={child: states_for(child), parent:
parent_states},
        )

    def binary_multi_cpd(child: str, parents: list[str],
yes_probability_fn):
        parent_spaces = [states_for(parent) for parent in parents]
        combinations = list(itertools.product(*parent_spaces))
        yes_values = [float(yes_probability_fn(*combination)) for
combination in combinations]
        return TabularCPD(
            variable=child,
            variable_card=2,
            values=[[1.0 - value for value in yes_values],
yes_values],
            evidence=parents,
            evidence_card=[len(space) for space in parent_spaces],
            state_names={
                child: states_for(child),
```

```
                **{parent: states_for(parent) for parent in
parents},
            },
        )

    return (
        DiscreteBayesianNetwork,
        EDGES,
        NODE_STATE_SPACE,
        STATE_SPACES,
        TE_CPT_DF,
        TE_PARENTS,
        TabularCPD,
        VariableElimination,
        binary_cpd,
        binary_multi_cpd,
        escalation_yes_probability,
        evidence_yes_probability,
        itertools,
        lopc_yes_probability,
        peaked_prior,
        shutdown_yes_probability,
        states_for,
    )
```

- Cell 3 - Validate the incoming MQTT payload and convert continuous process values into Bayesian Network states.

Show code

```
@app.cell
def _(ROOT_CONFIDENCE, utc_now_iso):
    """Source validation, discretization, and one-snapshot
analysis."""
    import hashlib

    REQUIRED_SOURCE_FIELDS = [
        "d103_chloride",
        "d103_ph",
        "wash_water_flow",
        "desalter_salt_ptb",
        "desalter_bsw",
        "wash_water_rate",
        "rrd_ph",
        "rrd_chloride",
        "rrd_total_iron",
        "sap_evidence",
```

```
        "injection_quill_fail",
        "timestamp",
    ]

    def parse_boolean(value) -> bool:
        if isinstance(value, bool):
            return value
        if isinstance(value, (int, float)):
            return bool(value)
        if isinstance(value, str):
            normalized = value.strip().lower()
            if normalized in {"true", "1", "yes", "y", "on"}:
                return True
            if normalized in {"false", "0", "no", "n", "off"}:
                return False
        raise ValueError(f"invalid boolean value: {value!r}")

    def normalize_sap(value) -> str:
        normalized = str(value).strip().lower()
        if normalized in {"0", "none", "normal", "", "nan"}:
            return "None"
        if normalized in {"1", "work_order", "work order", "minor",
"warning"}:
            return "Minor corrosion observed"
        if normalized in {
            "2",
            "inspection_finding",
            "inspection finding",
            "confirmed",
            "abnormal",
        }:
            return "Confirmed corrosion / thickness loss"
        raise ValueError(f"unsupported sap_evidence value:
{value!r}")

    def normalize_source_payload(payload: dict) -> dict:
        if not isinstance(payload, dict):
            raise TypeError("source MQTT payload must be a JSON
object")

        lower = {str(key).lower(): value for key, value in
payload.items()}
        missing = [field for field in REQUIRED_SOURCE_FIELDS if
field not in lower]
        if missing:
            raise ValueError(f"source payload is missing fields:
{missing}")

        normalized = {
            "d103_chloride": float(lower["d103_chloride"]),
            "d103_ph": float(lower["d103_ph"]),
```

```
            "wash_water_flow": float(lower["wash_water_flow"]),
            "desalter_salt_ptb": float(lower["desalter_salt_ptb"]),
            "desalter_bsw": float(lower["desalter_bsw"]),
            "wash_water_rate": float(lower["wash_water_rate"]),
            "rrd_ph": float(lower["rrd_ph"]),
            "rrd_chloride": float(lower["rrd_chloride"]),
            "rrd_total_iron": float(lower["rrd_total_iron"]),
            "sap_evidence": normalize_sap(lower["sap_evidence"]),
            "injection_quill_fail":
parse_boolean(lower["injection_quill_fail"]),
            "timestamp": str(lower["timestamp"]).strip(),
        }

        numeric_ranges = {
            "d103_chloride": (0.0, 1000.0),
            "d103_ph": (0.0, 14.0),
            "wash_water_flow": (0.0, 10000.0),
            "desalter_salt_ptb": (0.0, 1000.0),
            "desalter_bsw": (0.0, 100.0),
            "wash_water_rate": (0.0, 100.0),
            "rrd_ph": (0.0, 14.0),
            "rrd_chloride": (0.0, 10000.0),
            "rrd_total_iron": (0.0, 10000.0),
        }
        for field, (minimum, maximum) in numeric_ranges.items():
            value = normalized[field]
            if not minimum <= value <= maximum:
                raise ValueError(f"{field} out of accepted range:
{value}")

        if not normalized["timestamp"]:
            raise ValueError("timestamp cannot be empty")
        return normalized

    def source_message_id(payload: dict) -> str:
        # Timestamp is the primary event key. Hash protects against
two different
        # snapshots accidentally sharing the same timestamp.
        ordered = "|".join(str(payload[field]) for field in
REQUIRED_SOURCE_FIELDS)
        return hashlib.sha256(ordered.encode("utf-8")).hexdigest()

    def discretize_source(payload: dict) -> tuple[dict, dict]:
        t1 = (
            "LOW"
            if payload["d103_chloride"] < 5.0
            else "NORMAL"
            if payload["d103_chloride"] <= 20.0
            else "HIGH"
        )
```

```
        chemical_failure_score = int(payload["d103_ph"] < 5.5) +
int(
            payload["d103_chloride"] > 10.0
        )
        t2 = (
            "HIGH"
            if chemical_failure_score == 2
            else "NORMAL"
            if chemical_failure_score == 1
            else "LOW"
        )

        t3 = (
            "HIGH"
            if payload["wash_water_flow"] < 6.0
            else "NORMAL"
            if payload["wash_water_flow"] <= 10.0
            else "LOW"
        )

        pb1 = (
            "YES"
            if payload["desalter_salt_ptb"] <= 1.0
            and payload["desalter_bsw"] <= 0.2
            else "NO"
        )

        pb2 = (
            "FAIL"
            if payload["wash_water_rate"] < 4.0
            else "IL_OK"
            if payload["wash_water_rate"] < 6.0
            else "SL_OK"
        )

        pb3 = (
            "YES"
            if payload["rrd_ph"] >= 5.5 and payload["rrd_chloride"]
<= 10.0
            else "NO"
        )

        root_centers = {
            "T1_HighChlorideLoad": t1,
            "T2_ChemProtectionFail": t2,
            "T3_WashWaterLow": t3,
            "PB1_Desalter_OK": pb1,
            "PB2_WashQty_State": pb2,
            "PB3_Neutralization_OK": pb3,
            # The source model currently has no MB fields. Keep both
mitigations
```

```
            # available by default, matching the previous notebook
UI defaults.
            "MB1_ESD_OK": "YES",
            "MB2_TSV_OK": "YES",
        }

        evidence = {
            "EV_TOTIRON_High": "YES" if payload["rrd_total_iron"] >=
10.0 else "NO",
            "EV_SAP_Positive": "YES" if payload["sap_evidence"] !=
"None" else "NO",
            "EV_QuillFailure": "YES" if
payload["injection_quill_fail"] else "NO",
        }
        return root_centers, evidence

    return (
        REQUIRED_SOURCE_FIELDS,
        discretize_source,
        normalize_source_payload,
        parse_boolean,
        source_message_id,
    )
```

- Cell 4 - Build a Bayesian Network for the current snapshot, execute inference, and generate output payloads.

Show code

```
@app.cell
def _(
    DiscreteBayesianNetwork,
    EDGES,
    ROOT_CONFIDENCE,
    STATE_SPACES,
    TE_CPT_DF,
    TE_PARENTS,
    TabularCPD,
    VariableElimination,
    binary_cpd,
    binary_multi_cpd,
    discretize_source,
    escalation_yes_probability,
    evidence_yes_probability,
    itertools,
    lopc_yes_probability,
    peaked_prior,
```

```
    shutdown_yes_probability,
    states_for,
    utc_now_iso,
):
    """Build a fresh BN for the current snapshot and execute
inference."""

    def factor_probability(factor, variable: str, state: str) ->
float:
        states = factor.state_names[variable]
        values = list(map(float, factor.values))
        return float(values[states.index(state)])

    def build_model(root_centers: dict):
        model = DiscreteBayesianNetwork(EDGES)

        root_cpds = []
        for node_id, center_state in root_centers.items():
            states = states_for(node_id)
            probabilities = peaked_prior(node_id, center_state,
ROOT_CONFIDENCE)
            root_cpds.append(
                TabularCPD(
                    variable=node_id,
                    variable_card=len(states),
                    values=[[probability] for probability in
probabilities],
                    state_names={node_id: states},
                )
            )

        te_values = [
            TE_CPT_DF["P_NORMAL"].astype(float).tolist(),
            TE_CPT_DF["P_DEVELOPING"].astype(float).tolist(),
            TE_CPT_DF["P_CONFIRMED"].astype(float).tolist(),
        ]
        te_cpd = TabularCPD(
            variable="TE_CorrosionState",
            variable_card=3,
            values=te_values,
            evidence=TE_PARENTS,
            evidence_card=[len(states_for(parent)) for parent in
TE_PARENTS],
            state_names={
                "TE_CorrosionState":
STATE_SPACES["TOP_EVENT_STATE"],
                **{parent: states_for(parent) for parent in
TE_PARENTS},
            },
        )
```

```
        evidence_cpds = [
            binary_cpd(
                "EV_TOTIRON_High",
                "TE_CorrosionState",
                lambda state: evidence_yes_probability(state,
"TOTIRON"),
            ),
            binary_cpd(
                "EV_SAP_Positive",
                "TE_CorrosionState",
                lambda state: evidence_yes_probability(state,
"SAP"),
            ),
            binary_cpd(
                "EV_QuillFailure",
                "TE_CorrosionState",
                lambda state: evidence_yes_probability(state,
"QUILL"),
            ),
        ]

        consequence_cpds = [
            binary_multi_cpd(
                "C1_LOPC",
                ["TE_CorrosionState", "MB1_ESD_OK", "MB2_TSV_OK"],
                lopc_yes_probability,
            ),
            binary_multi_cpd(
                "C2_UnplannedShutdown",
                ["TE_CorrosionState", "MB1_ESD_OK"],
                shutdown_yes_probability,
            ),
            binary_multi_cpd(
                "C3_Escalation",
                ["C1_LOPC", "MB1_ESD_OK", "MB2_TSV_OK"],
                escalation_yes_probability,
            ),
        ]

        model.add_cpds(*root_cpds, te_cpd, *evidence_cpds,
*consequence_cpds)
        if not model.check_model():
            raise RuntimeError("Bayesian network model validation
failed")
        return model

    def analyze_snapshot(payload: dict, previous_risk_state: str |
None = None) -> dict:
        root_centers, evidence = discretize_source(payload)
        model = build_model(root_centers)
        inference = VariableElimination(model)
```

```
        q_te = inference.query(["TE_CorrosionState"],
evidence=evidence, show_progress=False)
        q_c1 = inference.query(["C1_LOPC"], evidence=evidence,
show_progress=False)
        q_c2 = inference.query(["C2_UnplannedShutdown"],
evidence=evidence, show_progress=False)
        q_c3 = inference.query(["C3_Escalation"], evidence=evidence,
show_progress=False)

        p_normal = factor_probability(q_te, "TE_CorrosionState",
"NORMAL")
        p_developing = factor_probability(q_te, "TE_CorrosionState",
"DEVELOPING")
        p_confirmed = factor_probability(q_te, "TE_CorrosionState",
"CONFIRMED")
        probabilities = {
            "NORMAL": p_normal,
            "DEVELOPING": p_developing,
            "CONFIRMED": p_confirmed,
        }
        risk_state = max(probabilities, key=probabilities.get)
        analysis_timestamp = utc_now_iso()

        analysis_payload = {
            "p_normal": p_normal,
            "p_developing": p_developing,
            "p_confirmed": p_confirmed,
            "total_corrosion_risk": p_developing + p_confirmed,
            "lopc_probability": factor_probability(q_c1, "C1_LOPC",
"YES"),
            "shutdown_probability": factor_probability(
                q_c2, "C2_UnplannedShutdown", "YES"
            ),
            "escalation_probability": factor_probability(q_c3,
"C3_Escalation", "YES"),
            "timestamp": analysis_timestamp,
        }

        risk_payload = {
            "risk_state": risk_state,
            "previous_state": previous_risk_state or risk_state,
            "model_confidence": max(probabilities.values()),
            "timestamp": analysis_timestamp,
        }

        return {
            "source_timestamp": payload["timestamp"],
            "root_states": root_centers,
            "evidence": evidence,
            "analysis_payload": analysis_payload,
```

```
            "risk_payload": risk_payload,
            "risk_state": risk_state,
        }

    return analyze_snapshot, build_model, factor_probability
```

- Cell 5 - Subscribe to the source topic, process each incoming snapshot, run Bayesian analysis, and publish the results.

Show code

```
@app.cell
def _(
    ANALYSIS_TOPIC,
    AUTOSTART,
    MQTT_BROKER_HOST,
    MQTT_BROKER_PORT,
    MQTT_CLIENT_ID,
    MQTT_PASSWORD,
    MQTT_USERNAME,
    OUTPUT_QOS,
    RISK_TOPIC,
    SOURCE_QOS,
    SOURCE_TOPIC,
    analyze_snapshot,
    json,
    mqtt,
    normalize_source_payload,
    source_message_id,
    threading,
    utc_now_iso,
):
    """Long-running MQTT subscriber and publisher service."""

    class CorrosionBNService:
        def __init__(self):
            self.lock = threading.RLock()
            self.client = None
            self.running = False
            self.connected = False
            self.last_message_id = None
            self.last_source_timestamp = None
            self.last_analysis_timestamp = None
            self.previous_risk_state = None
            self.processed_count = 0
            self.duplicate_count = 0
            self.error_count = 0
```

```
            self.last_error = None
            self.last_result = None
            self.connection_rc = None

        def _create_client(self):
            # Use MQTT 3.1.1 and callback API v1 for broad
compatibility with
            # EMQX and different paho-mqtt releases bundled in
Notebook images.
            kwargs = {
                "client_id": MQTT_CLIENT_ID,
                "clean_session": True,
                "protocol": mqtt.MQTTv311,
                "transport": "tcp",
            }
            try:
                client = mqtt.Client(

callback_api_version=mqtt.CallbackAPIVersion.VERSION1,
                    **kwargs,
                )
            except (AttributeError, TypeError):
                client = mqtt.Client(**kwargs)

            client.username_pw_set(MQTT_USERNAME, MQTT_PASSWORD)
            client.on_connect = self._on_connect
            client.on_disconnect = self._on_disconnect
            client.on_message = self._on_message
            client.reconnect_delay_set(min_delay=1, max_delay=30)
            client.enable_logger()
            return client

        def _on_connect(self, client, userdata, flags, rc,
properties=None):
            code = int(rc)
            with self.lock:
                self.connected = code == 0
                self.connection_rc = code
                self.last_error = None if code == 0 else f"MQTT
connection rejected, rc={code}"
            if code == 0:
                sub_rc, mid = client.subscribe(SOURCE_TOPIC,
qos=SOURCE_QOS)
                if sub_rc != mqtt.MQTT_ERR_SUCCESS:
                    with self.lock:
                        self.last_error = f"MQTT subscribe failed,
rc={sub_rc}"
                    return
                print(f"[BN SERVICE] connected and subscribed to
{SOURCE_TOPIC}")
            else:
```

```
                print(f"[BN SERVICE ERROR] connection rejected rc=
{code}")

        def _on_disconnect(self, client, userdata, rc,
properties=None):
            with self.lock:
                self.connected = False
                self.connection_rc = int(rc) if rc is not None else
None
                if self.running and rc:
                    self.last_error = f"MQTT disconnected
unexpectedly, rc={rc}"
            print(f"[BN SERVICE] disconnected: {rc}")

        def _publish_json(self, topic: str, payload: dict):
            info = self.client.publish(
                topic,
                payload=json.dumps(payload, separators=(",", ":"),
ensure_ascii=False),
                qos=OUTPUT_QOS,
                retain=False,
            )
            info.wait_for_publish(timeout=10)
            if info.rc != mqtt.MQTT_ERR_SUCCESS:
                raise RuntimeError(f"MQTT publish failed for
{topic}: rc={info.rc}")

        def _on_message(self, client, userdata, msg):
            try:
                raw_payload = json.loads(msg.payload.decode("utf-
8"))
                payload = normalize_source_payload(raw_payload)
                message_id = source_message_id(payload)

                with self.lock:
                    if message_id == self.last_message_id:
                        self.duplicate_count += 1
                        return
                    previous_risk_state = self.previous_risk_state

                result = analyze_snapshot(payload,
previous_risk_state)

                # Publish only after both payloads have been
generated successfully.
                self._publish_json(ANALYSIS_TOPIC,
result["analysis_payload"])
                self._publish_json(RISK_TOPIC,
result["risk_payload"])

                with self.lock:
```

```
                    self.last_message_id = message_id
                    self.last_source_timestamp =
payload["timestamp"]
                    self.last_analysis_timestamp =
result["analysis_payload"]["timestamp"]
                    self.previous_risk_state = result["risk_state"]
                    self.processed_count += 1
                    self.last_result = result
                    self.last_error = None

                print(
                    "[BN SERVICE] processed",
                    payload["timestamp"],
                    "->",
                    result["risk_state"],
                )
            except Exception as exc:
                with self.lock:
                    self.error_count += 1
                    self.last_error = f"{type(exc).__name__}: {exc}"
                print("[BN SERVICE ERROR]", self.last_error)

        def start(self):
            with self.lock:
                if self.running:
                    return False
                if not MQTT_PASSWORD:
                    raise RuntimeError(
                        "TIER0_MQTT_PASSWORD is empty. Set it before
starting the service."
                    )
                self.client = self._create_client()
                self.running = True
            try:
                # Synchronous connect surfaces DNS, socket, and
authentication
                # failures immediately instead of leaving
running=True forever.
                rc = self.client.connect(
                    MQTT_BROKER_HOST,
                    MQTT_BROKER_PORT,
                    keepalive=30,
                )
                if rc != mqtt.MQTT_ERR_SUCCESS:
                    raise RuntimeError(f"MQTT connect() failed, rc=
{rc}")
                self.client.loop_start()
                return True
            except Exception as exc:
                with self.lock:
                    self.running = False
```

```
                    self.connected = False
                    self.last_error = f"{type(exc).__name__}: {exc}"
                    self.client = None
                raise

        def stop(self):
            with self.lock:
                if not self.running:
                    return False
                client = self.client
                self.running = False
                self.connected = False
                self.client = None
            try:
                client.disconnect()
            finally:
                client.loop_stop()
            return True

        def restart(self):
            self.stop()
            return self.start()

        def status(self) -> dict:
            with self.lock:
                return {
                    "running": self.running,
                    "connected": self.connected,
                    "connection_rc": self.connection_rc,
                    "client_id": MQTT_CLIENT_ID,
                    "broker": f"{MQTT_BROKER_HOST}:
{MQTT_BROKER_PORT}",
                    "source_topic": SOURCE_TOPIC,
                    "analysis_topic": ANALYSIS_TOPIC,
                    "risk_topic": RISK_TOPIC,
                    "processed_count": self.processed_count,
                    "duplicate_count": self.duplicate_count,
                    "error_count": self.error_count,
                    "last_source_timestamp":
self.last_source_timestamp,
                    "last_analysis_timestamp":
self.last_analysis_timestamp,
                    "previous_risk_state": self.previous_risk_state,
                    "last_error": self.last_error,
                }

    # Reuse a previous service instance when the cell is re-
executed, preventing
    # duplicate subscribers inside the same Python process.
    existing = globals().get("_CORROSION_BN_SERVICE")
    if existing is not None:
```

```
        try:
            existing.stop()
        except Exception:
            pass

    CORROSION_BN_SERVICE = CorrosionBNService()
    globals()["_CORROSION_BN_SERVICE"] = CORROSION_BN_SERVICE

    AUTOSTART_ERROR = None
    if AUTOSTART:
        try:
            CORROSION_BN_SERVICE.start()
        except Exception as exc:
            AUTOSTART_ERROR = f"{type(exc).__name__}: {exc}"

    return AUTOSTART_ERROR, CORROSION_BN_SERVICE, CorrosionBNService
```

- Cell 6 - Display the MQTT connection status and service statistics.

Show code

```
@app.cell
def _(AUTOSTART_ERROR, CORROSION_BN_SERVICE, mo, pd):
    """Service status and usage instructions."""
    status = CORROSION_BN_SERVICE.status()

    blocks = [
        mo.md("## MQTT analysis service"),
        pd.DataFrame([status]),
    ]
    if status.get("last_error"):
        blocks.append(mo.md(f"### Connection / processing
error\n`{status['last_error']}`"))
    elif status.get("running") and not status.get("connected"):
        blocks.append(mo.md("### Connecting\nThe MQTT client has
started but has not completed the broker connection yet."))

    if AUTOSTART_ERROR:
        blocks.append(
            mo.md(
                f"""
                ### Service not started

                `{AUTOSTART_ERROR}`

                Set the MQTT password in the Notebook environment
and run this cell again:
```

```
                ~~~python
                import os
                # MQTT credentials are already configured in this
notebook.
                ~~~
                """
            )
        )
    else:
        blocks.append(
            mo.md(
                """
                The service starts automatically and processes one
complete source
                snapshot for every new MQTT message. No manual
analysis trigger or SQL
                polling is required.

                Available controls in Python:

                ~~~python
                CORROSION_BN_SERVICE.status()
                CORROSION_BN_SERVICE.stop()
                CORROSION_BN_SERVICE.start()
                CORROSION_BN_SERVICE.restart()
                ~~~
                """
            )
        )

    mo.vstack(blocks)
    return status,
```

- Cell 7 - Display the most recent Bayesian analysis result.

Show code

```
@app.cell
def _(CORROSION_BN_SERVICE, mo):
    """Display the most recent analysis result when one is
available."""
    latest = CORROSION_BN_SERVICE.last_result
    if latest is None:
        mo.md("### Latest result\nNo source message has been
processed yet.")
    else:
```

```
        mo.vstack(
            [
                mo.md("### Latest source snapshot result"),
                mo.md(f"- Source timestamp:
`{latest['source_timestamp']}`"),
                mo.md(f"- Risk state: `{latest['risk_state']}`"),
                mo.md("#### Root states"),
                latest["root_states"],
                mo.md("#### Evidence"),
                latest["evidence"],
                mo.md("#### Corrosion analysis payload"),
                latest["analysis_payload"],
                mo.md("#### Corrosion risk payload"),
                latest["risk_payload"],
            ]
        )
    return


if __name__ == "__main__":
    app.run()
```

2 Run all cells and go to **UNS** to check the results.

Building Bow-tie App

1 In **Builder**, enter the application requirements in the dialog, and start building.

> ⚠ **What must be stated**
> Remember to tell the agent to use data from UNS. Specific topic paths are required; otherwise, the agent might have trouble displaying the correct data.

> 🚀 **In case you are building apps on another Tier0**
> Use a different **clientId** (add a different suffix) to connect to the UNS broker to avoid client ID conflicts.

Show Builder prompt

```
    Build a modern industrial Bow-Tie Analysis application for CDU
Atmospheric Overhead Corrosion.
```

Use the following UNS topics as the only data sources:

- Refinery/CDU_Plant/Metric/Corrosion_Monitoring
- Refinery/CDU_Plant/Metric/Corrosion_Analysis
- Refinery/CDU_Plant/State/Corrosion_Risk

The application must update automatically when new UNS messages arrive.

The application should help operators understand current corrosion risk, contributing threats, barrier conditions, and possible consequences.

Create a single-page dashboard with:

1. A Bow-Tie diagram as the main section:
- Threats on the left
- Preventative barriers between threats and the top event
- Corrosion as the central top event
- Mitigative barriers after the top event
- Consequences on the right

2. A prominent risk summary showing:
- Current risk state
- Model confidence
- Normal, developing, and confirmed probabilities
- Total corrosion risk

3. A compact live process data panel showing key values from Corrosion_Monitoring, including chloride, pH, wash water flow, desalter performance, wash water rate, total iron, SAP evidence, and injection quill status.

4. Consequence indicators for:
- Loss of primary containment
- Unplanned shutdown
- Escalation risk

5. Visual status rules:
- Green for normal or healthy conditions
- Amber/yellow for developing risk or warning conditions
- Red for confirmed risk, failed barriers, or severe consequences
- Gray when data is unavailable
- Blue only for interaction and informational highlights

6. Interaction:
- Selecting a threat, barrier, or consequence should show its current state, related process values, and a short explanation.
- Update the interface automatically when new UNS data arrives.

```
    - Show the latest data timestamp and connection/subscription status
from the built-in SDK.

    Design requirements:

    Keep the existing visual design language and styling throughout the
application.

    Use a modern industrial operations dashboard style:
    - Dark blue-gray color palette instead of pure black.
    - Layered panels with subtle gradients.
    - Card-based layout with large rounded corners, around 16-20px.
    - Thin borders used for separation instead of heavy shadows.
    - Soft depth and restrained glow effects.
    - Clean spacing with generous padding and consistent alignment.
    - IBM Plex Sans for body text.
    - Space Grotesk for headings.
    - Compact, information-dense cards with clear visual hierarchy.
    - Uppercase section labels with increased letter spacing.
    - Professional refinery / SCADA interface rather than business
analytics.
    - Flat illustrations and SVG graphics instead of skeuomorphic
elements.
    - Smooth, subtle transitions and hover effects.

    Overall appearance should feel clean, modern, technical, and
operational.

    Avoid:
    - Pure black backgrounds
    - Glassmorphism
    - Cyberpunk styling
    - Neon lighting
    - Overly decorative effects
    - Business analytics dashboard styling

    Use a clean, professional refinery operations style with clear
hierarchy, compact cards, readable labels, and responsive layout.

    Use the payload fields exactly as defined in the corresponding UNS
models. Do not invent additional fields, calculated values, or mock data.
```

2. Once the application is complete after certain rounds of refining, deploy it.

3. Go to **Launchpad**, open the application and check.

BEST PRACTICE

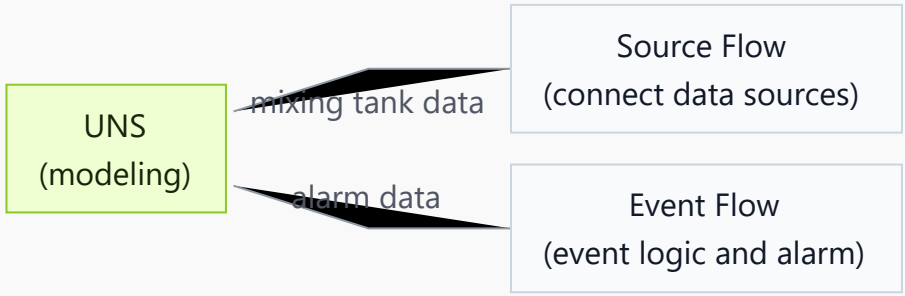
Building Shopfloor Workflow with Agent

Use the UNS agent to build workflows that cover data modeling, data connection, and data events.

UNS agent provides a faster and easier way to build factory data models, connect data from factories and create alarm events based on the data through conversation in natural language.

Workflow Background

A mixing tank mixes materials while heating them. Build a data model and collect temperature, water level, and heater status to determine whether the tank is overheated and trigger an alarm.



How to Build the Workflow

1. Start a conversation with the UNS agent in **UNS** with `full_access`.
2. Enter the prompt to build UNS models.

```
Create a data model representing the temperature, water level and heater status of a mixing tank, temperature and level in one metric topic and heater status in a state topic.
```

3. Once you confirm that the model is complete, enter the prompt and let the agent connect the corresponding data to the model.

```
Create a source flow to send data to these 2 topics, simulate reasonable
data.
```

When you have actual data sources
Tell the agent your data source information for connection.

4 Enter the prompt to create an event with the following logic.

- Low water level warning: Triggered when the water level is below 20% and the heater is on.
- Dry-heating alarm: Triggered when the water level is below 15%, temperature is above 90°C and the heater is on.

```
Create an Event Flow for the following alarm logic:
  - Warning: Trigger when level < 20 and heater_status = true.
  - Critical: Trigger when level < 15, temperature > 90, and heater_status =
true.
  - Clear the active alarm when level > 25 or heater_status = false.
Create a new topic under the existing UNS model to receive the alarm result.
Include the alarm level, message, active status, and timestamp in the
output.
```

5 Check the results on **UNS**.

Note
Continue talking to the agent after each step to refine the outcome.

BEST PRACTICE

Displaying Data with Digital Twin

View UNS data in 3D models and scenes in real time.

The digital twin feature in Tier0 is for you to view live UNS data on 3D models in lifelike scenes. In this section, we bring a workshop to life. It contains the following machines:

- 2 lathes
- 1 robot arm working between these lathes
- 1 inbound buffer station and 1 outbound buffer
- 1 quality checker
- 1 Unified Control System
- 2 raw material shelves and 2 product shelves
- 1 AGV

Collecting Data from Workshop

Collect all data to **UNS**, including data that controls machine movements and indicates machine status and the workshop environment.

1. Build data models in **UNS**.

Show UNS model JSON


```
{
"namespace": [
    {
    "name": "Metric",
    "alias": "metric_151a7edd68bf4269a229",
    "type": "PATH",
    "children": [
        {
        "name": "RobotArm",
        "alias": "Robot_0_9d5dc450dc1f4c0c9a86",
        "type": "TOPIC",
        "topicType": "METRIC",
        "extendProperties": {
            "J0_Cur": "0"
```

```
        },
        "fields": [
            {
            "name": "J1_Cur",
            "type": "FLOAT"
            },
            {
            "name": "J1_Init",
            "type": "FLOAT"
            },
            {
            "name": "J2_Cur",
            "type": "FLOAT"
            },
            {
            "name": "J2_Init",
            "type": "FLOAT"
            },
            {
            "name": "J3_Cur",
            "type": "FLOAT"
            },
            {
            "name": "J3_Init",
            "type": "FLOAT"
            },
            {
            "name": "J4_Cur",
            "type": "FLOAT"
            },
            {
            "name": "J4_Init",
            "type": "FLOAT"
            },
            {
            "name": "J5_Cur",
            "type": "FLOAT"
            },
            {
            "name": "J5_Init",
            "type": "FLOAT"
            },
            {
            "name": "J6_Cur",
            "type": "FLOAT"
            },
            {
            "name": "J6_Init",
            "type": "FLOAT"
            },
            {
```

```
            "name": "Model_Name",
            "type": "STRING"
            },
            {
            "name": "Max_Payload_kg",
            "type": "FLOAT"
            },
            {
            "name": "Reach_Radius_mm",
            "type": "FLOAT"
            },
            {
            "name": "Gripper_Force_N",
            "type": "FLOAT"
            }
        ],
        "enableHistory": "TRUE",
        "mockData": "FALSE"
        },
        {
        "name": "ProLathe-01",
        "alias": "ProLathe_ef325e0a9db047d6abac",
        "type": "TOPIC",
        "topicType": "METRIC",
        "fields": [
            {
            "name": "J_Changer_Cur",
            "type": "FLOAT"
            },
            {
            "name": "J_Door_Cur",
            "type": "FLOAT"
            },
            {
            "name": "J_Finger_Cur",
            "type": "FLOAT"
            },
            {
            "name": "J_Spindle_Cur",
            "type": "FLOAT"
            },
            {
            "name": "J_TailStock_Cur",
            "type": "FLOAT"
            },
            {
            "name": "J_X_Cur",
            "type": "FLOAT"
            },
            {
            "name": "J_Y_Cur",
```

```
            "type": "FLOAT"
            },
            {
            "name": "J_Z_Cur",
            "type": "FLOAT"
            },
            {
            "name": "Model_Name",
            "type": "STRING"
            },
            {
            "name": "Max_Placement_Rate_cph",
            "type": "INTEGER"
            },
            {
            "name": "Placement_Speed_cph",
            "type": "INTEGER"
            },
            {
            "name": "Feeder_Usage_pct",
            "type": "INTEGER"
            },
            {
            "name": "Machine_State_code",
            "type": "INTEGER"
            }
        ],
        "enableHistory": "FALSE",
        "mockData": "FALSE"
        },
        {
        "name": "ProLathe-02",
        "alias": "ProLathe2_ef325e0a9db047d6abac",
        "type": "TOPIC",
        "topicType": "METRIC",
        "fields": [
            {
            "name": "J_Changer_Cur",
            "type": "FLOAT"
            },
            {
            "name": "J_Door_Cur",
            "type": "FLOAT"
            },
            {
            "name": "J_Finger_Cur",
            "type": "FLOAT"
            },
            {
            "name": "J_Spindle_Cur",
            "type": "FLOAT"
```

```
        },
        {
        "name": "J_TailStock_Cur",
        "type": "FLOAT"
        },
        {
        "name": "J_X_Cur",
        "type": "FLOAT"
        },
        {
        "name": "J_Y_Cur",
        "type": "FLOAT"
        },
        {
        "name": "J_Z_Cur",
        "type": "FLOAT"
        },
        {
        "name": "Model_Name",
        "type": "STRING"
        },
        {
        "name": "Max_Placement_Rate_cph",
        "type": "INTEGER"
        },
        {
        "name": "Placement_Speed_cph",
        "type": "INTEGER"
        },
        {
        "name": "Feeder_Usage_pct",
        "type": "INTEGER"
        },
        {
        "name": "Machine_State_code",
        "type": "INTEGER"
        }
    ],
    "enableHistory": "FALSE",
    "mockData": "FALSE"
    },
    {
    "name": "AGV",
    "alias": "AGV_7eddee0054a0",
    "type": "TOPIC",
    "topicType": "METRIC",
    "fields": [
        {
        "name": "Position_X_Cur",
        "type": "FLOAT"
        },
```

```
{
"name": "Position_X_Init",
"type": "FLOAT"
},
{
"name": "Position_Y_Cur",
"type": "FLOAT"
},
{
"name": "Position_Y_Init",
"type": "FLOAT"
},
{
"name": "Rotation_Y_Cur",
"type": "FLOAT"
},
{
"name": "Rotation_Y_Init",
"type": "FLOAT"
},
{
"name": "Tray_Height_Cur",
"type": "FLOAT"
},
{
"name": "Tray_Height_Init",
"type": "FLOAT"
},
{
"name": "State_Cur",
"type": "STRING"
},
{
"name": "State_Init",
"type": "STRING"
},
{
"name": "Position_Z_Cur",
"type": "FLOAT"
},
{
"name": "Position_Z_Init",
"type": "FLOAT"
},
{
"name": "Model_Name",
"type": "STRING"
},
{
"name": "Max_Load_kg",
"type": "FLOAT"
```

```
        },
        {
        "name": "Task_Progress_pct",
        "type": "INTEGER"
        },
        {
        "name": "Remaining_Distance_m",
        "type": "FLOAT"
        },
        {
        "name": "AGV_State_code",
        "type": "INTEGER"
        }
    ],
    "enableHistory": "FALSE",
    "mockData": "FALSE"
    },
    {
    "name": "RobotArm-1",
    "alias": "RobotArm_1_b9f06e21346d",
    "type": "TOPIC",
    "topicType": "METRIC",
    "fields": [
        {
        "name": "J1_Cur",
        "type": "FLOAT"
        },
        {
        "name": "J1_Init",
        "type": "FLOAT"
        },
        {
        "name": "J2_Cur",
        "type": "FLOAT"
        },
        {
        "name": "J2_Init",
        "type": "FLOAT"
        },
        {
        "name": "J3_Cur",
        "type": "FLOAT"
        },
        {
        "name": "J3_Init",
        "type": "FLOAT"
        },
        {
        "name": "J4_Cur",
        "type": "FLOAT"
        },
```

```
            {
            "name": "J4_Init",
            "type": "FLOAT"
            },
            {
            "name": "J5_Cur",
            "type": "FLOAT"
            },
            {
            "name": "J5_Init",
            "type": "FLOAT"
            },
            {
            "name": "J6_Cur",
            "type": "FLOAT"
            },
            {
            "name": "J6_Init",
            "type": "FLOAT"
            }
        ],
        "enableHistory": "TRUE",
        "mockData": "FALSE"
        }
    ]
    },
    {
    "name": "State",
    "alias": "state_3912d0df3fee",
    "type": "PATH",
    "children": [
        {
        "name": "RawMaterialStorage",
        "alias": "RawMaterialS_c3307f680405",
        "type": "TOPIC",
        "topicType": "STATE",
        "fields": [
            {
            "name": "StockAmount",
            "type": "INTEGER"
            }
        ],
        "enableHistory": "FALSE",
        "mockData": "FALSE"
        },
        {
        "name": "ProductStorage",
        "alias": "ProductStora_3c145f140bd9",
        "type": "TOPIC",
        "topicType": "STATE",
        "fields": [
```

```
            {
            "name": "StockAmount",
            "type": "INTEGER"
            }
        ],
        "enableHistory": "FALSE",
        "mockData": "FALSE"
        },
        {
        "name": "InboundBuffer",
        "alias": "LoaderBuffer_9feece9b3231",
        "type": "TOPIC",
        "topicType": "STATE",
        "fields": [
            {
            "name": "Amount",
            "type": "INTEGER"
            }
        ],
        "enableHistory": "FALSE",
        "mockData": "FALSE"
        },
        {
        "name": "OutboundBuffer",
        "alias": "OutboundBuff_75433b98fde6",
        "type": "TOPIC",
        "topicType": "STATE",
        "fields": [
            {
            "name": "Amount",
            "type": "INTEGER"
            }
        ],
        "enableHistory": "FALSE",
        "mockData": "FALSE"
        }
    ]
    }
]
}
```


2 Use a **Source Flow** to connect data sources to UNS models.

- Robot Arm

Show Robot Arm flow JSON

```
[
    {
        "id": "03933a7eb1a7b992",
        "type": "function",
        "z": "8daeb61ebd9e5f52",
        "name": "Cache Data",
        "func": "const CONFIG = {\n    fields: ['Joint_1', 'Joint_2',
'Joint_3', 'Joint_4', 'Joint_5', 'Joint_6'],\n    payloadFields: {\n
value: 'CurrentValue',\n        initialValue: 'InitialValue',\n
name: 'Name',\n        timeStamp: 'timeStamp',\n        time_stamp:
'time_stamp',\n        quality: 'quality'\n    }\n};\n\nlet cache =
flow.get('dataCache') || { instant: {}, metadata: {} };\n\nconst
topicParts = msg.topic.split('/');\nconst fieldName =
topicParts[topicParts.length - 1];\n\nlet payload = msg.payload;\nif
(typeof payload === 'string') {\n    try {\n        payload =
JSON.parse(payload);\n    } catch (e) {\n        return null;\n
}\n}\n\nlet value = payload[CONFIG.payloadFields.value];\nif (value ===
undefined) return null;\n\nconst metadata = {\n    timeStamp:
payload[CONFIG.payloadFields.timeStamp],\n    time_stamp:
payload[CONFIG.payloadFields.time_stamp],\n    quality:
payload[CONFIG.payloadFields.quality],\n    name:
payload[CONFIG.payloadFields.name],\n    initialValue:
payload[CONFIG.payloadFields.initialValue]\n};\n\nif
(CONFIG.fields.includes(fieldName)) {\n    cache.instant[fieldName] =
value;\n    cache.metadata[fieldName] = metadata;\n} else {\n    return
null;\n}\n\nflow.set('dataCache', cache);\nreturn null;",
        "outputs": 0,
        "timeout": "",
        "noerr": 0,
        "initialize": "",
        "finalize": "",
        "libs": [],
        "x": 880,
        "y": 500,
        "wires": []
    },
    {
        "id": "0c4dbdcbe5e3db92",
        "type": "mqtt in",
        "z": "8daeb61ebd9e5f52",
        "name": "Subscribe Joint_1~6",
        "topic":
"FactorySim_Supos/Flow_Group__1/MechEquips/Transporter_Kawasaki_BX100N/Mo
tor_Joints/+",
        "qos": "1",
        "datatype": "auto",
        "broker": "c11decb07a68083d",
        "nl": false,
        "rap": true,
        "rh": 0,
```

```
        "inputs": 0,
        "x": 610,
        "y": 480,
        "wires": [
            [
                "03933a7eb1a7b992"
            ]
        ]
    },
    {
        "id": "ba42bca5aa9dc561",
        "type": "inject",
        "z": "8daeb61ebd9e5f52",
        "name": "100ms Timer",
        "props": [
            {
                "p": "payload"
            },
            {
                "vt": "str",
                "p": "topic"
            }
        ],
        "repeat": "0.1",
        "crontab": "",
        "once": true,
        "onceDelay": "0.1",
        "topic": "",
        "payload": "",
        "payloadType": "date",
        "x": 520,
        "y": 600,
        "wires": [
            [
                "0fc94449c377ad9a"
            ]
        ]
    },
    {
        "id": "0fc94449c377ad9a",
        "type": "function",
        "z": "8daeb61ebd9e5f52",
        "name": "Aggregate & Publish (Flat)",
        "func": "const CONFIG = {\n    fieldMapping: {\n
'Joint_1': 'J1',\n        'Joint_2': 'J2',\n        'Joint_3': 'J3',\n
'Joint_4': 'J4',\n        'Joint_5': 'J5',\n        'Joint_6': 'J6'\n
},\n    missingStrategy: 'lastValue',\n    outputTopic: 'robot/state',\n
windowMs: 100\n};\n\nconst FIELDS =
Object.keys(CONFIG.fieldMapping);\n\nlet cache = flow.get('dataCache') ||
{ instant: {}, metadata: {} };\nlet lastValues = flow.get('lastValues')
|| {};\nlet lastMetadata = flow.get('lastMetadata') || {};\n\nlet output
```

```
= {\n    timestamp: new Date().toISOString()\n};\n\nfor (const field of
FIELDS) {\n    const shortName = CONFIG.fieldMapping[field];\n    \n
if (cache.instant[field] !== undefined) {\n        output[shortName +
'_Cur'] = cache.instant[field];\n        output[shortName + '_Init'] =
cache.metadata[field] ? cache.metadata[field].initialValue : 0;\n
output[shortName + '_Qly'] = cache.metadata[field] ?
cache.metadata[field].quality : 0;\n        \n        lastValues[field] =
cache.instant[field];\n        lastMetadata[field] =
cache.metadata[field];\n    } else {\n        let fallbackValue =
CONFIG.missingStrategy === 'lastValue' \n            ? (lastValues[field]
!== undefined ? lastValues[field] : 0) \n            : 0;\n        \n
output[shortName + '_Cur'] = fallbackValue;\n        output[shortName +
'_Init'] = lastMetadata[field] ? lastMetadata[field].initialValue : 0;\n
output[shortName + '_Qly'] = -1;\n    }\n}\n\nmsg.payload =
output;\nmsg.topic = CONFIG.outputTopic;\n\nflow.set('dataCache', {
instant: {}, metadata: {} });\nflow.set('lastValues',
lastValues);\nflow.set('lastMetadata', lastMetadata);\n\nreturn msg;",
        "outputs": 1,
        "timeout": "",
        "noerr": 0,
        "initialize": "",
        "finalize": "",
        "libs": [],
        "x": 760,
        "y": 600,
        "wires": [
            [
                "c9eee2264fe78846",
                "a685279629e73d9a",
                "4ce075fda1d3d993",
                "8e8779918eab6f89"
            ]
        ]
    },
    {
        "id": "c9eee2264fe78846",
        "type": "mqtt out",
        "z": "8daeb61ebd9e5f52",
        "name": "Publish robot/state",
        "topic": "Metric/RobotArm",
        "qos": "1",
        "retain": "false",
        "respTopic": "",
        "contentType": "",
        "userProps": "",
        "correl": "",
        "expiry": "",
        "broker": "9836929a70c74929",
        "x": 1070,
        "y": 540,
        "wires": []
```

```
    },
    {
        "id": "c11decb07a68083d",
        "type": "mqtt-broker",
        "name": "supos-demo.supos.app",
        "broker": "supos-demo.supos.app",
        "port": 1883,
        "clientid": "",
        "autoConnect": true,
        "usetls": false,
        "protocolVersion": 4,
        "keepalive": 60,
        "cleansession": true,
        "autoUnsubscribe": true,
        "birthTopic": "",
        "birthQos": "0",
        "birthRetain": "false",
        "birthPayload": "",
        "birthMsg": {},
        "closeTopic": "",
        "closeQos": "0",
        "closeRetain": "false",
        "closePayload": "",
        "closeMsg": {},
        "willTopic": "",
        "willQos": "0",
        "willRetain": "false",
        "willPayload": "",
        "willMsg": {},
        "userProps": "",
        "sessionExpiry": ""
    },
    {
        "id": "9836929a70c74929",
        "type": "mqtt-broker",
        "z": "8daeb61ebd9e5f52",
        "name": "emqx",
        "broker": "mqtt.pre.tier0.dev",
        "port": "1883",
        "clientid": "3364626855530 88&336463047745872",
        "autoConnect": true,
        "usetls": false,
        "protocolVersion": "4",
        "keepalive": "60",
        "cleansession": true,
        "autoUnsubscribe": true,
        "birthTopic": "",
        "birthQos": "0",
        "birthRetain": "false",
        "birthPayload": "",
        "birthMsg": {},
```

```
        "closeTopic": "",
        "closeQos": "0",
        "closeRetain": "false",
        "closePayload": "",
        "closeMsg": {},
        "willTopic": "",
        "willQos": "0",
        "willRetain": "false",
        "willPayload": "",
        "willMsg": {},
        "userProps": "",
        "sessionExpiry": ""
    }
]
```

- AGV

Show AGV flow JSON

```
[
    {
        "id": "fa46cf31be8d751e",
        "type": "tab",
        "label": "Metric/AGV",
        "disabled": false,
        "info": ""
    },
    {
        "id": "1f0e9d8c7b6a5a41",
        "type": "mqtt in",
        "z": "fa46cf31be8d751e",
        "name": "Subscribe AGV telemetry",
        "topic":
"FactorySim_Supos/Flow_Group__1/MechEquips/AGV_zOnron/Motor_Joints/+",
        "qos": "1",
        "datatype": "auto",
        "broker": "c11decb07a68083d",
        "nl": false,
        "rap": true,
        "rh": 0,
        "inputs": 0,
        "x": 190,
        "y": 380,
        "wires": [
            [
                "2a1b3c4d5e6f7081"
            ]
```

```
        ]
    },
    {
        "id": "2a1b3c4d5e6f7081",
        "type": "function",
        "z": "fa46cf31be8d751e",
        "name": "Cache AGV telemetry",
        "func": "const CONFIG = {\n    fields: [\n        'Position_X',\n
'Position_Y',\n        'Position_Z',\n        'Rotation_Y',\n
'Tray_Height',\n        'State'\n    ],\n    payloadFields: {\n
value: 'CurrentValue',\n        initialValue: 'InitialValue',\n
name: 'Name',\n        timeStamp: 'timeStamp',\n        timeStampAlt:
'time_stamp'\n    }\n};\n\nlet cache = flow.get('agvDataCache') || {
instant: {}, metadata: {} };\n\nconst topicParts = (msg.topic ||
'').split('/');\nconst fieldName = topicParts[topicParts.length - 1];\nif
(!CONFIG.fields.includes(fieldName)) {\n    return null;\n}\n\nlet
payload = msg.payload;\nif (typeof payload === 'string') {\n    try {\n
payload = JSON.parse(payload);\n    } catch (error) {\n        return
null;\n    }\n}\n\nif (typeof payload !== 'object' || payload === null)
{\n    return null;\n}\n\nconst currentValue =
payload[CONFIG.payloadFields.value];\nif (currentValue === undefined) {\n
return null;\n}\n\ncache.instant[fieldName] =
currentValue;\ncache.metadata[fieldName] = {\n    name:
payload[CONFIG.payloadFields.name] || fieldName,\n    initialValue:
payload[CONFIG.payloadFields.initialValue],\n    timeStamp:
payload[CONFIG.payloadFields.timeStampAlt] ??
payload[CONFIG.payloadFields.timeStamp] ??
0\n};\n\nflow.set('agvDataCache', cache);\nreturn null;",
        "outputs": 0,
        "timeout": "",
        "noerr": 0,
        "initialize": "",
        "finalize": "",
        "libs": [],
        "x": 470,
        "y": 380,
        "wires": []
    },
    {
        "id": "3b2c1d0e9f8a7b61",
        "type": "inject",
        "z": "fa46cf31be8d751e",
        "name": "100ms Timer",
        "props": [
            {
                "p": "payload"
            },
            {
                "p": "topic",
                "vt": "str"
            }
```

```
        ],
        "repeat": "0.1",
        "crontab": "",
        "once": true,
        "onceDelay": "0.1",
        "topic": "",
        "payload": "",
        "payloadType": "date",
        "x": 150,
        "y": 500,
        "wires": [
            [
                "4c3d2e1f0a9b8c71"
            ]
        ]
    },
    {
        "id": "4c3d2e1f0a9b8c71",
        "type": "function",
        "z": "fa46cf31be8d751e",
        "name": "Aggregate AGV UNS payload",
        "func": "const CONFIG = {\n    fields: [\n        'Position_X',\n
'Position_Y',\n        'Position_Z',\n        'Rotation_Y',\n
'Tray_Height',\n        'State'\n    ],\n    outputTopic: 'Metric/AGV',\n
defaultNumeric: 0,\n    defaultString: ''\n};\n\nlet cache =
flow.get('agvDataCache') || { instant: {}, metadata: {} };\nlet
lastValues = flow.get('agvLastValues') || {};\nlet lastMetadata =
flow.get('agvLastMetadata') || {};\nlet lastTimestamp =
flow.get('agvLastTimestamp') || 0;\n\nlet output = {};\nlet
candidateTimestamps = [];\n\nfor (const field of CONFIG.fields) {\n
const hasCurrentFrameValue =
Object.prototype.hasOwnProperty.call(cache.instant, field);\n    const
metadata = hasCurrentFrameValue ? cache.metadata[field] :
lastMetadata[field];\n    const currentValue = hasCurrentFrameValue ?
cache.instant[field] : lastValues[field];\n    const isStringField =
field === 'State';\n\n    output[field + '_Cur'] = currentValue !==
undefined\n        ? currentValue\n        : (isStringField ?
CONFIG.defaultString : CONFIG.defaultNumeric);\n    output[field +
'_Init'] = metadata && metadata.initialValue !== undefined\n        ?
metadata.initialValue\n        : (isStringField ? CONFIG.defaultString :
CONFIG.defaultNumeric);\n\n    if (hasCurrentFrameValue) {\n
lastValues[field] = cache.instant[field];\n        lastMetadata[field] =
cache.metadata[field];\n\n        const rawTime = cache.metadata[field] ?
Number(cache.metadata[field].timeStamp) : NaN;\n        if
(!Number.isNaN(rawTime)) {\n
candidateTimestamps.push(rawTime);\n        }\n    }\n}\n\nif
(candidateTimestamps.length > 0) {\n    lastTimestamp =
Math.max(...candidateTimestamps);\n}\n\noutput.timestamp =
lastTimestamp;\n\nmsg.topic = CONFIG.outputTopic;\nmsg.payload =
output;\n\nflow.set('agvDataCache', { instant: {}, metadata: {}
});\nflow.set('agvLastValues', lastValues);\nflow.set('agvLastMetadata',
```

```
lastMetadata);\nflow.set('agvLastTimestamp', lastTimestamp);\n\nreturn
msg;",
        "outputs": 1,
        "timeout": "",
        "noerr": 0,
        "initialize": "",
        "finalize": "",
        "libs": [],
        "x": 460,
        "y": 500,
        "wires": [
            [
                "6d5e4f3a2b1c0d91",
                "7c6b5a4d3e2f1098"
            ]
        ]
    },
    {
        "id": "6d5e4f3a2b1c0d91",
        "type": "mqtt out",
        "z": "fa46cf31be8d751e",
        "name": "Publish Metric/AGV",
        "topic": "Metric/AGV",
        "qos": "1",
        "retain": "false",
        "respTopic": "",
        "contentType": "",
        "userProps": "",
        "correl": "",
        "expiry": "",
        "broker": "f2d431c2d2264341",
        "x": 710,
        "y": 400,
        "wires": []
    },
    {
        "id": "7c6b5a4d3e2f1098",
        "type": "debug",
        "z": "fa46cf31be8d751e",
        "name": "Debug AGV payload",
        "active": true,
        "tosidebar": true,
        "console": false,
        "complete": "payload",
        "targetType": "msg",
        "statusVal": "",
        "statusType": "auto",
        "x": 740,
        "y": 580,
        "wires": []
    },
```

```
    {
        "id": "c11decb07a68083d",
        "type": "mqtt-broker",
        "name": "supos-demo.supos.app",
        "broker": "supos-demo.supos.app",
        "port": 1883,
        "clientid": "",
        "autoConnect": true,
        "usetls": false,
        "protocolVersion": 4,
        "keepalive": 60,
        "cleansession": true,
        "autoUnsubscribe": true,
        "birthTopic": "",
        "birthQos": "0",
        "birthRetain": "false",
        "birthPayload": "",
        "birthMsg": {},
        "closeTopic": "",
        "closeQos": "0",
        "closeRetain": "false",
        "closePayload": "",
        "closeMsg": {},
        "willTopic": "",
        "willQos": "0",
        "willRetain": "false",
        "willPayload": "",
        "willMsg": {},
        "userProps": "",
        "sessionExpiry": ""
    },
    {
        "id": "f2d431c2d2264341",
        "type": "mqtt-broker",
        "z": "fa46cf31be8d751e",
        "name": "emqx",
        "broker": "mqtt.pre.tier0.dev",
        "port": "1883",
        "clientid": "336462685553088&341668146888752",
        "autoConnect": true,
        "usetls": false,
        "protocolVersion": "4",
        "keepalive": "60",
        "cleansession": true,
        "autoUnsubscribe": true,
        "birthTopic": "",
        "birthQos": "0",
        "birthRetain": "false",
        "birthPayload": "",
        "birthMsg": {},
        "closeTopic": "",
```

```
        "closeQos": "0",
        "closeRetain": "false",
        "closePayload": "",
        "closeMsg": {},
        "willTopic": "",
        "willQos": "0",
        "willRetain": "false",
        "willPayload": "",
        "willMsg": {},
        "userProps": "",
        "sessionExpiry": ""
    }
]
```

- Lathe 1

Show Lathe 1 flow JSON



```
[
    {
        "id": "88fed08889390f28",
        "type": "tab",
        "label": "Metric/ProLathe-01",
        "disabled": false,
        "info": ""
    },
    {
        "id": "f2bae08de3b55bb0",
        "type": "mqtt in",
        "z": "88fed08889390f28",
        "name": "Subscribe ProLathe joints",
        "topic":
"FactorySim_Supos/Flow_Group__1/MechEquips/porcess_prolathe_gs_260ms_1/Mo
tor_Joints/+",
        "qos": "1",
        "datatype": "auto",
        "broker": "6f8ade8021b85933",
        "nl": false,
        "rap": true,
        "rh": 0,
        "inputs": 0,
        "x": 590,
        "y": 340,
        "wires": [
            [
                "ac84eab3cf5bb664",
                "8a8a5572465dac5b"
```

```
            ]
        ]
    },
    {
        "id": "ac84eab3cf5bb664",
        "type": "function",
        "z": "88fed08889390f28",
        "name": "Cache Data",
        "func": "const CONFIG = {\n    fields: [\"JointChanger\",
\"JointDoors\", \"JointFinger\", \"JointSpindle\", \"JointTailStock\",
\"Joint_X\", \"Joint_Y\", \"Joint_Z\"],\n    payloadFields: {\n
value: 'CurrentValue',\n        initialValue: 'InitialValue',\n
name: 'Name',\n        timeStamp: 'timeStamp',\n        time_stamp:
'time_stamp',\n        quality: 'quality'\n    }\n};\n\nlet cache =
flow.get('dataCache') || { instant: {}, metadata: {} };\n\nconst
topicParts = msg.topic.split('/');\nconst fieldName =
topicParts[topicParts.length - 1];\n\nlet payload = msg.payload;\nif
(typeof payload === 'string') {\n    try {\n        payload =
JSON.parse(payload);\n    } catch (e) {\n        return null;\n
}\n}\n\nlet value = payload[CONFIG.payloadFields.value];\nif (value ===
undefined) return null;\n\nconst metadata = {\n    timeStamp:
payload[CONFIG.payloadFields.timeStamp],\n    time_stamp:
payload[CONFIG.payloadFields.time_stamp],\n    quality:
payload[CONFIG.payloadFields.quality],\n    name:
payload[CONFIG.payloadFields.name],\n    initialValue:
payload[CONFIG.payloadFields.initialValue]\n};\n\nif
(CONFIG.fields.includes(fieldName)) {\n    cache.instant[fieldName] =
value;\n    cache.metadata[fieldName] = metadata;\n} else {\n    return
null;\n}\n\nflow.set('dataCache', cache);\nreturn null;",
        "outputs": 0,
        "timeout": "",
        "noerr": 0,
        "initialize": "",
        "finalize": "",
        "libs": [],
        "x": 860,
        "y": 360,
        "wires": []
    },
    {
        "id": "ca4b6b2146df5093",
        "type": "inject",
        "z": "88fed08889390f28",
        "name": "100ms Timer",
        "props": [
            {
                "p": "payload"
            },
            {
                "vt": "str",
                "p": "topic"
```

```
            }
        ],
        "repeat": "0.1",
        "crontab": "",
        "once": true,
        "onceDelay": "0.1",
        "topic": "",
        "payload": "",
        "payloadType": "date",
        "x": 500,
        "y": 460,
        "wires": [
            [
                "e47cbdc7b7dae23f"
            ]
        ]
    },
    {
        "id": "e47cbdc7b7dae23f",
        "type": "function",
        "z": "88fed08889390f28",
        "name": "Aggregate & Publish (Flat)",
        "func": "const CONFIG = {\n    fieldMapping: {\n
\"JointChanger\": \"J_Changer\",\n    \"JointDoors\": \"J_Door\",\n
\"JointFinger\": \"J_Finger\",\n    \"JointSpindle\": \"J_Spindle\",\n
\"JointTailStock\": \"J_TailStock\",\n    \"Joint_X\": \"J_X\",\n
\"Joint_Y\": \"J_Y\",\n    \"Joint_Z\": \"J_Z\"\n},\n    missingStrategy:
'lastValue',\n    outputTopic: \"Metric/porcess_prolathe_gs_260ms_1\",\n
windowMs: 100\n};\n\nconst FIELDS =
Object.keys(CONFIG.fieldMapping);\n\nlet cache = flow.get('dataCache') ||
{ instant: {}, metadata: {} };\nlet lastValues = flow.get('lastValues')
|| {};\nlet lastMetadata = flow.get('lastMetadata') || {};\nlet output =
{};\n\noutput.timestamp = new Date().toISOString();\n\nfor (const field
of FIELDS) {\n    const shortName = CONFIG.fieldMapping[field];\n\n    if
(cache.instant[field] !== undefined) {\n        output[shortName +
'_Cur'] = Number(cache.instant[field]);\n        output[shortName +
'_Init'] = cache.metadata[field] && cache.metadata[field].initialValue
!== undefined\n            ? Number(cache.metadata[field].initialValue)\n
: 0;\n        lastValues[field] = cache.instant[field];\n
lastMetadata[field] = cache.metadata[field] || {};\n    } else {\n
let fallbackValue = CONFIG.missingStrategy === 'lastValue'\n            ?
(lastValues[field] !== undefined ? lastValues[field] : 0)\n            :
0;\n        output[shortName + '_Cur'] = Number(fallbackValue);\n
output[shortName + '_Init'] = lastMetadata[field] &&
lastMetadata[field].initialValue !== undefined\n            ?
Number(lastMetadata[field].initialValue)\n            : 0;\n
}\n}\n\nmsg.payload = output;\nmsg.topic =
CONFIG.outputTopic;\n\nflow.set('dataCache', { instant: {}, metadata: {}
});\nflow.set('lastValues', lastValues);\nflow.set('lastMetadata',
lastMetadata);\n\nreturn msg;",
        "outputs": 1,
```

```
        "timeout": "",
        "noerr": 0,
        "initialize": "",
        "finalize": "",
        "libs": [],
        "x": 740,
        "y": 460,
        "wires": [
            [
                "ad22bd2f8b228324"
            ]
        ]
    },
    {
        "id": "ad22bd2f8b228324",
        "type": "mqtt out",
        "z": "88fed08889390f28",
        "name": "Metric/Prolathe",
        "topic": "Metric/ProLathe-01",
        "qos": "1",
        "retain": "false",
        "respTopic": "",
        "contentType": "",
        "userProps": "",
        "correl": "",
        "expiry": "",
        "broker": "c14ef68b0e3b48ac",
        "x": 1060,
        "y": 360,
        "wires": []
    },
    {
        "id": "8a8a5572465dac5b",
        "type": "debug",
        "z": "88fed08889390f28",
        "name": "Debug Output",
        "active": true,
        "tosidebar": true,
        "console": false,
        "complete": "payload",
        "statusVal": "",
        "statusType": "auto",
        "x": 980,
        "y": 540,
        "wires": []
    },
    {
        "id": "6f8ade8021b85933",
        "type": "mqtt-broker",
        "name": "supos-demo.supos.app",
        "broker": "supos-demo.supos.app",
```

```
        "port": 1883,
        "clientid": "",
        "autoConnect": true,
        "usetls": false,
        "protocolVersion": 4,
        "keepalive": 60,
        "cleansession": true,
        "autoUnsubscribe": true,
        "birthTopic": "",
        "birthQos": "0",
        "birthRetain": "false",
        "birthPayload": "",
        "birthMsg": {},
        "closeTopic": "",
        "closeQos": "0",
        "closeRetain": "false",
        "closePayload": "",
        "closeMsg": {},
        "willTopic": "",
        "willQos": "0",
        "willRetain": "false",
        "willPayload": "",
        "willMsg": {},
        "userProps": "",
        "sessionExpiry": ""
    },
    {
        "id": "c14ef68b0e3b48ac",
        "type": "mqtt-broker",
        "z": "88fed08889390f28",
        "name": "emqx",
        "broker": "mqtt.pre.tier0.dev",
        "port": "1883",
        "clientid": "3364626855530 88&337035452420768",
        "autoConnect": true,
        "usetls": false,
        "protocolVersion": "4",
        "keepalive": "60",
        "cleansession": true,
        "autoUnsubscribe": true,
        "birthTopic": "",
        "birthQos": "0",
        "birthRetain": "false",
        "birthPayload": "",
        "birthMsg": {},
        "closeTopic": "",
        "closeQos": "0",
        "closeRetain": "false",
        "closePayload": "",
        "closeMsg": {},
        "willTopic": "",
```

```
        "willQos": "0",
        "willRetain": "false",
        "willPayload": "",
        "willMsg": {},
        "userProps": "",
        "sessionExpiry": ""
    }
]
```


- Lathe 2

Show Lathe 2 flow JSON


```
[
    {
        "id": "c974bfc5fbc79b0e",
        "type": "tab",
        "label": "Metric/ProLathe-02",
        "disabled": false,
        "info": ""
    },
    {
        "id": "2ff367f08cbe470f",
        "type": "mqtt in",
        "z": "c974bfc5fbc79b0e",
        "name": "Subscribe ProLathe joints",
        "topic":
"FactorySim_Supos/Flow_Group__1/MechEquips/porcess_prolathe_gs_260ms_2/Mo
tor_Joints/+",
        "qos": "1",
        "datatype": "auto",
        "broker": "6f8ade8021b85933",
        "nl": false,
        "rap": true,
        "rh": 0,
        "inputs": 0,
        "x": 590,
        "y": 340,
        "wires": [
            [
                "7c54f7852fb5430e"
            ]
        ]
    },
    {
        "id": "7c54f7852fb5430e",
        "type": "function",
```

```
        "z": "c974bfc5fbc79b0e",
        "name": "Cache Data",
        "func": "const CONFIG = {\n    fields: [\"JointChanger\",
\"JointDoors\", \"JointFinger\", \"JointSpindle\", \"JointTailStock\",
\"Joint_X\", \"Joint_Y\", \"Joint_Z\"],\n    payloadFields: {\n
value: 'CurrentValue',\n        initialValue: 'InitialValue',\n
name: 'Name',\n        timeStamp: 'timeStamp',\n        time_stamp:
'time_stamp',\n        quality: 'quality'\n    }\n};\n\nlet cache =
flow.get('dataCache') || { instant: {}, metadata: {} };\n\nconst
topicParts = msg.topic.split('/');\nconst fieldName =
topicParts[topicParts.length - 1];\n\nlet payload = msg.payload;\nif
(typeof payload === 'string') {\n    try {\n        payload =
JSON.parse(payload);\n    } catch (e) {\n        return null;\n
}\n}\n\nlet value = payload[CONFIG.payloadFields.value];\nif (value ===
undefined) return null;\n\nconst metadata = {\n    timeStamp:
payload[CONFIG.payloadFields.timeStamp],\n    time_stamp:
payload[CONFIG.payloadFields.time_stamp],\n    quality:
payload[CONFIG.payloadFields.quality],\n    name:
payload[CONFIG.payloadFields.name],\n    initialValue:
payload[CONFIG.payloadFields.initialValue]\n};\n\nif
(CONFIG.fields.includes(fieldName)) {\n    cache.instant[fieldName] =
value;\n    cache.metadata[fieldName] = metadata;\n} else {\n    return
null;\n}\n\nflow.set('dataCache', cache);\nreturn null;",
        "outputs": 0,
        "timeout": "",
        "noerr": 0,
        "initialize": "",
        "finalize": "",
        "libs": [],
        "x": 860,
        "y": 360,
        "wires": []
    },
    {
        "id": "a69b0f994c614213",
        "type": "inject",
        "z": "c974bfc5fbc79b0e",
        "name": "100ms Timer",
        "props": [
            {
                "p": "payload"
            },
            {
                "vt": "str",
                "p": "topic"
            }
        ],
        "repeat": "0.1",
        "crontab": "",
        "once": true,
        "onceDelay": "0.1",
```

```
        "topic": "",
        "payload": "",
        "payloadType": "date",
        "x": 500,
        "y": 460,
        "wires": [
            [
                "a182dc40725e4801"
            ]
        ]
    },
    {
        "id": "a182dc40725e4801",
        "type": "function",
        "z": "c974bfc5fbc79b0e",
        "name": "Aggregate & Publish (Flat)",
        "func": "const CONFIG = {\n    fieldMapping: {\n
\"JointChanger\": \"J_Changer\",\n    \"JointDoors\": \"J_Door\",\n
\"JointFinger\": \"J_Finger\",\n    \"JointSpindle\": \"J_Spindle\",\n
\"JointTailStock\": \"J_TailStock\",\n    \"Joint_X\": \"J_X\",\n
\"Joint_Y\": \"J_Y\",\n    \"Joint_Z\": \"J_Z\"\n},\n    missingStrategy:
'lastValue',\n    outputTopic: \"Metric/porcess_prolathe_gs_260ms_2\",\n
windowMs: 100\n};\n\nconst FIELDS =
Object.keys(CONFIG.fieldMapping);\n\nlet cache = flow.get('dataCache') ||
{ instant: {}, metadata: {} };\nlet lastValues = flow.get('lastValues')
|| {};\nlet lastMetadata = flow.get('lastMetadata') || {};\nlet output =
{};\n\noutput.timestamp = new Date().toISOString();\n\nfor (const field
of FIELDS) {\n    const shortName = CONFIG.fieldMapping[field];\n\n    if
(cache.instant[field] !== undefined) {\n        output[shortName +
'_Cur'] = Number(cache.instant[field]);\n        output[shortName +
'_Init'] = cache.metadata[field] && cache.metadata[field].initialValue
!== undefined\n            ? Number(cache.metadata[field].initialValue)\n
: 0;\n        lastValues[field] = cache.instant[field];\n
lastMetadata[field] = cache.metadata[field] || {};\n    } else {\n
let fallbackValue = CONFIG.missingStrategy === 'lastValue'\n            ?
(lastValues[field] !== undefined ? lastValues[field] : 0)\n            :
0;\n        output[shortName + '_Cur'] = Number(fallbackValue);\n
output[shortName + '_Init'] = lastMetadata[field] &&
lastMetadata[field].initialValue !== undefined\n            ?
Number(lastMetadata[field].initialValue)\n            : 0;\n
}\n}\n\nmsg.payload = output;\n\nmsg.payload.Model_Name =
\"Turbo\";\nmsg.payload.Max_Carry_Weight_kg = 15.0;\n\nmsg.topic =
CONFIG.outputTopic;\n\nflow.set('dataCache', { instant: {}, metadata: {}
});\nflow.set('lastValues', lastValues);\nflow.set('lastMetadata',
lastMetadata);\n\nreturn msg;",
        "outputs": 1,
        "timeout": "",
        "noerr": 0,
        "initialize": "",
        "finalize": "",
        "libs": [],
```

```
        "x": 740,
        "y": 460,
        "wires": [
            [
                "b4e1817aa3744ab9",
                "84c88149d8d24eae"
            ]
        ]
    },
    {
        "id": "b4e1817aa3744ab9",
        "type": "mqtt out",
        "z": "c974bfc5fbc79b0e",
        "name": "Metric/Prolathe2",
        "topic": "Metric/ProLathe-02",
        "qos": "1",
        "retain": "false",
        "respTopic": "",
        "contentType": "",
        "userProps": "",
        "correl": "",
        "expiry": "",
        "broker": "6befe0f78bf64759",
        "x": 1060,
        "y": 360,
        "wires": []
    },
    {
        "id": "84c88149d8d24eae",
        "type": "debug",
        "z": "c974bfc5fbc79b0e",
        "name": "Debug Output",
        "active": true,
        "tosidebar": true,
        "console": false,
        "complete": "payload",
        "statusVal": "",
        "statusType": "auto",
        "x": 980,
        "y": 540,
        "wires": []
    },
    {
        "id": "6f8ade8021b85933",
        "type": "mqtt-broker",
        "name": "supos-demo.supos.app",
        "broker": "supos-demo.supos.app",
        "port": 1883,
        "clientid": "",
        "autoConnect": true,
        "usetls": false,
```

```
        "protocolVersion": 4,
        "keepalive": 60,
        "cleansession": true,
        "autoUnsubscribe": true,
        "birthTopic": "",
        "birthQos": "0",
        "birthRetain": "false",
        "birthPayload": "",
        "birthMsg": {},
        "closeTopic": "",
        "closeQos": "0",
        "closeRetain": "false",
        "closePayload": "",
        "closeMsg": {},
        "willTopic": "",
        "willQos": "0",
        "willRetain": "false",
        "willPayload": "",
        "willMsg": {},
        "userProps": "",
        "sessionExpiry": ""
    },
    {
        "id": "6befe0f78bf64759",
        "type": "mqtt-broker",
        "z": "c974bfc5fbc79b0e",
        "name": "emqx",
        "broker": "mqtt.pre.tier0.dev",
        "port": "1883",
        "clientid": "336462685553088&338098843755632",
        "autoConnect": true,
        "usetls": false,
        "protocolVersion": "4",
        "keepalive": "60",
        "cleansession": true,
        "autoUnsubscribe": true,
        "birthTopic": "",
        "birthQos": "0",
        "birthRetain": "false",
        "birthPayload": "",
        "birthMsg": {},
        "closeTopic": "",
        "closeQos": "0",
        "closeRetain": "false",
        "closePayload": "",
        "closeMsg": {},
        "willTopic": "",
        "willQos": "0",
        "willRetain": "false",
        "willPayload": "",
        "willMsg": {},
```

```
        "userProps": "",
        "sessionExpiry": ""
    }
]
```

Visualizing Workshop Scene

1. In **Model**, import models exist in the workshop, map UNS data to and create instances for them.
2. Go to **Scene** and add instances in the workshop to a scene from **Instance Library**.
3. Move models to their positions in the workshop and set environment lighting and other details.

Reference

API Reference

API integration essentials and a machine-readable OpenAPI reference for Tier0.

Starting Up

- Use `https://<your-tier0-host>/openapi/v1` as the base URL.
- Send `x-api-key` to OpenAPI endpoints except for `/assets/files/public`. Any files marked as `public` can be downloaded without an API key.
- Call `POST /auth/whoami` first to confirm the API key identity and effective permissions.

> **OpenAPI Reference**
> Open the interactive **API Reference** for request schemas, response schemas, and executable requests.

Handling Results

UNS batch operations can partially fail even with HTTP `200` results. Inspect every result before assuming success.

Custom HTTP In Routes

For APIs developed inside **Source Flow** and **Event Flow**, their input and response are defined by the deployed flow.

- **Source Flow** uses `/flow/source/` and is public by default. Add ingress or flow-level access control before exposing sensitive behavior.
- **Event Flow** uses `/flow/event/` and requires `x-api-key` with `flow.read` permission.

> **Example custom API in Source Flow or Event Flow**
> For example, API URL is `/integrations/echo` and you need an `HTTP In` node to develop the API:
>
> - Route in `HTTP In` is `/integrations/echo`, Tier0 will complete the rest.

- External call URL is `https://tier0.example.com/flow/source/integrations/echo` or `https://tier0.example.com/flow/event/integrations/echo`.

Troubleshooting

- `401` / `403`: Check the API key, scope, and owner permissions.
- `404`: Check the HTTP method, route prefix, node URL, and deployment.
- `No response`: Make sure every flow path reaches an `HTTP Response` node without discarding the original request and response properties.

REFERENCE

CLI Commands Reference

CLI and agent skill command reference.

Search commands, flags, or examples...

Global flags These flags can be used with any `tier0` command.

`--json` Output raw JSON `--debug` Print HTTP request/response details

Basic operations

`tier0 doctor`

Diagnose local Tier0 CLI connectivity and auth

Usage

```
tier0 doctor
```

Notes

- Checks local configuration, OpenAPI connectivity, API key presence, and API key identity.

Example

```
tier0 doctor
```

`tier0 config`

View or update configuration

Usage

```
tier0 config
```

Flags

`--base-url`
Set the platform base URL

`--api-key`
Set the API key (alternative to tier0 login)

`--lang`
Deprecated compatibility flag; CLI output is English-only
deprecated

Examples

```
tier0 config
```

```
tier0 config --base-url https://tier0.dev
```

```
tier0 config --api-key sk-per-xxxxxx
```

`tier0 login`

Authenticate via Device Flow

Usage

```
tier0 login
```

Flags

`--no-wait`

Print the authorization URL and exit without polling

`--setup-code`

Poll for an existing setup code instead of creating a new one

`--base-url`

Override the platform base URL

Notes

- Without `--no-wait`, the CLI polls every 5 seconds for up to 10 minutes.
- With `--no-wait` `--json`, the CLI returns `verification_url`, `setup_code`, and `expires_in`.

Examples

```
tier0 login
```

```
tier0 login --no-wait --json
```

```
tier0 login --setup-code <setup_code>
```

`tier0 api`

Call an API endpoint directly

Usage

```
tier0 api <endpoint>
```

Flags

`--body`

Request body JSON string

`--body-file`

Read request body from file

`--method`

HTTP method (GET|POST|PUT|DELETE)
default: `POST`

`--dry-run`

Preview the request without executing it

Notes

- `--body` and `--body-file` are mutually exclusive.

Examples

```
tier0 api /openapi/v1/uns/browse --body '{"path":"/"}'
```

```
tier0 api /openapi/v1/uns/read --body '{"topics":["demo"]}'
```

```
tier0 api /openapi/v1/uns/write --body-file body.json
```

`tier0 upgrade`

Upgrade CLI to the latest version

Usage

```
tier0 upgrade
```

Flags

`--dry-run`

Check for updates without installing

`-v, --version`

Upgrade to a specific version (default: latest)

Examples

```
tier0 upgrade
```

```
tier0 upgrade --dry-run
```

`tier0 uninstall`

Uninstall tier0 CLI

Usage

```
tier0 uninstall
```

Flags

`--purge`

Also delete config file (credentials)

`--remove-skills`

Also remove the Tier0 Skill from detected AI agents

`--keep-skills`

Keep agent skills (deprecated; this is now the default)
deprecated

Notes

- Agent Skills and config are kept by default.
- The local bundled skills directory is removed during CLI uninstall.

Examples

```
tier0 uninstall
```

```
tier0 uninstall --purge
```

```
tier0 uninstall --remove-skills
```

```
tier0 uninstall --purge --remove-skills
```

`tier0 version`

Show version

Usage

```
tier0 version
```

Example

```
tier0 version
```

Skills

`tier0 skills install`

Install or repair the Skill embedded in this CLI

Usage

```
tier0 skills install
```

Flags

`--force`
Replace the active Skill with the embedded baseline

`--no-sync`
Install locally without syncing detected AI agents

Examples

```
tier0 skills install
```

```
tier0 skills install --force
```

`tier0 skills list`

List installed skills

alias: ls

Usage

```
tier0 skills list
```

Example

```
tier0 skills list
```

`tier0 skills update`

Upgrade skills to the latest version

alias: upgrade

Usage

```
tier0 skills update
```

Flags

`--dry-run`
Check for updates without installing

Examples

```
tier0 skills update
```

```
tier0 skills update --dry-run
```

`tier0 skills version`

Show skills version info

alias: ver

Usage

```
tier0 skills version
```

Example

```
tier0 skills version
```

`tier0 skills status`

Show active and embedded Skill status

Usage

```
tier0 skills status
```

Example

```
tier0 skills status
```

`tier0 skills sync`

Sync the active Skill to detected AI agents

Usage

```
tier0 skills sync
```

Example

```
```

```
tier0 skills sync
```

Auth

`tier0 auth whoami`

Show current API key identity

Usage

```
tier0 auth whoami
```

Notes

- Shows user, workspace, key name, key type, roles, and permissions.

Examples

```
tier0 auth whoami
```

```
tier0 auth whoami --json
```

UNS

`tier0 uns browse`

Browse UNS namespace tree

Usage

```
tier0 uns browse
```

Flags

`-p, --path`

Path to browse in the UNS tree
default: `/`

`-d, --max-depth`

Max recursion depth (0 = unlimited)
default: `1`

`--include-metadata`

Include node metadata

`--include-leaf-value`

Include leaf node values

Examples

```
tier0 uns browse
```

```
tier0 uns browse --path /devices
```

```
tier0 uns browse --path / --max-depth 2
```

`tier0 uns search`

Search UNS topics

Usage

```
tier0 uns search
```

Flags

`-k, --keyword`
Search by name keyword

`--path-prefix`
Filter by path prefix
default: `/`

`--topic-type`
Filter by topic type

`--page`
Page number
default: `1`

`-l, --size`
Page size (max results)
default: `20`

`--include-metadata`
Include node metadata

`--include-leaf-value`
Include leaf node values

Examples

```
tier0 uns search --keyword temp
```

```
tier0 uns search --path-prefix /devices --size 50
```

```
tier0 uns search --keyword temp --include-metadata
```

`tier0 uns read`

Read current value of UNS topics

Usage

```
tier0 uns read [topic...]
```

```
tier0 uns read --topic <topic>
```

Flags

`-t, --topic`

Topic name(s) to read (repeatable; positional args are also accepted)

repeatable

`--include-metadata`

Include topic metadata (topicType, fields, description)

`--include-leaf-value`

Include leaf node values

Notes

- At least one topic is required via positional args or `--topic`.

Examples

```
tier0 uns read demo
```

```
tier0 uns read --topic demo
```

```

```

```
tier0 uns read temp humidity
```

```
tier0 uns read --topic sensor1 --include-metadata
```

`tier0 uns write`

Write value to a UNS topic

Usage

```
tier0 uns write
```

Flags

`-t, --topic`
Topic name to write to (required)
required

`-v, --value`
JSON value string (mutually exclusive with --file)

`-f, --file`
Read value from file (mutually exclusive with --value)

`--qos`
MQTT QoS level (0/1/2)
default: `0`

`--retain`
Set MQTT retain flag

`--dry-run`
Preview the request without executing it

Notes

- `--value` and `--file` are mutually exclusive.
- The value must be a valid JSON object matching the topic fields.

Examples

```
tier0 uns write --topic demo --value '{"temp":25}'
```

```
tier0 uns write --topic demo --file payload.json
```

```
tier0 uns write --topic sensor1 --value '{"on":true}' --qos 1 --retain
```

`tier0 uns history`

Query historical data for topics

Usage

```
tier0 uns history
```

Flags

`-t, --topic`

Topic name(s) (repeatable, required)

required repeatable

`--start`

Start time: relative (-1h/-30m/-7d), ISO 8601, or 'now' (required)

required

`--end`

End time: relative, ISO 8601, or 'now' (default: now)

default: `now`

`--page`

Page number

default: `1`

`-l, --size`

Page size (max data points)
default: `100`

`--interval`

Aggregation interval (e.g. 1m, 1h, 1d)

`--fn`

Aggregation function (avg/max/min/sum/count)

`--field`

Aggregation field name

Examples

```
tier0 uns history -t demo --start -1h
```

```
tier0 uns history -t demo --start -24h --end now --fn avg --interval 1h
```

```
tier0 uns history -t demo --start 2026-01-01T00:00:00Z --end 2026-01-02T00:00:00Z
```

`tier0 uns create`

Create UNS namespace nodes

Usage

```
tier0 uns create
```

Flags

`-t, --topic`

Topic path or leaf name (required if not using --file)

`--parent`

Parent path prefix (optional, combined with --topic)

`--type`

Node type: 'path' (folder) or 'topic' (data point)

`-d, --display-name`

Display name

`--description`

Description

`--alias`

Alias

`-f, --file`

Read namespace definition from JSON file ({"namespace":[]} or bare array)

`--topic-type`

Deprecated: topic type is now derived from the path (Metric/Action/State folder before leaf)
deprecated

`--fields`

Schema fields JSON array

`--fields-file`

Read schema fields JSON array from file

`--dry-run`

Preview the request without executing it

Notes

- When not using `--file`, `--topic` and `--type` are required.
- `--fields` and `--fields-file` are mutually exclusive.
- For topic nodes, the segment immediately before the leaf must be Metric, Action, or State.

Examples

```
tier0 uns create --topic Plant/Line1/Metric/Temperature --type topic
```

```
tier0 uns create --parent Factory1/Line1/Station1 --topic
Metric/ProductionCount --type topic
```

```
tier0 uns create --topic Plant/Line1 --type path --display-name 'Line 1'
```

```
tier0 uns create --file namespace.json
```

`tier0 uns update`

Update UNS topic metadata

Usage

```
tier0 uns update
```

Flags

`-p, --path`
Topic path to update (required)
required

`-n, --name`
New name

`--alias`
New alias

`--description`
New description

`--clear-description`
Clear the description without relying on an empty shell argument

`-d, --display-name`

New display name

`--extend-properties`

Extended properties JSON object

`--fields`

Schema fields JSON array

`--fields-file`

Read schema fields JSON array from file

`--update-mask`

Fields to update (repeatable, e.g. name,description,fields)
repeatable

`--dry-run`

Preview the request without executing it

Notes

- `--description` and `--clear-description` are mutually exclusive.
- `--fields` and `--fields-file` are mutually exclusive.
- At least one field to update is required.

Examples

```
tier0 uns update --path Plant/Line1/Metric/Temperature --display-name
'Line 1 Temp'
```

```
tier0 uns update --path Plant/Line1 --description 'Production line 1' --
update-mask description
```

```
tier0 uns update --path Plant/Line1/Metric/Temperature --fields
'[{"name":"temp","type":"float","unit":"C"}]' --update-mask fields
```

`tier0 uns delete`

Delete UNS node(s)

Usage

```
tier0 uns delete
```

Flags

`-p, --path`

Node path(s) to delete (repeatable, required)

required repeatable

`--hard`

Hard delete (irreversible)

`-y, --yes`

Confirm high-risk operation (required)

required

`--dry-run`

Preview the request without executing it

Notes

- Soft delete is the default and can be restored.
- `--hard` is irreversible.
- High-risk execution requires `--yes` unless using `--dry-run`.

Examples

```
tier0 uns delete --path factory/line1/sensor/temp --yes
```

```
tier0 uns delete --path factory/line1/sensor/temp --path
factory/line1/sensor/humi --yes
```

```
tier0 uns delete --path factory/line1/sensor/temp --hard --yes
```

`tier0 uns restore`

Restore topic from history

Usage

```
tier0 uns restore
```

Flags

`-p, --path`
Topic path to restore (required)
required

`-y, --yes`
Confirm high-risk operation (required)
required

`--dry-run`
Preview the request without executing it

Notes

- High-risk execution requires `--yes` unless using `--dry-run`.

Example

```
tier0 uns restore --path Plant/Line1/Metric/Temperature --yes
```

Node-RED Flow

`tier0 flow list`

List flows

alias: ls

Usage

```
tier0 flow list
```

Flags

`-k, --keyword`
Filter by name keyword

`-t, --type`
Filter by type (SourceFlow/EventFlow)

`--source`
Show SourceFlow only

`--event`
Show EventFlow only

Notes

- `--source` and `--event` are mutually exclusive.
- `--type` cannot be combined with `--source` or `--event`.

Examples

```
tier0 flow list
```

```
tier0 flow list --source --json
```

`tier0 flow get`

Get flow details

Usage

```
tier0 flow get --id <id>
```

```
tier0 flow get <id>
```

Flags

`--id`
Flow ID
default: `0`

Notes

- A positive Flow ID is required via `--id` or as a positional argument.

Examples

```
tier0 flow get --id 1 --json
```

```
tier0 flow get 1 --json
```

`tier0 flow create`

Create a new flow

Usage

```
tier0 flow create
```

Flags

`-n, --name`

Flow name (required)

required

`-t, --type`

Flow type: SourceFlow | EventFlow (required)

required

`--source`

Set type to SourceFlow

`--event`

Set type to EventFlow

`--desc`

Description

`--template`

Initial template JSON string

`--template-file`

Read initial template from file

`--dry-run`

Preview the request without executing it

Notes

- Use exactly one of `--type`, `--source`, or `--event` to select a Flow type.
- `--template` and `--template-file` are mutually exclusive.

Examples

```
tier0 flow create --name "modbus-collector" --source --desc "Modbus TCP
collector"
```

```
tier0 flow create --name "alert-handler" --event --desc "Temperature
alarm processor"
```

`tier0 flow update`

Update flow metadata

Usage

```
tier0 flow update --id <id>
```

```
tier0 flow update <id>
```

Flags

`--id`
Flow ID (required)
default: `0`

`-n, --name`
New name

`--desc`
New description

`--template`
New template JSON string

`--template-file`
Read new template from file

`--favorite`
Mark as favorite

`--unfavorite`
Remove from favorites

`--dry-run`
Preview the request without executing it

Notes

- A positive Flow ID is required via `--id` or as a positional argument.
- `--template` and `--template-file` are mutually exclusive.

- `--favorite` and `--unfavorite` are mutually exclusive.
- At least one field to update is required.

Examples

```
tier0 flow update --id 1 --name "line1-collector"
```

```
tier0 flow update --id 1 --favorite
```

`tier0 flow data`

Get Node-RED canvas JSON

Usage

```
tier0 flow data --id <id>
```

```
tier0 flow data <id>
```

Flags

`--id`
Flow ID
default: `0`

`-o, --out`
Save output to file

Notes

- A positive Flow ID is required via `--id` or as a positional argument.

Example

```
tier0 flow data --id 1 --out flows.json
```

`tier0 flow deploy`

Deploy Node-RED canvas JSON

Usage

```
tier0 flow deploy --id <id>
```

```
tier0 flow deploy <id>
```

Flags

`--id`
Flow ID (required)
default: `0`

`--flows-json`
Node-RED canvas JSON string

`-f, --flows-file`
Read Node-RED canvas JSON from file (recommended)

`-y, --yes`
Confirm high-risk operation (required)
required

`--dry-run`
Preview the request without executing it

Notes

- A positive Flow ID is required via `--id` or as a positional argument.
- `--flows-json` and `--flows-file` are mutually exclusive.
- Deployment replaces all existing Node-RED nodes in the Flow.

- High-risk execution requires `--yes` unless using `--dry-run`.

Examples

```
tier0 flow deploy --id 1 -f flows.json --dry-run --json
```

```
tier0 flow deploy --id 1 -f flows.json --yes
```

`tier0 flow delete`

Delete flow(s)

alias: del, rm

Usage

```
tier0 flow delete --id <id>
```

```
tier0 flow delete <id>[,<id>...]
```

Flags

`--id`
Flow ID(s) to delete (repeatable)
repeatable

`-y, --yes`
Confirm high-risk operation (required)
required

`--dry-run`
Preview the request without executing it

Notes

- Positional arguments may contain comma-separated Flow IDs.
- Deletion stops the Node-RED container(s) and cannot be undone.
- High-risk execution requires `--yes` unless using `--dry-run`.

Example

```
tier0 flow delete --id 1 --yes
```

`tier0 flow nodes`

List available Node-RED node types

Usage

```
tier0 flow nodes [source|event]
```

```
tier0 flow nodes --type <SourceFlow|EventFlow>
```

```
tier0 flow nodes --source
```

```
tier0 flow nodes --event
```

Flags

`-t, --type`
Flow type (SourceFlow/EventFlow)

`--source`
Show SourceFlow nodes

`--event`

Show EventFlow nodes

Notes

- Accepts at most one positional Flow type.
- `--source` and `--event` are mutually exclusive.
- Do not combine positional/type input with `--source` or `--event`.

Examples

```
tier0 flow nodes source --json
```

```
tier0 flow nodes --source --json
```

```
tier0 flow nodes --event --json
```

Assets

`tier0 assets upload`

Upload a file to Tier0 object storage

Usage

```
tier0 assets upload <local-file>
```

Flags

`--business`

Business scene

default: `attachment`

`--use-by`

Usage scope: user|workspace|platform
default: `workspace`

`--visibility`

Visibility: public|private
default: `private`

`--app-instance-id`

AI app instance ID

`--session-id`

AI session ID

`--multipart-size`

Multipart part size (e.g. 10MB, min 5MB)
default: `CLI-defined default`

`--concurrency`

Multipart upload concurrency (number of parallel parts)
default: `4`

`--resume`

Resume an interrupted multipart upload

`--abort`

Abort an interrupted multipart upload and clean up its state

Notes

- Exactly one local file argument is required.
- Files larger than 100MB automatically use multipart upload.
- The CLI does not enforce a hard maximum file size; the server applies plan/quota limits.

Examples

```
tier0 assets upload ./report.csv
```

```
tier0 assets upload ./report.csv --visibility public --business
attachment
```

```
tier0 assets upload ./report.csv --use-by workspace --visibility private
```

```
tier0 assets upload ./large-backup.tar.gz
```

```
tier0 assets upload ./large-backup.tar.gz --resume
```

```
tier0 assets upload ./large-backup.tar.gz --abort
```

```
tier0 assets upload ./large-backup.tar.gz --multipart-size 20MB --
concurrency 8
```

`tier0 assets download`

Download a file from Tier0 object storage

Usage

```
tier0 assets download
```

Flags

`--file-path`

File path returned by upload

required

`-o, --output`

Output file path (default: stdout)

`--content-disposition`

Custom Content-Disposition

Examples

```
tier0 assets download --file-path workspace/.../report.csv -o
./report.csv
```

```
tier0 assets download --file-path workspace/.../report.csv
```

`tier0 assets url`

Get file access URL from Tier0 object storage

Usage

```
tier0 assets url
```

Flags

`--file-path`

File path returned by upload

required

`--expired-sec`

Presigned URL expiration seconds

default: `3600`

`--content-disposition`

Custom Content-Disposition

Examples

```
tier0 assets url --file-path workspace/.../report.csv
```

```
tier0 assets url --file-path workspace/.../report.csv --expired-sec 300
```

`tier0 assets delete`

Delete a file from Tier0 object storage

Usage

```
tier0 assets delete
```

Flags

`--file-path`
File path returned by upload
required

`-y, --yes`
Confirm high-risk operation (required)
required

`--dry-run`
Preview the request without executing it

Notes

- Deletion cannot be undone.
- High-risk execution requires `--yes` unless using `--dry-run`.

Examples

```
tier0 assets delete --file-path workspace/.../report.csv --dry-run --
json
```

```
tier0 assets delete --file-path workspace/.../report.csv --yes
```

REFERENCE

Embedded Builder Skills

Reference for the AI skills embedded in Tier0 Builder.

Tier0 App Builder currently includes 7 active embedded AI skills. These skills help the agent turn requirements into industrial applications, align applications with the Tier0 platform scaffold, repair UI issues, and keep preview environments stable.

`app-srs`

Meaning: The requirements-analysis entry point for new industrial-enterprise apps. It is used for WMS, MES, ERP, CRM, QMS, EAM, approval, inventory, supplier, customer, scheduling, and similar enterprise workflows.

Example:

```
Use $app-srs to design a quality exception management system for an automotive parts factory.
```

Natural language can also trigger it:

```
Generate a warehouse management app with receiving, putaway, picking, shipping, and inventory counting.
```

What it does:

1. Analyzes business roles, workflows, states, and exceptions.
2. Checks whether the requirement is clear enough.
3. Generates a complete SRS.
4. Saves it to `specs/spec.md` by default.
5. Hands the result to `app-builder` by default unless the user asks for requirements only.

`app-builder`

Meaning: The core skill for actually building the app.

Example:

```
Use $app-builder to build the first version from specs/spec.md.
```

Internal work types: The agent decides which work type to use based on the prompt and current project state.

- **First Release**: Builds the first version from an SRS.
- **Delta Feature**: Adds a substantial module or workflow to an existing app.
- **Light Edit**: Handles small field, copy, style, or validation changes without running the full build flow.

What it checks:

- Visible entry points.
- Executable user paths.
- Data or state changes.
- Role permissions.
- Build and preview verification.

> **Tip**
> If a first-time build has no `specs/spec.md`, the agent will create one with `app-srs` first.

`align-platform-app`

Meaning: A migration skill for aligning an existing app with the Tier0 platform scaffold.

```
Use $align-platform-app to migrate this old Vite React project into the Tier0
platform scaffold.
```

Suitable sources:

- Vite, CRA, Next.js, React Router, static HTML, and full-stack Node/React apps.
- Older Tier0 apps.
- Apps that rely on LocalStorage, IndexedDB, Zustand persist, in-memory mocks, or simulated data.

What it does:

- Aligns the app to the Tier0 TanStack Start scaffold.
- Migrates persistent business data to Drizzle/PostgreSQL.
- Aligns routes, APIs, services, and database setup.
- Removes app-owned login, logout, and role-switching flows.
- Connects to Gateway-owned identity and roles.
- Preserves platform contracts such as `/api/health`, `/api/manifest`, and `artifact.toml`.
- Verifies preview and production startup.

`migrate-node-red-app`

Meaning: A Node-RED migration skill for aligning a Node-RED application with the Tier0 platform scaffold. It is usually used together with `align-platform-app`.

Example:

```
Use $align-platform-app and $migrate-node-red-app to migrate this Node-RED
backend into a Tier0 TanStack Start app.
```

What it does:

- Inventories flows, subflows, function nodes, change nodes, and switch nodes.
- Analyzes HTTP, MQTT, database, scheduled jobs, and external API calls.
- Builds a migration mapping table.
- Separates CRUD, workflow, scheduling, integration, and temporary simulation behavior.
- Moves business logic into services.

- Moves interfaces into API routes.
- Moves state into PostgreSQL.
- Moves long-running work and integrations into workers, queues, or external services.
- Verifies browser, CRUD, workflow, and integration cutover separately.

> ⚠ **Caution**
> It does not handle ordinary preview or packaging issues by itself.

`ui-repair`

Meaning: A repair skill for existing apps. It is used for UI quality issues, not for first-time app generation.

Example:

```
Use $ui-repair to fix overflow and form misalignment in the order edit dialog
on small screens.
```

Suitable issues:

- Misaligned forms or required markers.
- Dialog or drawer overflow.
- Accidental horizontal scrolling.
- Clipped select menus or dropdowns.
- Broken table or chart layout.
- Flat pages with weak visual hierarchy.
- Mixed UI languages.
- PDA, kiosk, dashboard, and industrial interface layout problems.

> ⚠ **Caution**
> The skill should locate the smallest owning component and fix the affected surface without rebuilding the whole app. For complex responsive work, use `responsive-craft`.

`responsive-craft`

Meaning: A responsive-layout skill for audits, new responsive builds, and multi-breakpoint previews.

Invocation modes:

- **audit**: Audits and repairs an existing responsive layout.
- **build**: Builds a responsive page from scratch.
- **preview**: Starts a multi-breakpoint live preview.

Examples:

```
Use $responsive-craft audit to fix the mobile layout of this dashboard.
```

```
Use $responsive-craft build to create an inventory workspace for phone,
tablet, and desktop.
```

```
Use $responsive-craft preview to show 375, 768, 1024, and 1440 px breakpoints.
```

Execution styles:

- **Adaptive**: Moves quickly and confirms design forks as they appear.
- **Guided**: Writes responsive behavior specs for each component before implementation.

`preview-runtime-stability`

Meaning: A runtime troubleshooting skill for preview failures during app development. It does not build business features.

Example:

```

```

```
Use $preview-runtime-stability to diagnose "Preview loading timed out".
```

Suitable issues:

- Missing or partial `node_modules`.
- npm `ERESOLVE`.
- Vite or TanStack Start startup failures.
- Port `5173` conflicts.
- `/api/health` failures.
- SDK SSR module errors.
- `artifact.toml` preview port mismatch.
- Frequent full-page reloads during HMR.
- Stale Node or Vite processes that did not exit.

Standard flow:

1. Check dependency installation.
2. Check port `5173`.
3. Probe the health endpoint.
4. Reuse the preview when it is healthy.
5. Clean up the old preview when it is unhealthy.
6. Restart and verify the port, health endpoint, and HMR behavior.

Skill Selection Guide

- Use `app-srs` first when the requirement is unclear or when you need a durable specification.
- Use `app-builder` when the implementation scope is already clear.
- Use `align-platform-app` and `migrate-node-red-app` for migration work.
- Use `ui-repair`, `responsive-craft`, or `preview-runtime-stability` when the app already exists and the problem is visual, responsive, or runtime-specific.

REFERENCE

User Management

Tier0 uses membership, licenses, and custom user accounts to let customers manage platform access and user permissions.

Tier0 manages user permissions and access differently across editions.

- Cloud uses Workspace member invitations to manage who can access a Workspace.
- Enterprise admin assigns user accounts and roles to determine correspondingly accessible functions.

Access Control

TAB: CLOUD

- Workspace owner or existing members can invite new members by phone or email.
- After receiving an invitation code or link, new members register an account and join the Workspace.
- Set each new member's seat type (Operator or Builder) when sending the invitation to determine which features they can access.
    - **Builder**: Can access all functional modules.
    - **Operator**: Can access only the application center.
- Account registration requires two-step verification.

> ⚠ **Caution**
> Workspace membership does not automatically grant access to the platform; users must register the invited phone or email to use the platform.

> ⓘ **Note**
> After registering an account, you can also apply for a new Workspace.

TAB: ENTERPRISE

- After signing in for the first time with the default account, the admin changes the password, then adds user accounts and assigns roles in **Settings**.
- Admin determines who can use the platform by assigning accounts, and which functions the account can access by assigning roles:
    - **Admin**: Can access all functional modules.
    - **Operator**: Can only access the **Launchpad** where all accessible applications are listed.

> ⓘ **Note**
> Sign in with the account created by the admin and use functions within the permissions of the assigned role.

Account Management

TAB: CLOUD

Action	Who	Description
Change seat type	Owner / Builder	• The owner can change any member's seat type. • A Builder can change only an Operator's seat type.
Delete Workspace	Owner	Deleting the Workspace immediately erases all data. This cannot be undone.
Transfer Workspace	Owner	—
Remove member	Owner / Builder	• The owner can remove any member.

Action	Who	Description
		• A Builder can remove only an Operator.
Delete account	All members	• The owner must delete or transfer their Workspace first. • The member's account immediately leaves all Workspaces. • Account data is erased immediately and cannot be recovered.

TAB: ENTERPRISE

- An Admin can reset passwords, edit user roles, disable accounts, and delete accounts.
- The role of the initial default account cannot be edited, and the account cannot be disabled or deleted.
- Deleting an account immediately erases all its data. This cannot be undone.

REFERENCE

Tier0 Cloud Network and Data Security Overview

Network boundaries, tenant isolation, data protection, and security operations for the Tier0 Cloud production SaaS service.

Scope: The Tier0 Cloud production SaaS service, deployed in the AWS Singapore Region.
Document date: October 22, 2026

Tier0 Cloud uses network access controls, tenant isolation, identity and access management, data encryption, and backup and recovery measures to reduce the risk of network attacks and unauthorized access. This overview describes the platform's security measures and the controls that limit the spread of security incidents to other tenants, internal platform services, and customer networks.

> ⓘ **Edge edition**
> This overview applies only to Tier0 Cloud. For Edge setup, see **Installation**.

1 Network Boundaries and Access Control

Public Web and API access to Tier0 Cloud uses TLS encryption and passes through a unified edge entry point and application gateway. Core service endpoints validate access to the origin and reject direct requests that do not meet the validation requirements, reducing the risk of bypassing the edge entry point to access the origin.

Production environments are isolated from development and staging environments at the cloud account, network, cluster, and database levels, with access credentials managed separately. Administrative access to production uses an authenticated, dedicated network channel.

2 Tenant Isolation and Incident Containment

2.1 Workload Isolation

Tenant workloads run in separate Kubernetes namespaces and are separated from platform services and management components through dedicated node pools and scheduling constraints. These controls limit direct interaction between tenant applications and platform

components, reducing the impact on other services if an application experiences a security incident.

2.2 Network Isolation

Inbound and outbound traffic for tenant namespaces follows network policies that deny traffic by default and allow it only through explicit rules. The platform automatically configures and synchronizes these policies, which are enforced by the cluster networking components.

Inbound traffic is allowed only from workloads belonging to the same tenant and platform services that perform necessary business functions. Outbound rules allow cluster DNS resolution, services belonging to the same tenant, designated platform gateways, messaging services, and assigned database endpoints. Public internet access is subject to network policies. Except for explicitly allowed business dependencies, these policies restrict access to private network addresses and block direct access to cloud instance metadata addresses.

These measures limit unauthorized connections between tenants and lateral movement from tenant workloads into the platform's internal network. A security incident affecting one tenant application does not automatically grant access to other tenants or platform management components. Authorized business connections remain subject to the applicable network rules and identity and access controls.

2.3 Data and Identity Isolation

Tenant application data is logically isolated at the database level. Where a separate database unit is allocated, network access rules also restrict connection sources.

The platform assigns access permissions according to service responsibilities, limiting each workload's access to the resources needed for its functions. Network isolation and identity-based authorization work together to limit the resources an affected workload can access.

2.4 Customer Network Access Boundaries

The tenant isolation measures described in this section apply within the Tier0 Cloud platform. When customers connect internal systems through connectors, gateways, or other integrations, the access boundaries of their networks also depend on the connection method, routing, firewall rules, and permissions granted to the credentials used.

Customers should allow only the destination addresses, ports, and account permissions required for their business needs, and segment integration components from other internal systems. Platform isolation cannot replace network access controls on the customer side. Both parties should determine and verify the access scope for the specific integration to limit the risk of an incident spreading to other customer systems through authorized connections.

3 Data Protection

The platform protects data in transit, at rest, and during access through the following measures:

- Public Web and API communication, as well as access to managed databases and caches, uses TLS encryption.
- Managed databases, caches, and object storage have encryption at rest enabled.
- Object storage blocks public access and restricts data access through encryption requirements, HTTPS requirements, and resource permissions.
- Access to tenant data is subject to both database isolation and identity and access controls.

4 Backup and Recovery

Tier0 Cloud backs up production databases and service-level databases included in the backup scope. Backup data is stored in dedicated object storage with restricted access and managed according to the applicable retention policies.

Backup jobs use a separate identity with permissions limited to the required read and write operations at designated storage locations. This identity is not granted permission to delete backup objects. Storage lifecycle rules remove expired backups, reducing the risk that compromised backup credentials could be used to delete recovery data.

Integrity checks are performed after backups complete, and restore validation in an isolated environment checks that backups are usable. Application configurations are version-controlled, and build artifacts are stored in a private container image registry to support service recovery and version rollback.

5 Administrative Access and Production Changes

Administrative access to production uses individually assigned identities and permissions based on the principle of least privilege. Console identities used for sensitive operations require multi-

factor authentication.

Routine inspections use a restricted, read-only identity that can view workload status, logs, and events, but cannot read sensitive credentials, modify resources, or execute remote commands. Production changes use separately authorized identities and processes.

Infrastructure configurations, deployment manifests, and operations scripts are version-controlled. Production changes must define their objectives, scope of impact, verification steps, and rollback plans. Records include the executing identity, configuration differences, and verification results to support traceability and recovery.

6 Monitoring and Security Incident Response

The platform centrally collects production metrics, logs, and health information and sends anomaly alerts to operational response channels. Monitoring covers service availability, error rates, database health, certificate validity, backup job status, and cluster status.

When a security incident occurs, response efforts prioritize containing its impact and preserving relevant logs, configurations, and the incident timeline. Service is restored after the necessary isolation and investigation, followed by an incident review and tracked corrective actions. Production changes during incident response follow established authorization and recordkeeping processes.

7 Security Contact

For security assessments, customer due diligence, or suspected vulnerabilities, contact your Tier0 account manager or the security team. Non-public architecture and control details may be provided under appropriate confidentiality arrangements. Do not include access credentials, private keys, or sensitive customer data in your initial contact.

REFERENCE

ISA-95 Equipment Hierarchy

Reference levels for modeling physical assets and control systems with ISA-95.

The ISA-95 Equipment Hierarchy outlines how physical assets and control systems are structured within an organization.

It is broken into the following levels:

- **Enterprise**: The top level, representing the entire organization. This includes overall business processes and decision-making entities, typically supported by ERP systems.
- **Site**: Represents a specific geographic location or facility. Each site can have its own management systems and local production control.
- **Area**: An operational division within a site, often a functional area such as packaging, assembly, or warehousing.
- **Process Cell**: The basic unit of manufacturing within an area. It encompasses all the equipment required to produce a specific product or perform a specific function.
- **Unit**: A distinct part of the process cell, responsible for a specific function such as mixing, heating, or assembling.
- **Equipment Module**: A modular piece of equipment that performs a specific task within a unit. For example, in a mixing unit, the module could be a heating element.
- **Control Module**: The lowest level, representing individual devices or controllers such as sensors, pumps, actuators, or PID control loops that handle direct control of physical processes.

REFERENCE

Standard Deployment Plan

Reference four-VM standard deployment plan for Tier0 Enterprise.

Objective

The simplified edition does not use Docker Swarm. It deploys one `enterprise` application node with a total of `4` virtual machines:

- `1` enterprise node hosts `tier0-enterprise + EMQX`.
- `2` database nodes form a PostgreSQL + TimescaleDB primary/standby cluster.
- `1` Witness node participates in database supervision, quorum, and failover.

The simplified edition reduces deployment complexity. The tradeoff is that the application layer and EMQX are single points of failure. Only the database layer keeps `DB VIP + Witness` high availability.

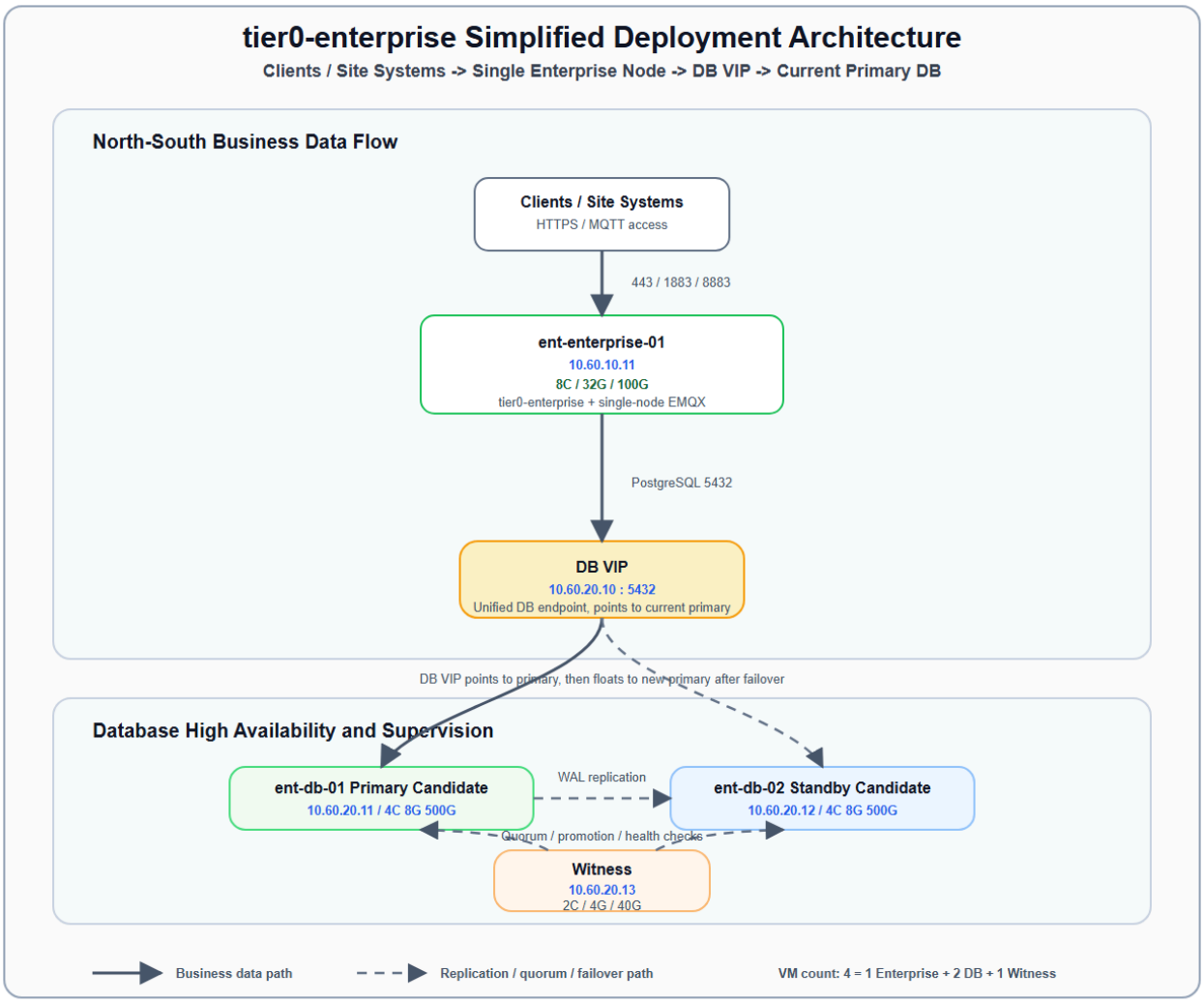
Planned IP Addresses

The following IP addresses are placeholders. Replace them with the real site network before implementation.

Type	Name	Planned IP	Description
Application node	`ent-enterprise-01`	`10.60.10.11`	Single enterprise node hosting `tier0-enterprise + EMQX`
Logical entry	`DB VIP`	`10.60.20.10`	Unified database endpoint, points only to the current primary
Database node	`ent-db-01`	`10.60.20.11`	PostgreSQL + TimescaleDB primary candidate

Type	Name	Planned IP	Description
Database node	`ent-db-02`	`10.60.20.12`	PostgreSQL + TimescaleDB standby candidate
Quorum node	`ent-db-witness-01`	`10.60.20.13`	`repmgr witness` / database quorum supervisor

Overall Topology



Core paths:

- Business path: clients / site systems -> `ent-enterprise-01` -> `DB VIP` -> current primary DB.
- `tier0-enterprise` and `EMQX` both run on `ent-enterprise-01`.

- Database access still goes through `DB VIP`; direct application access to `ent-db-01/02` is prohibited.
- `Witness` participates only in database quorum, promotion, and health checks. It is not on the business path.

Database Supervision and Failover

Phase	Path	Description
Normal access	`ent-enterprise-01 -> DB VIP -> current primary`	Application always connects to `10.60.20.10:5432`
Normal replication	`current primary -> current standby`	PostgreSQL WAL streaming replication
Normal supervision	`DB-01/DB-02 -> Witness`	`repmgrd` reports status; Witness participates in quorum
Primary failure	`standby -> Witness`	Standby requests quorum confirmation to avoid split-brain
Standby promotion	`standby promote`	Standby is promoted after Witness confirmation
VIP floating	`DB VIP -> new primary`	`10.60.20.10` binds to the new primary
Application recovery	`ent-enterprise-01 -> DB VIP -> new primary`	Application recovers after connection-pool reconnect

Node Configuration

Node	Role	vCPU	Memory	Storage	System disk recommendation	Data disk recommendation	Main components
`ent-enterprise-01`	Single enterprise node	`8`	`32G`	`100G`	Single `/` system disk, 100G	No separate data disk	tier0-enterprise, EMQX, optional Nginx/HAProxy
`ent-db-01`	PostgreSQL/TSDB primary candidate	`4`	`8G`	`500G`	`/` 80G	`/pgdata` 360G, `/pgwal` 40G, `/backup-staging` 20G	PostgreSQL, TimescaleDB, repmgr, keepalived
`ent-db-02`	PostgreSQL/TSDB standby candidate	`4`	`8G`	`500G`	`/` 80G	`/pgdata` 360G, `/pgwal` 40G, `/backup-staging` 20G	PostgreSQL, TimescaleDB, repmgr, keepalived
`ent-db-`	Witness / quoru	`2`	`4G`	`40G`	`/` 40G	Not planned	repmgr witness,

Node	Role	vCPU	Memory	Storage	System disk recommendation	Data disk recommendation	Main components
`witness-01`	m supervisor					separately	monitoring agent, audit logs

Configuration Summary

Category	Count	Per-node spec	vCPU subtotal	Memory subtotal	Storage subtotal
enterprise application node	`1`	`8C / 32G / 100G`	`8C`	`32G`	`100G`
PostgreSQL + TimescaleDB nodes	`2`	`4C / 8G / 500G`	`8C`	`16G`	`1000G`
Witness node	`1`	`2C / 4G / 40G`	`2C`	`4G`	`40G`
Total	`4`	-	`18C`	`52G`	`1140G`

Converted total:

- Total VMs: `4`
- Total vCPU: `18`
- Total memory: `52G`
- Total storage: `1.14T`

Connectivity Matrix

Operations Access

Target	Method	Port	Description
All VMs	`ssh -i ~/.ssh/chen-rsa ops@<IP>`	`22/TCP`	Use after unified key-based SSH is implemented; use site account before that
`ent-enterprise-01`	`https://10.60.10.11`	`443/TCP`	Business entry
`DB VIP`	`psql -h 10.60.20.10 -p 5432`	`5432/TCP`	Allow only enterprise node and operations whitelist

Application and EMQX Ports

Source	Target	Port	Purpose
Users / site systems	`ent-enterprise-01`	`80/TCP`	HTTP entry or HTTPS redirect, optional
Users / site systems	`ent-enterprise-01`	`443/TCP`	HTTPS business entry
Devices / site systems	`ent-enterprise-01`	`1883/TCP`	EMQX MQTT plaintext, optional
Devices / site systems	`ent-enterprise-01`	`8883/TCP`	EMQX MQTT TLS, recommended
Devices / site systems	`ent-enterprise-01`	`8083/TCP`	MQTT WebSocket, optional
Devices / site systems	`ent-enterprise-01`	`8084/TCP`	MQTT WebSocket TLS, optional

Source	Target	Port	Purpose
Operations network	`ent-enterprise-01`	`18083/TCP`	EMQX Dashboard, operations whitelist only
`ent-enterprise-01`	`DB VIP`	`5432/TCP`	PostgreSQL business access

Database and Supervision Communication

Source	Target	Port	Purpose
`ent-db-01`	`ent-db-02`	`5432/TCP`	PostgreSQL WAL streaming replication
`ent-db-02`	`ent-db-01`	`5432/TCP`	Reverse replication after recovery
`ent-db-01/02`	`ent-db-witness-01`	`5432/TCP`	repmgr metadata / witness communication
`ent-db-01`	`ent-db-02`	`VRRP`	DB VIP floating; allow protocol `112`

Deployment Recommendations

enterprise Application Node

- Do not deploy Docker Swarm.
- Deploy `tier0-enterprise` and `EMQX` on the same `ent-enterprise-01` node.
- `ent-enterprise-01` is an application-layer single point of failure. If it fails, recover manually or rebuild from backup.
- The node has only one `100G` system disk and is not suitable for long-term business data.
- `tier0-enterprise` business data must be stored in PostgreSQL/TimescaleDB.

- Uploads, imports/exports, and reports may be stored locally and backed up. If later upgraded to the standard edition, migrate these files to NFS/NAS or object storage.

Single-node EMQX

- EMQX is a single-node stateful service in the simplified edition.
- Pin its data directory on `ent-enterprise-01`, for example `/data/emqx`.
- Back up configuration, certificates, plugins, and rule-engine configuration.
- No message-layer HA is provided. Use the standard edition with a three-node EMQX cluster if this is unacceptable.

PostgreSQL + TimescaleDB

- PostgreSQL: `16` or the site-certified version.
- TimescaleDB: stable version compatible with the selected PostgreSQL major version.
- Replication: physical streaming replication.
- Failover: `repmgrd + witness`.
- Database entry: `DB VIP`.
- Connection pool: PgBouncer can run on `ent-enterprise-01` to reduce database connection pressure.

Witness

- Witness stores only `repmgr witness` metadata and supervision logic.
- Witness does not host business SQL.
- Witness failure does not stop the current primary, but reduces trusted automatic failover capability.

Port and Firewall Baseline

Port / protocol	Allowed scope	Purpose
`22/TCP`	Operations network to 4 VMs	SSH

Port / protocol	Allowed scope	Purpose
`80/TCP`	User network to `ent-enterprise-01`	HTTP redirect or internal plaintext entry
`443/TCP`	User network to `ent-enterprise-01`	HTTPS business entry
`1883/TCP`	Devices / site systems to `ent-enterprise-01`	EMQX MQTT plaintext, optional
`8883/TCP`	Devices / site systems to `ent-enterprise-01`	EMQX MQTT TLS, recommended
`8083/TCP`	Devices / site systems to `ent-enterprise-01`	EMQX WebSocket, optional
`8084/TCP`	Devices / site systems to `ent-enterprise-01`	EMQX WebSocket TLS, optional
`18083/TCP`	Operations network to `ent-enterprise-01`	EMQX Dashboard, whitelist only
`5432/TCP`	enterprise to `DB VIP`; DB/Witness internal	PostgreSQL / repmgr
`112/VRRP`	Between DB nodes	DB VIP floating
`9100/TCP`	Monitoring system to 4 VMs	node exporter, optional

HA Capabilities and Failure Boundaries

Failure scenario	Expected behavior	Business impact
`ent-enterprise-01` fails	Application and EMQX both stop; manual recovery or rebuild from backup is required	Full application-layer outage

Failure scenario	Expected behavior	Business impact
Current DB primary fails	Standby is promoted after Witness quorum; DB VIP floats	Write interruption for seconds to minutes; application must reconnect
DB standby fails	Primary continues serving, replication alert is raised	No direct business outage, but DB hot standby is lost
Witness fails	Current primary continues serving, but automatic failover confidence is reduced	No direct business outage, restore Witness first
DB VIP floating fails	Database may have a new primary, but unified entry is unreachable	Business outage; manual VIP binding required

Pre-Go-Live Checklist

Check item	Pass criteria
4 VM hostnames are fixed	`hostnamectl` matches the plan
Unified SSH works	Operations terminal can access all 4 VMs with key-based SSH
Time synchronization	Time drift across 4 VMs is below `1s`
Firewall is open	Ports from section 9 are allowed by scope
enterprise service is healthy	`tier0-enterprise` health check passes
EMQX state is verified	EMQX node, listening ports, and persistent directory are healthy
enterprise disk usage is safe	`/` usage is below `70%`; log and cache cleanup is configured

Check item	Pass criteria
DB primary/standby is healthy	`repmgr cluster show` shows primary, standby, and witness healthy
DB VIP works	enterprise node can connect to `10.60.20.10:5432`
Primary DB failover drill	Standby can be promoted and DB VIP can float
Backup restore drill	Backup can be restored to a test database

Implementation Order

1. Create 4 VMs and fix hostnames, IPs, time synchronization, and SSH.
2. Initialize OS baseline, firewall, mounts, and kernel parameters.
3. Deploy `tier0-enterprise` on `ent-enterprise-01`.
4. Deploy single-node EMQX on `ent-enterprise-01`.
5. Deploy PostgreSQL + TimescaleDB primary/standby.
6. Deploy repmgr witness, repmgrd, and DB VIP floating scripts.
7. Verify connectivity, EMQX recovery, database failover, backup restore, and routine checks.
8. Finalize runbooks, credential archives, and routine check tasks.

Risks and Open Items

- The simplified edition has an application-layer single point. If `ent-enterprise-01` fails, the whole business service is unavailable.
- EMQX is single-node and does not provide message-layer HA.
- `ent-enterprise-01` has only a single `100G` system disk. Logs, cache, attachments, and temporary files must be controlled.
- If application-layer or EMQX HA is required later, switch to the standard edition.
- Database RPO/RTO is not confirmed. This primary/standby design cannot guarantee zero data loss.

- DB VIP floating scripts must be verified on site, especially whether the virtualization/network platform allows VRRP.

REFERENCE

High Availability Deployment Plan

Reference deployment plan for a highly available Tier0 Enterprise environment.

Plan Objective

This V2 plan deploys a logical Tier0 Enterprise Fleet Center in an offline or private environment. Six machines share the business, middleware, database, storage, and agent roles. Users and internal Enterprise services always connect through fixed VIP addresses. After one machine or one proxy fails, Keepalived, HAProxy, repmgr, RustFS, and Fleet Agent take over at their own layers.

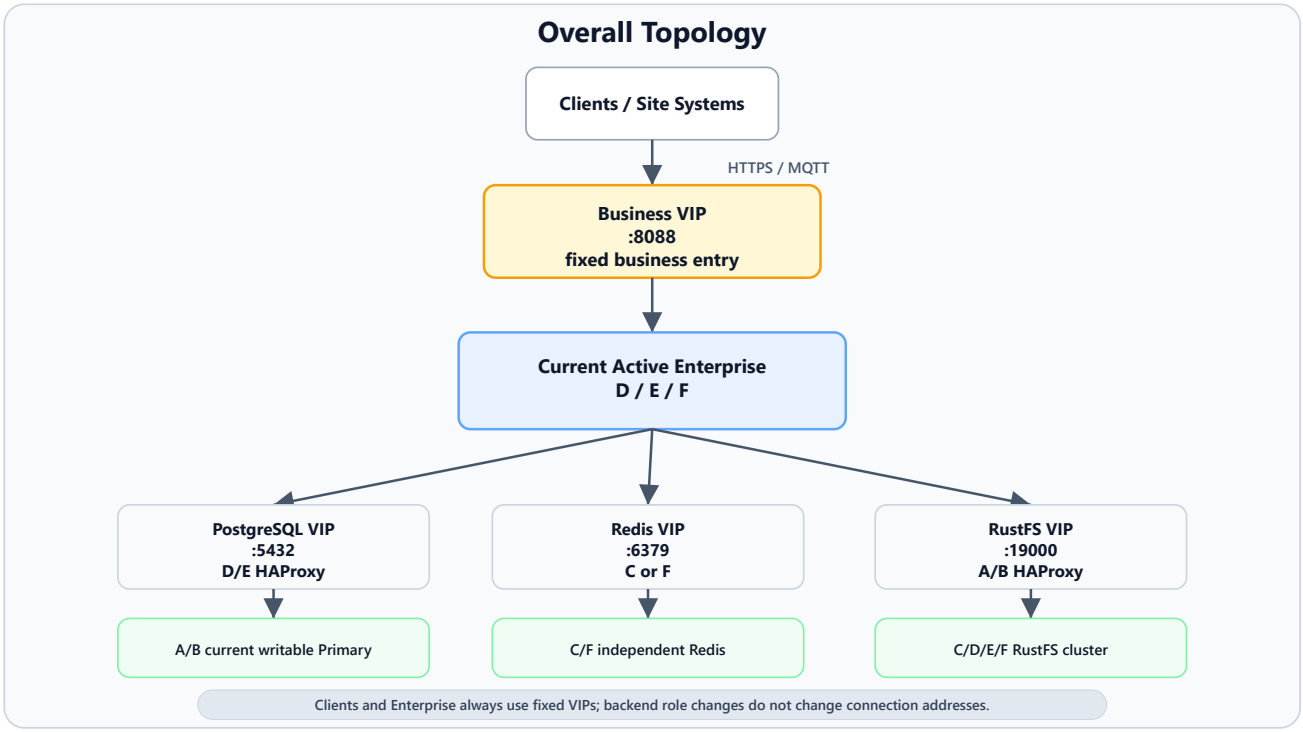
The plan targets single-machine failure recovery in one network segment. It is an active-standby disaster recovery design, not a multi-data-center, consensus-election, or zero-data-loss high availability cluster.

The design goals are:

- The Fleet Center entry can move to a healthy Standby node after the current business node fails.
- PostgreSQL, Redis, and RustFS use fixed VIP addresses, so Enterprise does not need to change `.env` when backend roles change.
- Local business files for App, Flow, Marimo, License, and similar runtime assets can be synchronized from Active to Standby.
- One Cluster License maps to one logical installation and is not activated again after Enterprise switches nodes.
- A separate deployer initializes the six machines, assigns roles, deploys services, monitors status, and validates failover.
- VIP failover, file synchronization, independent Redis instances, database replication, and object-storage redundancy are clearly separated.

This plan applies only to Fleet Center. Branch Enterprise nodes are still installed as normal single-machine deployments and do not join this six-machine active-standby design.

Overall Topology



The core request path is:

```
Clients / site systems
    -> Business VIP :8088
    -> Current Active Enterprise (D / E / F)
    |-- PostgreSQL VIP :5432 -> D/E HAProxy -> A/B current writable Primary
    |-- Redis VIP      :6379 -> C or F independent Redis
    `-- RustFS VIP     :19000 -> A/B HAProxy -> C/D/E/F RustFS cluster
```

The four VIPs are fixed entries in the business configuration. When Enterprise switches nodes or middleware backend roles change, clients and Enterprise do not change connection addresses.

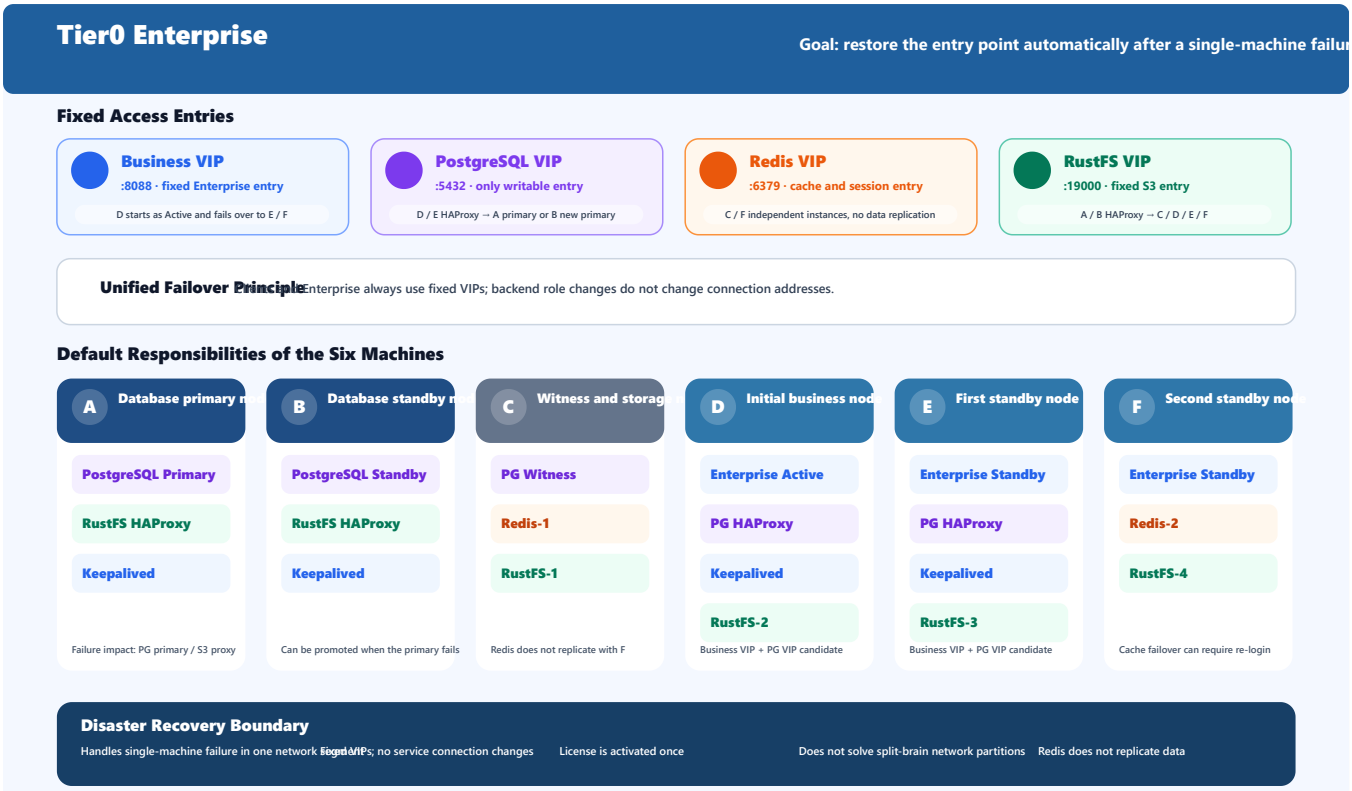
Planned IP Addresses

The following addresses are examples only. Replace them with real addresses from the same on-site IPv4 subnet before implementation. The four VIPs must not be occupied by DHCP, static hosts, or other VRRP instances.

Name	Example IP	Description
deployer-controller	`10.60.10.20`	Independent deployer; not counted as one of the six business machines
ent-ha-01 / A	`10.60.10.11`	PostgreSQL Primary and related roles
ent-ha-02 / B	`10.60.10.12`	PostgreSQL Standby and related roles
ent-ha-03 / C	`10.60.10.13`	Witness, Redis-1, RustFS-1
ent-ha-04 / D	`10.60.10.14`	Initial Active Enterprise and related roles
ent-ha-05 / E	`10.60.10.15`	Enterprise Standby and related roles
ent-ha-06 / F	`10.60.10.16`	Enterprise Standby and related roles
Business VIP	`10.60.10.101`	Unified Enterprise entry, default `8088`
PostgreSQL VIP	`10.60.10.102`	Unified database entry, default `5432`
Redis VIP	`10.60.10.103`	Unified Redis entry, default `6379`
RustFS VIP	`10.60.10.104`	Unified S3 entry, default `19000`

Before implementation, confirm the business NIC name, IPv4 prefix, gateway, DNS, NTP, hostnames, SSH user, disk mount points, and firewall allowlist. VIPs, node IPs, and NICs must not be guessed by scripts and used directly in production.

Six-Machine Responsibilities



Machine	Default deployed components	Key responsibility
A	PostgreSQL Primary, RustFS HAProxy, Keepalived	Initial database primary; first RustFS proxy candidate
B	PostgreSQL Standby, RustFS HAProxy, Keepalived	Streaming database standby; second RustFS proxy candidate
C	PostgreSQL Witness, Redis-1, RustFS-1	Database witness/status record; Redis and RustFS data node
D	PG HAProxy, Keepalived, RustFS-2, Enterprise, Fleet Agent, rsync	Initial business Active; first PostgreSQL proxy candidate
E	PG HAProxy, Keepalived, RustFS-3, Enterprise, Fleet Agent, rsync	Business Standby; second PostgreSQL proxy candidate

Machine	Default deployed components	Key responsibility
F	Redis-2, RustFS-4, Enterprise, Fleet Agent, rsync	Business Standby; second Redis instance

This allocation is for single-machine failure tolerance and balanced resource usage across the six machines. The deployer maps machines to A-F by the natural order of hostnames/IP addresses and selects default roles, but the final assignment must still be reviewed against the on-site disk layout and failure domains.

Machine Configuration Checklist

The Enterprise business layer is single-active/standby. At any time, only the node holding the Business VIP serves business requests. Scale the business layer vertically by increasing CPU, memory, and disk on one business node. Do not estimate concurrency by multiplying the number of Enterprise nodes.

Machine	Node type	vCPU	Memory	Base storage	Disk recommendation
A	PostgreSQL Primary node	8C	16G	1T	`/` about 80G; remaining space for PostgreSQL data, WAL, and backup staging
B	PostgreSQL Standby node	8C	16G	1T	`/` about 80G; remaining space for PostgreSQL data,

Machine	Node type	vCPU	Memory	Base storage	Disk recommendation
					WAL, and backup staging
C	Witness / middleware node	4C	8G	500G	Allocate fixed mount points for system, Redis, and RustFS data
D	Initial Active Enterprise node	8C	32G	500G	Allocate fixed mount points for system, containers, Enterprise synced files, and RustFS data
E	Standby Enterprise node	8C	32G	500G	Same as D
F	Standby Enterprise node	8C	32G	500G	Same as D

Category	Count	Per-node spec	vCPU subtotal	Memory subtotal	Storage subtotal
PostgreSQL database nodes	2	`8C / 16G / 1T`	16C	32G	2T
Witness / middleware node	1	`4C / 8G / 500G`	4C	8G	500G
Enterprise business nodes	3	`8C / 32G / 500G`	24C	96G	1.5T
Total	**6**	-	**44C**	**136G**	**4T**

Configuration Summary

Unified Enterprise Connection Configuration

D, E, and F keep the same Enterprise configuration.

Dependency	Configuration principle
PostgreSQL	Connect to PostgreSQL VIP, not A/B node IPs
Redis	Connect to Redis VIP, not C/F node IPs
File storage	Use `FILESTORE_DRIVER=s3`; point the S3 endpoint to RustFS VIP
Fleet identity	Three nodes share one logical `installationId`; each node has its own `memberId`
Fleet Agent	One process-level Fleet Agent runs on each Enterprise node

Dependency	Configuration principle
License	Activate once through the Business VIP; include the Bundle in the HA fileset

`installationId` and `memberId` are generated and persisted by the installation process. Operators should not invent them manually or copy `memberId` between machines.

VIP and Candidate Nodes

VIP	Candidate nodes	Health basis	Takeover result
Business VIP	D, E, F	Fleet Agent readiness, business container state, sync generation	Only the Owner starts the Enterprise HA business group
PostgreSQL VIP	D, E	HAProxy/Keepalived state; HAProxy routes only to the writable Primary	Enterprise keeps the same database address
Redis VIP	C, F	Redis `PING` and local service state	Switches to the other independent instance; cache/session may be lost
RustFS VIP	A, B	HAProxy and RustFS backend health	Proxy entry moves while the S3 address stays unchanged

Component Deployment Design

Enterprise, Fleet Agent, and Business VIP

- D, E, and F install the same full Enterprise offline ZIP.
- Fleet Agent is a host-level process service, not a business container.

- Keepalived manages the Business VIP and calls the Fleet Agent readiness endpoint to decide whether the local node can hold the VIP.
- Only the Business VIP Owner runs the business group; the other two nodes stay Standby.
- A node that loses the VIP must stop the business group. A recovered node must rejoin as Standby first.
- EMQX runs as part of the single-active business group. Do not restart it outside Agent control or build a separate election mechanism across the three machines.

PostgreSQL Primary/Standby, Witness, and Unified Entry

- A starts as Primary, B starts as Streaming Standby, and C runs repmgr Witness.
- Witness records cluster state and participates in failure decisions. It does not store full business data and does not serve reads or writes.
- When the primary fails, repmgr promotes a healthy Standby to the new Primary.
- D/E HAProxy routes `5432` traffic only to the current writable Primary.
- PostgreSQL VIP floats between the D/E proxy nodes. Proxy failover and database primary/standby failover are decoupled.
- Acceptance must prove both “only one writable Primary” and “Standby resumes streaming”.

Redis Independent Instances

- C and F each run one standalone Redis instance.
- The two Redis instances do not form Sentinel, Cluster, or primary/replica replication.
- Redis VIP only switches the connection entry. When the current instance fails, the VIP moves to the other healthy instance.
- Redis may only store data that can be lost or rebuilt. Empty cache and user re-login after switching are within the design boundary.
- If Redis later stores non-discardable state, upgrade to a replication/arbitration design instead of keeping the independent-instance assumption.

RustFS Four-Node Cluster

- C, D, E, and F form a four-node distributed RustFS object storage cluster.

- HAProxy on A and B proxies all healthy RustFS backends.
- RustFS VIP floats between A and B. Enterprise always accesses S3 through this VIP.
- A single RustFS node failure is handled by the cluster mechanism; a single proxy failure is handled by VIP failover.
- Do not synchronize object data with rsync, hot copy, or direct edits to RustFS mount directories.
- RustFS node count, disk count, and erasure-coding availability boundaries must follow the cluster check results of the deployed version.

App, Flow, Marimo, and License File Synchronization

Fleet Agent synchronizes only local business files that cannot be placed directly in PostgreSQL/RustFS but must exist on the new Active node:

- App runtime directories and `node_modules`
- Flow, Marimo, Notebook, and related local runtime files
- License Bundle
- Other HA filesets explicitly registered by the current implementation

Synchronization uses system `rsync` over deployer-managed SSH trust between business nodes. Each release creates a complete generation/manifest. A Standby node can take over only after confirming that the generation is complete. PostgreSQL data directories, Redis data directories, and RustFS data directories are not part of this synchronization scope.

Connection Methods and Network Baseline

Access Relationships

Source	Target	Purpose
Users / site systems	Business VIP	Web/API business access
D/E/F Enterprise	PostgreSQL VIP	Business database reads/writes
D/E/F Enterprise	Redis VIP	Cache and session
D/E/F Enterprise	RustFS VIP	S3 object reads/writes

Source	Target	Purpose
Deployer controller	A-F	SSH, deployment, status collection, operations
D/E/F	D/E/F	rsync/SSH file synchronization
A/B/C	A/B/C	PostgreSQL streaming replication, repmgr, Witness communication
RustFS members/proxies	C/D/E/F	RustFS cluster and S3 backend communication
Nodes sharing a VIP	Corresponding candidates	VRRP advertisement and VIP movement

Ports and Protocols

Allow ports by minimum required source-to-target relationships. Do not expose all management ports directly to the user network.

Port / protocol	Suggested allowlist	Purpose
`22/TCP`	Deployer -> A-F; D/E/F mutual sync	SSH, deployment, rsync
`8088/TCP`	User network -> Business VIP	Enterprise business entry
`5432/TCP`	D/E/F -> PostgreSQL VIP; PG/proxy internal access	PostgreSQL, streaming replication, proxy
`6379/TCP`	D/E/F -> Redis VIP; health-check nodes -> C/F	Redis access and health checks
`19000/TCP`	D/E/F -> RustFS VIP	Enterprise fixed S3 entry
`9000/TCP`	A/B -> C/D/E/F; RustFS members	RustFS S3 backend/cluster communication; confirm

Port / protocol	Suggested allowlist	Purpose
		by version
`9001/TCP`	Controlled operations network -> RustFS admin entry	Optional RustFS admin endpoint; do not expose to user network
`19731/TCP`	Localhost / Keepalived / controlled operations network	Fleet Agent HTTP health and control interface
`18080/TCP`	Operations network -> deployer controller	Deployer HTTPS UI
`112/VRRP`	Candidate nodes for each VIP	Keepalived VIP movement; this is an IP protocol number, not TCP/UDP
`1883/8883`	Site device network -> business entry, per product configuration	MQTT/MQTTS

Failover and Capability Boundaries

Failure scenario	Expected behavior	Boundary
D Active business node fails	Business VIP moves to E or F; the new Owner starts the business group after readiness passes	Requires the latest HA fileset generation to be complete on Standby
PostgreSQL A Primary fails	B is promoted to Primary; PostgreSQL VIP continues routing to the writable Primary through D/E proxy	Data loss depends on streaming replication state
Redis current node fails	Redis VIP moves to the other independent Redis instance	Cache/session may be lost

Failure scenario	Expected behavior	Boundary
RustFS proxy A or B fails	RustFS VIP moves to the other proxy	RustFS data still depends on the four-node cluster health
Single RustFS data node fails	RustFS cluster continues service if its redundancy conditions are met	Actual boundary depends on disk count and erasure-coding configuration

Network partition, simultaneous multi-node failure, incorrect VRRP priority, full disks, clock drift, and manual changes outside the deployer are outside the normal automatic recovery promise and must be covered by operations procedures.

Backup, Monitoring, and Inspection

- PostgreSQL requires independent logical or physical backup and restore verification.
- RustFS requires capacity, disk, and cluster health monitoring.
- Redis must be treated as cache/session unless a future replication design is introduced.
- Fleet Agent and Keepalived state must be monitored on D/E/F.
- rsync generation lag must be monitored. A Standby with stale generation must not take over business traffic.
- Regular failover drills must include business VIP, PostgreSQL promotion, Redis VIP switch, RustFS proxy switch, and restore from backup.

Pre-Go-Live Checks

- All six machines can be reached by SSH from the deployer controller.
- Hostnames, IP addresses, NIC names, NTP, DNS, and firewall rules are fixed and recorded.
- The four VIPs are unused and can move between their candidate nodes.
- Disk mount points are fixed and survive reboot.
- Enterprise, PostgreSQL, Redis, RustFS, Keepalived, HAProxy, repmgr, Fleet Agent, and rsync configurations are generated by the deployer and reviewed.
- The Cluster License is activated once through the Business VIP.
- Backup and restore steps are rehearsed before production traffic is introduced.

Implementation Sequence

1. Prepare the deployer controller and the six machines.
2. Confirm IP, VIP, NIC, disk, DNS, NTP, SSH, and firewall information.
3. Upload the Enterprise offline package and configuration material.
4. Deploy PostgreSQL primary/standby and Witness.
5. Deploy Redis independent instances and Redis VIP.
6. Deploy RustFS four-node cluster and RustFS VIP.
7. Deploy Enterprise on D/E/F, Fleet Agent, rsync filesets, and Business VIP.
8. Activate License through the Business VIP.
9. Run functional validation and failover drills.
10. Record final topology, passwords/keys, backup policy, monitoring targets, and recovery procedures.

Acceptance Criteria

Functional Acceptance

- Users can access Enterprise through the Business VIP.
- Enterprise reads and writes PostgreSQL only through PostgreSQL VIP.
- Enterprise uses Redis only through Redis VIP.
- Enterprise uses RustFS only through RustFS VIP.
- App, Flow, Marimo, Notebook, and License files are present on the Active node after failover.
- Branch Enterprise can still be installed independently and does not join this six-machine HA group.

Failover Drill Acceptance

- Stop the Active business node: Business VIP moves to a healthy Standby and the business group starts only there.
- Stop PostgreSQL Primary: Standby is promoted and PostgreSQL VIP continues pointing to the writable Primary.

- Stop the Redis VIP owner: Redis VIP moves to the other instance, with cache/session loss accepted.
- Stop the RustFS proxy owner: RustFS VIP moves to the other proxy.
- Stop one RustFS data node: object access remains within the actual RustFS redundancy boundary.
- Recover failed nodes: they return as Standby or healthy members and do not overwrite newer data or files.

Risks and Items to Confirm On Site

- Whether the network allows VRRP protocol `112`.
- Whether the four VIP addresses are truly unused.
- Whether the disk mount layout and capacity meet actual file retention and RustFS redundancy requirements.
- Whether Redis only stores disposable data.
- Whether PostgreSQL replication lag and backup policy meet the business RPO.
- Whether operations staff understand the difference between VIP failover, file synchronization, database replication, and object storage redundancy.

Conclusion

The current plan centers on six machines, four fixed VIPs, and single-active Enterprise. It decouples the business entry, database entry, cache entry, and object storage entry. It can cover common single-machine failures within the same network segment and significantly reduces the work required after failover to change connection configuration, reactivate License, and manually copy App files.

At the same time, three boundaries must be accepted: Redis does not replicate data, business filesets are synchronized asynchronously, and network partition has no consensus arbitration. Therefore, this plan is suitable to define as a **Fleet Center six-machine single-active active-standby disaster recovery plan**. It should not promise zero interruption, zero data loss, or strict cross-data-center high availability.

REFERENCE

SLA and High Availability Boundaries

SLA recommendations, high availability boundaries, acceptance gates, drills, and hardening items for the standard Tier0 Enterprise deployment.

This document applies to the six-VM fully offline deployment: three application / Swarm nodes, two PostgreSQL HA nodes, and one witness node.

Architecture Baseline

Layer	Current Design	High Availability Boundary
Application runtime	Enterprise New compose runs on the fixed `tier0_app_active_node`.	`backend`, `redis`, `EMQX`, `SourceFlow`, `EventFlow`, `Marimo`, and `OPC UA Server` are not automatically multi-replica services yet.
HTTP entry	Three application nodes run keepalived APP VIP + HAProxy.	The VIP can float. HAProxy forwards to the active node on `8088/TCP` and checks `/healthz`.
MQTT / OPC UA	EMQX uses `1883/8883/8083/8084`; OPC UA uses `4840`.	These connect directly to the active node. They are not included in TCP VIP proxying, so protocol-entry HA is not promised.
Database	PostgreSQL / TimescaleDB streaming replication + repmgr + witness + DB VIP.	If the primary fails, the standby can be promoted automatically. Asynchronous replication cannot promise strict zero RPO.

Layer	Current Design	High Availability Boundary
Backup and restore	`backup-db.sh` / `restore-db.sh`.	Provides recoverability, but it is not the same as online HA.

Recommended SLA

Service Object	Recommended Statement	Constraints
Platform HTTP entry	Start from monthly availability of `99.5%`.	VIP failover is not the same as automatic migration of the active application node.
Database access	Operations target: `RTO <= 2 minutes`.	RPO depends on replication lag. Historical failover tests on the same architecture were about 59-64 seconds.
MQTT / OPC UA	No HA SLA is promised in the current design.	HAProxy TCP proxying, a separate VIP, or service clustering must be added first.
Backup and restore	Daily backups; mandatory backup before major changes.	Restore time depends on data volume and on-site I/O.

Before claiming `99.9%` or higher, complete application automatic migration or multi-replica deployment, protocol-entry HA, monitoring alerts, and recovery drills. Do not describe planned capabilities as delivered capabilities.

Acceptance Gates

Each deployment or upgrade must satisfy at least the following:

- `verify-materials.sh` passes, and the delivery package SHA256 checks pass.

- The three Swarm nodes are `Ready`.
- `APP VIP /healthz` and `/readyz` return `200`, and the home page and `/uns` are accessible.
- The seven compose services on the active node are running.
- `DB VIP:5432` is accessible, and the primary, standby, and witness topology is normal.
- The default account can actually log in. Do not judge IAM initialization success only from ports, containers, or HTTP 200 responses.
- Smoke checks for HTTP, EMQX ports, WebSocket/WSS, and OPC UA TCP pass.

Inspection and Alerts

Run the following every day:

```
./bin/verify.sh --site config/site.conf
./bin/smoke-business-emqx.sh --site config/site.conf
```

Configure at least the following alerts:

- APP VIP `/healthz` or `/readyz` failure.
- DB VIP unreachable, no primary, split-brain risk, replication interruption, or replication lag above threshold.
- keepalived, HAProxy, PostgreSQL, or repmgrd abnormal status.
- Key containers such as `backend`, `redis`, and `EMQX` exit or restart frequently.
- Data disk usage exceeds `80% / 90%`.
- The latest backup failed or was not generated.

Drill Requirements

Drill	Frequency	Verification Criteria
APP VIP failover	Before delivery and quarterly.	After stopping keepalived on the current VIP node,

Drill	Frequency	Verification Criteria
		`/healthz` and `/readyz` recover.
DB primary failure	Before delivery and every six months.	The standby is promoted, DB VIP recovers, and the old primary is rebuilt as a standby.
Database restore	Monthly sample and before major changes.	The backup is restorable. After restore, `verify`, smoke checks, and login verification pass.
Material verification	Every package build.	Source materials, package SHA, and unpacking verification pass.

Incident Handling Principles

1. Confirm the impact scope first: HTTP, MQTT, OPC UA, database, and single-node impact.
2. Prioritize restoring VIP, HAProxy, the active application, and the database primary.
3. Preserve logs, data directories, and backups. Do not directly clean or overwrite the incident site.
4. For database incidents, determine the current primary first, then decide whether to fail over, rejoin, or restore.
5. After recovery, run `verify`, smoke checks, and actual login verification.

High-SLA Hardening Items

- Make the application multi-replica, or establish a verifiable automatic migration process for the active node.
- Cluster EMQX, or route it through HAProxy TCP / TLS with a dedicated VIP.
- Provide a high-availability entry for OPC UA through HAProxy TCP, a dedicated VIP, or service clustering.

- Use PostgreSQL synchronous replication, or define acceptable replication lag and RPO clearly.
- Automate backup scheduling, off-site replication, and regular restore drills.
- Connect containers, hosts, databases, logs, and black-box probes to unified alerting.

Related procedures: **Operations Runbook** and **Standard Port List**.

REFERENCE

Standard Port List

Firewall port list for the standard six-VM offline Tier0 Enterprise deployment.

The following table is based on the Enterprise New six-VM offline deployment. Customer firewalls should allow traffic from the minimum required source ranges only. Do not expose these ports directly to the public internet.

Source	Target	Port	Protocol	Purpose
Control host	Six target VMs	22	TCP	SSH / Ansible
Three application nodes, mutually	Three application nodes	2377	TCP	Docker Swarm management plane
Three application nodes, mutually	Three application nodes	7946	TCP/UDP	Swarm node discovery
Three application nodes, mutually	Three application nodes	4789	UDP	Swarm overlay
Three application nodes	Active application node	8088	TCP	HAProxy to Enterprise New backend; application network segment only
Business network segment	APP VIP	80	TCP	Platform HTTP entry, `/healthz`, `/readyz`

Source	Target	Port	Protocol	Purpose
Business network segment	APP VIP / active application node	1883	TCP	MQTT
Business network segment	APP VIP / active application node	8883	TCP	MQTT TLS
Business network segment	APP VIP / active application node	8083	TCP	MQTT WebSocket
Business network segment	APP VIP / active application node	8084	TCP	MQTT WebSocket TLS
Business network segment	APP VIP / active application node	4840	TCP	OPC UA Server
Controlled application network segment	Active application node	8089	TCP	Launchpad
Dynamic app access network segment	Active application node	30000-30199	TCP	App Gateway / dynamic application host ports
Application nodes, DB	DB VIP / DB nodes	5432	TCP	PostgreSQL / TimescaleDB

Source	Target	Port	Protocol	Purpose
nodes, and witness				
DB1, DB2, and witness	DB1, DB2, and witness	5432	TCP	Streaming replication / repmgr
Application nodes	NFS server	2049	TCP/UDP	Used when `NFS_ENABLED=true`

The current compose deployment does not expose Redis, SourceFlow, EventFlow, Marimo, or the EMQX Dashboard externally. These services are only called by the backend inside the Docker runtime network.

REFERENCE

Operations Runbook

Daily inspection, failover, backup, restore, cleanup, and troubleshooting procedures for the standard Tier0 Enterprise deployment.

This document is based on the Enterprise New offline application.

Daily Inspection

1. Run the following commands from the deployment package root directory:

```
./bin/verify.sh --site config/site.conf
./bin/smoke-business-emqx.sh --site config/site.conf
```

2. Confirm the following items:
   - The 3 Swarm nodes are `Ready`.
   - `APP VIP /healthz` and `/readyz` return `200`.
   - `DB VIP:5432` is accessible.
   - The PostgreSQL primary, standby, and witness topology is normal.
   - `backend`, `redis`, `emqx`, `sourceflow`, `eventflow`, `marimo`, and `opcua-server` are all running on the active application node.

Application Checks

1. Run the following commands on the active application node:

```
cd /opt/tier0-enterprise/tier0-deploy
bash bin/compose.sh ps
bash bin/compose.sh logs --tail=200 backend
curl -fsS http://127.0.0.1:8088/healthz
curl -fsS http://127.0.0.1:8088/readyz
```

> ⓘ Note

Do not bypass `bin/compose.sh` and run raw `docker compose` commands directly. The wrapper script loads the runtime environment generated by the installer.

2. Health-check interpretation:
   - `healthz` failure: check the backend process, image, and container logs first.
   - `readyz` failure: also check DB VIP, Redis, and database migration logs.
   - Home page reachable but login unavailable: confirm `ADMIN_INITIAL_PASSWORD` in `.env`, then check backend IAM migration logs and user tables. Do not judge deployment success only from the HTTP port.

Protocol Entry Checks

```
nc -vz <APP_VIP> 1883
nc -vz <APP_VIP> 8883
nc -vz <APP_VIP> 8083
nc -vz <APP_VIP> 8084
nc -vz <APP_VIP> 4840
```

> **Note**
> EMQX and OPC UA Server are fixed on the active application node. APP VIP failover only guarantees the `APP VIP:80` HTTP entry. Protocol ports are not routed through HAProxy TCP proxying, so this design does not promise high availability for protocol entries.

Database Checks

Run the following on database nodes:

```
systemctl status tier0-postgresql
systemctl status tier0-repmgrd
sudo -iu postgres psql -Atqc 'select pg_is_in_recovery();'
sudo -iu postgres /usr/local/bin/repmgr -f /etc/repmgr.conf cluster show
```

- `pg_is_in_recovery=false` means primary.

- `pg_is_in_recovery=true` means standby.
- DB VIP must only be located on the current primary.
- If standby replication is interrupted or the witness reports upstream errors, preserve the site first. Do not rerun the full deployment directly.

APP VIP Failover

1. Before testing, record the node that currently owns the VIP and verify:

```
curl -fsS http://<APP_VIP>/healthz
curl -fsS http://<APP_VIP>/readyz
```

2. After stopping `keepalived` on the current VIP node, confirm VIP takeover on another application node, then retest both URLs. The HAProxy backend points to the active application node on `8088/TCP`, so all three application nodes must be able to reach port 8088 on the active node.
3. After restoring `keepalived`, run `verify.sh` and smoke checks again. Do not stop active application containers during the drill.

DB Primary Failure Switchover

Before a failure drill, create a backup and confirm replication is healthy:

```
./bin/backup-db.sh --site config/site.conf
```

After stopping `tier0-postgresql` on the current primary, observe `repmgr` promoting the standby and DB VIP takeover. The target RTO is within 2 minutes. Historical tests on the same architecture were about 59-64 seconds.

> ⚠ **Caution**
>
> - The old primary must not be started directly back into the production topology. Stop PostgreSQL, repmgrd, and keepalived first; remove any remaining DB VIP; then run basebackup from the current new primary, register the node as a standby, and finally reconnect the witness.

- After failover, do not rerun the full installer directly. Static primary/standby definitions in the inventory may disturb the current topology.

Backup and Restore

- Backup:

```
./bin/backup-db.sh --site config/site.conf
```

- Restore:

```
./bin/restore-db.sh --site config/site.conf --restore-file /backup/tier0-db/postgres-YYYYmmddTHHMMSS.dump
```

> ⓘ **Note**
> Before restoration, confirm the current primary. After restoration, run `verify.sh`, smoke checks, and actual platform login verification for the default-user path.

Cleanup and Retest

```
./full-cleanup.sh --env install.env --yes
./install.sh --auto --yes
```

> ⚠ **Caution**
> Full cleanup deletes Tier0 runtime data, Docker/Swarm data, and database backups on the 6 nodes. It also performs final cleanup on the control node after the control container exits. Use this entry only for test environments that are confirmed disposable. If customer data or an incident site must be preserved, use normal `--cleanup` and complete a backup first.

Common Issues

Backend Startup Failure

```
cd /opt/tier0-enterprise/tier0-deploy
bash bin/compose.sh logs --tail=300 backend
bash bin/compose.sh ps
```

> ⓘ Note
> Focus on DB URL, DB VIP, Redis, migration, and mounted directory permissions.

EMQX Startup Failure

```
cd /opt/tier0-enterprise/tier0-deploy
bash bin/compose.sh logs --tail=200 emqx
```

> ⓘ Note
> Check certificates, configuration files, volume permissions, and whether ports `1883/8883/8083/8084` are occupied.

repmgrd Repeated Restarts

```
systemctl status tier0-repmgrd
tail -100 /var/log/repmgr/repmgr.log
sudo -iu postgres psql -Atqc 'show shared_preload_libraries;'
```

> ⓘ Note
> Confirm that `shared_preload_libraries` includes `repmgr`, and that the systemd unit uses `repmgrd --daemonize=false`.

Material Verification Failure

```
./bin/verify-materials.sh
```

> ⚠ **Caution**
> Do not skip SHA256 verification and continue deployment. Re-acquire the delivery package or rebuild the materials following the traceable process.

REFERENCE

OpenAPI Reference Appendix

Tier0 OpenAPI v1 · Operations: 36 · Schemas: 85

Operations

API field names, examples, and source descriptions follow the bundled English OpenAPI specification.

UNS

Read, write, and manage Unified Namespace data.

POST /uns/attachments

Upload a UNS attachment
Uploads a multipart attachment for a UNS node. Supply a file and at least one of unsId or topic; a valid unsId takes precedence.

Request body

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/attachments' \
  --header 'x-api-key: <api-key>'
```

`multipart/form-data`

Field	Type	Required	Description
`unsId`	`integer (int64)`	No	Numeric owner node ID. Supply unsId or topic; a valid unsId takes precedence.
`topic`	`string`	No	Full owner node path, used when a valid unsId is not supplied.
`fileName`	`string`	No	Original file name.
`sha256`	`string`	No	SHA-256 checksum of the file contents.
`file`	`string (binary)`	Yes	Binary file contents. The multipart field name must be file.

Responses

200 400 401 403 404 409 413 500

Success response example

`200 · application/json`


```
{
  "code": 0,
  "msg": "string",
  "data": {
    "unsId": 0,
    "fileName": "string",
    "filePath": "string",
    "fileUrl": "string",
    "contentType": "string"
  }
}
```


Success response fields

`200 · application/json`

Field	Type	Required	Description
`code`	`integer`	No	Response code
`msg`	`string`	No	Response message
`data`	`object`	No	UnsAttachmentUploadResp. See the referenced schema in the Schemas section.

Response status meanings

Status	Meaning	Media type
`200`	A successful response.	`application/json`
`400`	Correct the request parameters or Content-Type before retrying.	`application/json`
`401`	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	`text/plain`
`403`	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	`text/plain`
`404`	Resource not found. Verify paths, IDs, and current resource state.	–
`409`	Resource state conflict. Verify paths, IDs, and current resource state.	–
`413`	Request too large. Reduce the request before retrying.	–
`500`	Service error. Investigate the service and confirm the outcome before retrying.	–

POST `/uns/attachments/list`

List UNS attachments

Lists attachments for a UNS node or topic. Temporary file URLs can expire and must then be requested again.

Request body

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/attachments/list' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"unsId":0,"topic":"string","page":0,"size":0,"includeFileUrl":false}'
```

`application/json`

Field	Type	Required	Description
`unsId`	`integer (int64)`	No	Numeric UNS node ID.
`topic`	`string`	No	Full UNS topic path.
`page`	`integer (int64)`	No	Page number, starting at 1.
`size`	`integer (int64)`	No	Number of attachments per page.
`includeFileUrl`	`boolean (boolean)`	No	Whether to return temporary file URLs. Defaults to true.

Responses

`200` `400` `401` `403` `404` `409` `413` `500`

Success response example

`200 · application/json`

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "unsId": 0,
    "list": [
      "value"
    ],
    "total": 0,
    "page": 0,
    "size": 0
  }
}
```

Success response fields

`200 · application/json`

Field	Type	Required	Description
`code`	`integer`	No	Response code

Field	Type	Required	Description
msg	string	No	Response message
data	object	No	UnsAttachmentListResp. See the referenced schema in the Schemas section.

Response status meanings

Status	Meaning	Media type
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/browse

Browse the UNS tree

Returns a UNS subtree. Use an empty path for the workspace root.

Request body

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/browse' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"path":"string","max_depth":0,"include_metadata":false,"include_leaf_value":false}'
```

application/json

Field	Type	Required	Description
path	string	No	Root UNS path. Empty means the workspace root.
max_depth	integer (int64)	No	Maximum tree depth. Omit or use 0 for the server default.
include_metadata	boolean (boolean)	No	Whether returned nodes should include metadata.

Field	Type	Required	Description
include_leaf_value	boolean (boolean)	No	Whether returned leaf nodes should include their latest payload.

Responses

200 400 401 403 404 409 413 500

Success response example

200 · application/json


```
{
  "code": 0,
  "msg": "string",
  "data": {
    "tree": [
      "value"
    ]
  }
}
```


Success response fields

200 · application/json

Field	Type	Required	Description
code	integer	No	Response code
msg	string	No	Response message
data	object	No	BrowseResp. See the referenced schema in the Schemas section.

Response status meanings

Status	Meaning	Media type
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–

Status	Meaning	Media type
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/create

Create UNS nodes
Creates UNS nodes and nested subtrees. Check each item result and use the returned topic in subsequent calls.

Request body

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/create' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"namespace":
[{"name":"value","type":"value","children":"value","topicType":"value","fields":"value"}]}'
```

`application/json`

Field	Type	Required	Description
namespace	array<object>	Yes	UNS nodes to create. Nest children to create a subtree.

Responses

200 400 401 403 404 409 413 500

Success response example

`200 · application/json`

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "success": false,
    "results": [
      "value"
    ]
  }
}
```

Success response fields

`200 · application/json`

Field	Type	Required	Description
code	integer	No	Response code

Field	Type	Required	Description
msg	string	No	Response message
data	object	No	CreateResp. See the referenced schema in the Schemas section.

Response status meanings

Status	Meaning	Media type
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/delete

Delete UNS nodes

Deletes the listed nodes sequentially. By default this is a soft delete; the batch is not rolled back when a later item fails.

Request body

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/delete' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"topics":["string"],"hard_delete":false}'
```

application/json

Field	Type	Required	Description
topics	array<string>	Yes	UNS topic or path node paths to delete.
hard_delete	boolean (boolean)	No	Permanently delete instead of moving nodes to the recycle state.

Responses

200 400 401 403 404 409 413 500

Success response example

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "success": false
  }
}
```

Success response fields

200 · application/json

Field	Type	Required	Description
code	integer	No	Response code
msg	string	No	Response message
data	object	No	OpenapiEmptyResp. See the referenced schema in the Schemas section.

Response status meanings

Status	Meaning	Media type
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/history

Query UNS history
Queries historical topic values over an inclusive RFC 3339 time range. Results can be aggregated or sampled.

Request body

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/history' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"topics":["string"],"start_time":"string","end_time":"string","aggregation":
{"interval":"string","function":"string","field":"string","fields":["value"]},"page":0}'
```

application/json

Field	Type	Required	Description
topics	array<string>	Yes	UNS topic paths whose history should be queried.
start_time	string	Yes	Inclusive query start in RFC 3339 format.
end_time	string	Yes	Inclusive query end in RFC 3339 format.
aggregation	object	No	HistoryAggregation. See the referenced schema in the Schemas section.
page	integer (int64)	No	Page number, starting from 1.
size	integer (int64)	No	Page size; values above the server maximum are capped.

Responses

200 400 401 403 404 409 413 500

Success response example

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "success": false,
    "results": [
      "value"
    ],
    "page": 0,
    "size": 0,
    "total": 0
  }
}
```


Success response fields

`200 · application/json`

Field	Type	Required	Description
`code`	`integer`	No	Response code
`msg`	`string`	No	Response message
`data`	`object`	No	HistoryResp. See the referenced schema in the Schemas section.

Response status meanings

Status	Meaning	Media type
`200`	A successful response.	`application/json`
`400`	Correct the request parameters or Content-Type before retrying.	`application/json`
`401`	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	`text/plain`
`403`	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	`text/plain`
`404`	Resource not found. Verify paths, IDs, and current resource state.	–
`409`	Resource state conflict. Verify paths, IDs, and current resource state.	–
`413`	Request too large. Reduce the request before retrying.	–
`500`	Service error. Investigate the service and confirm the outcome before retrying.	–

POST `/uns/read`

Read UNS values

Returns value, quality, and timeStamp (VQT) for each requested topic. A failed topic does not discard successful items in the same batch.

Request body

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/read' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"topics":["string"],"include_metadata":false,"include_leaf_value":false}'
```

`application/json`

Field	Type	Required	Description
`topics`	`array<string>`	Yes	UNS topic paths to read.
`include_metadata`	`boolean (boolean)`	No	Whether each result should include UNS node metadata.
`include_leaf_value`	`boolean (boolean)`	No	Whether leaf nodes should include their latest payload.

Responses

200 400 401 403 404 409 413 500

Success response example

`200 · application/json`

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "success": false,
    "results": [
      "value"
    ]
  }
}
```

Success response fields

`200 · application/json`

Field	Type	Required	Description
`code`	`integer`	No	Response code
`msg`	`string`	No	Response message
`data`	`object`	No	ReadResp. See the referenced schema in the Schemas section.

Response status meanings

Status	Meaning	Media type
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/restore

Restore a deleted UNS node

Restores a soft-deleted UNS node. Permanently deleted nodes cannot be restored.

Request body

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/restore' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"path":"string"}'
```

application/json

Field	Type	Required	Description
path	string	Yes	Full UNS path of the node to restore. Use its path before deletion, not a node ID or HTTP URL.

Responses

200 400 401 403 404 409 413 500

Success response example

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "success": false
  }
}
```


Success response fields

`200 · application/json`

Field	Type	Required	Description
`code`	`integer`	No	Response code
`msg`	`string`	No	Response message
`data`	`object`	No	OpenapiEmptyResp. See the referenced schema in the Schemas section.

Response status meanings

Status	Meaning	Media type
`200`	A successful response.	`application/json`
`400`	Correct the request parameters or Content-Type before retrying.	`application/json`
`401`	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	`text/plain`
`403`	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	`text/plain`
`404`	Resource not found. Verify paths, IDs, and current resource state.	–
`409`	Resource state conflict. Verify paths, IDs, and current resource state.	–
`413`	Request too large. Reduce the request before retrying.	–
`500`	Service error. Investigate the service and confirm the outcome before retrying.	–

POST `/uns/search`

Search UNS nodes

Searches UNS node names, aliases, and paths, with optional type and path filters.

Request body

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/search' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data
'{"keyword":"string","topicType":"string","path_prefix":"string","include_metadata":false,"include_
leaf_value":false}'
```

`application/json`

Field	Type	Required	Description
`keyword`	`string`	No	Keyword matched against UNS node names, aliases, and paths.
`topicType`	`string`	No	Topic data category filter: STATE, ACTION, or METRIC.
`path_prefix`	`string`	No	Only return nodes under this UNS path prefix.
`include_metadata`	`boolean (boolean)`	No	Whether returned nodes should include metadata.
`include_leaf_value`	`boolean (boolean)`	No	Whether returned leaf nodes should include their latest payload.
`page`	`integer (int64)`	No	Page number, starting from 1.
`size`	`integer (int64)`	No	Page size.

Responses

200 400 401 403 404 409 413 500

Success response example

`200 · application/json`

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "objects": [
      "value"
    ],
    "page": 0,
    "size": 0,
    "total": 0
  }
}
```

Success response fields

`200 · application/json`

Field	Type	Required	Description
code	integer	No	Response code
msg	string	No	Response message
data	object	No	SearchResp. See the referenced schema in the Schemas section.

Response status meanings

Status	Meaning	Media type
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/unsBindFlow

Bind a flow to a UNS node
Binds a flow ID to a UNS node ID.

Request body

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/unsBindFlow' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"unsId":0,"flowId":0}'
```

application/json

Field	Type	Required	Description
unsId	integer (int64)	Yes	UNS node ID to bind.
flowId	integer (int64)	Yes	Flow ID to bind.

Responses

200 400 401 403 404 409 413 500

Success response example

200 · application/json


```
{
  "code": 0,
  "msg": "string",
  "data": {
    "success": false,
    "unsId": 0,
    "flowId": 0
  }
}
```


Success response fields

200 · application/json

Field	Type	Required	Description
code	integer	No	Response code
msg	string	No	Response message
data	object	No	UnsBindFlowResp. See the referenced schema in the Schemas section.

Response status meanings

Status	Meaning	Media type
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/update

Update a UNS node

Updates an existing UNS node. Empty strings, an empty fields array, and an empty extendProperties object retain existing values.

Request body

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/update' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data
'{"path":"string","name":"string","description":"string","displayName":"string","extendProperties":
{}}'
```

application/json

Field	Type	Required	Description
path	string	Yes	Existing UNS topic or path node to update.
name	string	No	New node name.
description	string	No	Updated node description.
displayName	string	No	Updated display name.
extendProperties	object	No	Updated custom metadata key-value pairs.
fields	array<object>	No	Updated payload schema fields.
alias	string	No	Updated topic alias.
enableHistory	boolean (boolean)	No	Supply true or false to change history storage. Omit to keep the existing setting.
updateMask	array<string>	No	Compatibility field ignored by the current Enterprise implementation. It cannot be used to clear fields.

Responses

200 400 401 403 404 409 413 500

Success response example

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "success": false
  }
}
```


Success response fields

`200 · application/json`

Field	Type	Required	Description
`code`	`integer`	No	Response code
`msg`	`string`	No	Response message
`data`	`object`	No	OpenapiEmptyResp. See the referenced schema in the Schemas section.

Response status meanings

Status	Meaning	Media type
`200`	A successful response.	`application/json`
`400`	Correct the request parameters or Content-Type before retrying.	`application/json`
`401`	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	`text/plain`
`403`	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	`text/plain`
`404`	Resource not found. Verify paths, IDs, and current resource state.	–
`409`	Resource state conflict. Verify paths, IDs, and current resource state.	–
`413`	Request too large. Reduce the request before retrying.	–
`500`	Service error. Investigate the service and confirm the outcome before retrying.	–

POST `/uns/write`

Write UNS values

Writes values to existing TOPIC nodes; wildcards are not supported. `value` must match the node payload schema. **Batch results:** check each `data.results` item, not only HTTP 200. **Retry:** confirm the outcome before retrying a write.

Request body

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/write' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"writes":
[{"topic":"value","value":"value","timeStamp":"value"}],"qos":0,"retain":false}'
```

`application/json`

Field	Type	Required	Description
`writes`	`array<object>`	Yes	Write operations to apply.
`qos`	`integer (int64)`	No	MQTT QoS: 0, 1, or 2.
`retain`	`boolean (boolean)`	No	Whether the MQTT message should be retained.

Responses

`200` `400` `401` `403` `404` `409` `413` `500`

Success response example

`200 · application/json`

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "success": false,
    "results": [
      "value"
    ]
  }
}
```

Success response fields

`200 · application/json`

Field	Type	Required	Description
`code`	`integer`	No	Response code
`msg`	`string`	No	Response message
`data`	`object`	No	WriteResp. See the referenced schema in the Schemas section.

Response status meanings

Status	Meaning	Media type
200	Inspect the success flag of each `data.results` item. HTTP 200 alone does not confirm that every write succeeded.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

Assets

Upload, download, and manage stored files.

POST /assets/files

Create a direct-upload form

Issues an object-store upload form; it does not transfer file contents. Submit every returned post field unchanged with the file.

Request body

```
curl --request POST 'https://<tier0-host>/openapi/v1/assets/files' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data
'{"fileName":"string","contentType":"string","size":0,"business":"string","useBy":"string"}'
```

application/json

Field	Type	Required	Description
fileName	string	Yes	Original file name.
contentType	string	No	MIME content type.

Field	Type	Required	Description
`size`	`integer (int64)`	Yes	File size in bytes. Business validation requires a value greater than 0 and at most 1 GiB.
`business`	`string`	No	Business domain or feature associated with the file.
`useBy`	`string`	No	Optional usage category: user, workspace, or platform.
`visibility`	`string`	No	File visibility, such as private or public.
`appInstanceId`	`string`	No	Associated application instance ID.
`sessionId`	`string`	No	Caller session ID.

Responses

`200` `400` `401` `403` `404` `409` `413` `500`

Success response example

`200 · application/json`

```
{
  "code": 0,
  "data": {
    "fileId": 0,
    "filePath": "string",
    "fileUrl": "string",
    "uploadUrl": "string",
    "expiresAt": 0
  },
  "msg": "string"
}
```

Success response fields

`200 · application/json`

Field	Type	Required	Description
`code`	`integer`	No	Response code
`data`	`object`	No	AssetFileUploadResp. See the referenced schema in the Schemas section.
`msg`	`string`	No	Response message

Response status meanings

Status	Meaning	Media type
`200`	A successful response.	`application/json`
`400`	Correct the request parameters or Content-Type before retrying.	`application/json`

Status	Meaning	Media type
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /assets/files/delete

Delete a file

Deletes a file identified by its server-issued storage path.

Request body

```
curl --request POST 'https://<tier0-host>/openapi/v1/assets/files/delete' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"filePath":"string"}'
```

application/json

Field	Type	Required	Description
filePath	string	Yes	Server-issued storage path returned by an upload API.

Responses

200 400 401 403 404 409 413 500

Success response example

200 · application/json

```
{
  "code": 0,
  "data": {
    "deleted": false
  },
  "msg": "string"
}
```

Success response fields

`200 · application/json`

Field	Type	Required	Description
`code`	`integer`	No	Response code
`data`	`object`	No	AssetFileDeleteResp. See the referenced schema in the Schemas section.
`msg`	`string`	No	Response message

Response status meanings

Status	Meaning	Media type
`200`	A successful response.	`application/json`
`400`	Correct the request parameters or Content-Type before retrying.	`application/json`
`401`	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	`text/plain`
`403`	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	`text/plain`
`404`	Resource not found. Verify paths, IDs, and current resource state.	–
`409`	Resource state conflict. Verify paths, IDs, and current resource state.	–
`413`	Request too large. Reduce the request before retrying.	–
`500`	Service error. Investigate the service and confirm the outcome before retrying.	–

`GET` `/assets/files/download`

Download a private file
Returns file bytes directly. Do not forward x-api-key or other credentials when a client follows a redirect to a cross-origin storage URL.

Request parameters

Name	Location	Type	Required	Description
`filePath`	query	`string`	Yes	Server-issued storage path returned by an upload API.
`responseContentDisposition`	query	`string`	No	Optional Content-Disposition value for the download response.

Responses

302 400 401 403 404 409 413 500

Response status meanings

Status	Meaning	Media type
302	Redirect to the final download URL.	–
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /assets/files/multipart/abort

Abort a multipart upload

Cancels a multipart upload session.

Request body

```
curl --request POST 'https://<tier0-host>/openapi/v1/assets/files/multipart/abort' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"fileKey":"string","uploadId":"string"}'
```

application/json

Field	Type	Required	Description
fileKey	string	Yes	Object-store key used for the multipart upload.
uploadId	string	Yes	Object-store multipart upload session ID.

Responses

200 400 401 403 404 409 413 500

Success response example

`200 · application/json`

```
{
  "code": 0,
  "data": {
    "aborted": 1
  },
  "msg": "string"
}
```

Success response fields

`200 · application/json`

Field	Type	Required	Description
`code`	`integer`	No	Response code
`data`	`object`	No	AssetFileMultipartAbortResp. See the referenced schema in the Schemas section.
`msg`	`string`	No	Response message

Response status meanings

Status	Meaning	Media type
`200`	A successful response.	`application/json`
`400`	Correct the request parameters or Content-Type before retrying.	`application/json`
`401`	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	`text/plain`
`403`	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	`text/plain`
`404`	Resource not found. Verify paths, IDs, and current resource state.	–
`409`	Resource state conflict. Verify paths, IDs, and current resource state.	–
`413`	Request too large. Reduce the request before retrying.	–
`500`	Service error. Investigate the service and confirm the outcome before retrying.	–

POST `/assets/files/multipart/complete`

Complete a multipart upload

Completes a multipart upload after every uploaded part number and ETag have been supplied.

Request body

```
curl --request POST 'https://<tier0-host>/openapi/v1/assets/files/multipart/complete' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"fileKey":"string","uploadId":"string","parts":[{"partNumber":"value","etag":"value"}]}'
```

`application/json`

Field	Type	Required	Description
`fileKey`	`string`	Yes	Object-store key used for the multipart upload.
`uploadId`	`string`	Yes	Object-store multipart upload session ID.
`parts`	`array<object>`	Yes	Uploaded part numbers and their ETags.

Responses

`200` `400` `401` `403` `404` `409` `413` `500`

Success response example

`200 · application/json`

```
{
  "code": 0,
  "data": {
    "filePath": "string",
    "fileUrl": "string",
    "sizeBytes": 0,
    "expiresAt": 0
  },
  "msg": "string"
}
```

Success response fields

`200 · application/json`

Field	Type	Required	Description
`code`	`integer`	No	Response code
`data`	`object`	No	AssetFileMultipartCompleteResp. See the referenced schema in the Schemas section.
`msg`	`string`	No	Response message

Response status meanings

Status	Meaning	Media type
`200`	A successful response.	`application/json`

Status	Meaning	Media type
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /assets/files/multipart/init

Initialize a multipart upload

Creates a multipart upload session and returns its ID, part size, and part count.

Request body

```
curl --request POST 'https://<tier0-host>/openapi/v1/assets/files/multipart/init' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"fileName":"string","contentType":"string","size":0,"partSize":0,"business":"string"}'
```

application/json

Field	Type	Required	Description
fileName	string	Yes	Original file name.
contentType	string	No	File MIME type, for example application/octet-stream.
size	integer (int64)	Yes	Total file size in bytes, greater than 0 and at most 1 GiB.
partSize	integer (int64)	No	Preferred part size in bytes. The server may normalize it.
business	string	No	Business domain or feature associated with the file.
useBy	string	No	Optional usage category: user, workspace, or platform.
visibility	string	No	File visibility: private or public.
appInstanceId	string	No	Associated application instance ID.
sessionId	string	No	Caller session ID.

Field	Type	Required	Description
bizId	string	No	Associated business identifier.

Responses

200 400 401 403 404 409 413 500

Success response example

200 · application/json


```
{
  "code": 0,
  "data": {
    "fileKey": "string",
    "filePath": "string",
    "uploadId": "string",
    "partSize": 0,
    "partCount": 0
  },
  "msg": "string"
}
```


Success response fields

200 · application/json

Field	Type	Required	Description
code	integer	No	Response code
data	object	No	AssetFileMultipartInitResp. See the referenced schema in the Schemas section.
msg	string	No	Response message

Response status meanings

Status	Meaning	Media type
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–

Status	Meaning	Media type
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /assets/files/multipart/part-urls

Create multipart upload URLs
Returns presigned PUT URLs for the requested one-based part numbers.

Request body

```
curl --request POST 'https://<tier0-host>/openapi/v1/assets/files/multipart/part-urls' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"fileKey":"string","uploadId":"string","partNumbers":[0]}'
```

application/json

Field	Type	Required	Description
fileKey	string	Yes	Object-store key used for the multipart upload.
uploadId	string	Yes	Object-store multipart upload session ID.
partNumbers	array<integer (int32)>	Yes	One-based part numbers to sign.

Responses

200 400 401 403 404 409 413 500

Success response example

200 · application/json

```
{
  "code": 0,
  "data": {
    "partUrls": [
      "value"
    ]
  },
  "msg": "string"
}
```

Success response fields

200 · application/json

Field	Type	Required	Description
`code`	`integer`	No	Response code
`data`	`object`	No	AssetFileMultipartPartUrlsResp. See the referenced schema in the Schemas section.
`msg`	`string`	No	Response message

Response status meanings

Status	Meaning	Media type
`200`	A successful response.	`application/json`
`400`	Correct the request parameters or Content-Type before retrying.	`application/json`
`401`	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	`text/plain`
`403`	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	`text/plain`
`404`	Resource not found. Verify paths, IDs, and current resource state.	–
`409`	Resource state conflict. Verify paths, IDs, and current resource state.	–
`413`	Request too large. Reduce the request before retrying.	–
`500`	Service error. Investigate the service and confirm the outcome before retrying.	–

`GET` `/assets/files/public`

Download a public file

Downloads bytes for a file explicitly marked public. An unguessable path is not authorization; do not mark sensitive files public.

Request parameters

Name	Location	Type	Required	Description
`filePath`	query	`string`	Yes	Server-issued storage path returned by an upload API.
`responseContentDisposition`	query	`string`	No	Optional Content-Disposition value for the download response.

Responses

`200` `400` `404` `409` `413` `500`

Response status meanings

Status	Meaning	Media type
200	Returns file bytes directly. Content-Type depends on the file type; no JSON envelope is used.	–
400	Correct the request parameters or Content-Type before retrying.	application/json
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

GET /assets/files/url

Create a temporary file URL
Creates a temporary URL for a file.

Request parameters

Name	Location	Type	Required	Description
filePath	query	string	Yes	Server-issued storage path returned by an upload API.
expiredSec	query	integer (int64)	No	Requested temporary URL lifetime in seconds.

Responses

200 400 401 403 404 409 413 500

Success response example

200 · application/json


```
{
  "code": 0,
  "data": {
    "fileUrl": "string",
    "expiresAt": 0
  },
  "msg": "string"
}
```


Success response fields

200 · application/json

Field	Type	Required	Description
code	integer	No	Response code
data	object	No	AssetFileUrlResp. See the referenced schema in the Schemas section.
msg	string	No	Response message

Response status meanings

Status	Meaning	Media type
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /assets/upload

Create a legacy asset upload URL

Legacy compatibility endpoint. The current Enterprise implementation returns an empty uploadUrl; use /assets/files for new integrations.

Request body

```
curl --request POST 'https://<tier0-host>/openapi/v1/assets/upload' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"fileName":"string","sizeBytes":0,"contentType":"string","size":0,"sha256":"string"}'
```

application/json

Field	Type	Required	Description
fileName	string	Yes	Original file name.
sizeBytes	integer (int64)	Yes	File size in bytes.

Field	Type	Required	Description
contentType	string	No	File MIME type, for example application/octet-stream.
size	integer (int64)	No	File size in bytes.
sha256	string	No	Optional SHA-256 checksum of the file content.
storageKey	string	No	Optional object-store key. Omit it to let the server generate one.
isPublic	boolean (boolean)	No	Whether the file should be accessible through the public download endpoint.
business	string	No	Business domain or feature that owns the uploaded file.
useBy	string	No	Usage scenario or owner category for the uploaded file.

Responses

200 400 401 403 404 409 413 500

Success response example

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "uploadUrl": "string",
    "filePath": "string",
    "fileUrl": "string"
  }
}
```

Success response fields

200 · application/json

Field	Type	Required	Description
code	integer	No	Response code
msg	string	No	Response message
data	object	No	AssetUploadResp. See the referenced schema in the Schemas section.

Response status meanings

Status	Meaning	Media type
200	Legacy upload response. The current Enterprise implementation returns an empty uploadUrl; use /assets/files for new integrations.	application/json

Status	Meaning	Media type
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

Authentication

Verify the current OpenAPI identity and permissions.

POST /auth/whoami

Inspect the current API key identity

Returns the current API-key identity and its effective permissions. Call this before integration work to verify access.

Responses

200 400 401 403 404 409 413 500

Success response example

200 · application/json


```
{
  "code": 0,
  "data": {
    "userID": 0,
    "userName": "string",
    "email": "string",
    "workspaceID": 0,
    "workspaceName": "string"
  },
  "msg": "string"
}
```


Success response fields

200 · application/json

Field	Type	Required	Description
`code`	`integer`	No	Response code
`data`	`object`	No	AuthWhoamiResp. See the referenced schema in the Schemas section.
`msg`	`string`	No	Response message

Response status meanings

Status	Meaning	Media type
`200`	A successful response.	`application/json`
`400`	Correct the request parameters or Content-Type before retrying.	`application/json`
`401`	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	`text/plain`
`403`	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	`text/plain`
`404`	Resource not found. Verify paths, IDs, and current resource state.	–
`409`	Resource state conflict. Verify paths, IDs, and current resource state.	–
`413`	Request too large. Reduce the request before retrying.	–
`500`	Service error. Investigate the service and confirm the outcome before retrying.	–

Flow

Manage Source Flow and Event Flow definitions.

POST `/flow/create`

Create a flow
Creates a SourceFlow or EventFlow and returns its numeric resource ID.

Request body

```
curl --request POST 'https://<tier0-host>/openapi/v1/flow/create' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"flowName":"string","flowType":"string","description":"string","template":"string"}'
```

`application/json`

Field	Type	Required	Description
flowName	string	Yes	Flow name, up to 63 Unicode characters.
flowType	string	Yes	Flow category: SourceFlow or EventFlow.
description	string	No	Flow description.
template	string	No	Optional flow template JSON.

Responses

200 400 401 403 404 409 413 500

Success response example

200 · application/json

```
{
  "code": 0,
  "data": {
    "id": 0,
    "brokerID": "string"
  },
  "msg": "string"
}
```

Success response fields

200 · application/json

Field	Type	Required	Description
code	integer	No	Response code
data	object	No	FlowCreateResp. See the referenced schema in the Schemas section.
msg	string	No	Response message

Response status meanings

Status	Meaning	Media type
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–

Status	Meaning	Media type
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /flow/delete

Delete flows

Deletes the listed flows sequentially; earlier deletions are not rolled back when a later item fails.

Request body

```
curl --request POST 'https://<tier0-host>/openapi/v1/flow/delete' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"ids":[0]}'
```

application/json

Field	Type	Required	Description
ids	array<integer (int64)>	Yes	Flow IDs to delete.

Responses

200 400 401 403 404 409 413 500

Success response example

200 · application/json

```
{
  "code": 0,
  "data": {
    "success": false
  },
  "msg": "string"
}
```

Success response fields

200 · application/json

Field	Type	Required	Description
code	integer	No	Response code

Field	Type	Required	Description
`data`	`object`	No	FlowEmptyResp. See the referenced schema in the Schemas section.
`msg`	`string`	No	Response message

Response status meanings

Status	Meaning	Media type
`200`	A successful response.	`application/json`
`400`	Correct the request parameters or Content-Type before retrying.	`application/json`
`401`	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	`text/plain`
`403`	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	`text/plain`
`404`	Resource not found. Verify paths, IDs, and current resource state.	–
`409`	Resource state conflict. Verify paths, IDs, and current resource state.	–
`413`	Request too large. Reduce the request before retrying.	–
`500`	Service error. Investigate the service and confirm the outcome before retrying.	–

POST `/flow/deploy`

Deploy a flow

Deploys a complete Node-RED canvas JSON string to a flow.

Request body

```
curl --request POST 'https://<tier0-host>/openapi/v1/flow/deploy' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"id":0,"flowsJson":"string"}'
```

`application/json`

Field	Type	Required	Description
`id`	`integer (int64)`	Yes	Flow ID to deploy.
`flowsJson`	`string`	Yes	Node-RED canvas JSON to deploy.

Responses

200 400 401 403 404 409 413 500

Success response example

200 · application/json

```
{
  "code": 0,
  "data": {
    "flowId": "string"
  },
  "msg": "string"
}
```

Success response fields

200 · application/json

Field	Type	Required	Description
code	integer	No	Response code
data	object	No	FlowDeployResp. See the referenced schema in the Schemas section.
msg	string	No	Response message

Response status meanings

Status	Meaning	Media type
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /flow/flowdata

Get flow canvas data
Returns the Node-RED canvas revision and node objects for a flow.

Request body

```
curl --request POST 'https://<tier0-host>/openapi/v1/flow/flowdata' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"id":0}'
```

application/json

Field	Type	Required	Description
id	integer (int64)	Yes	Flow ID.

Responses

200 400 401 403 404 409 413 500

Success response example

200 · application/json

```
{
  "code": 0,
  "data": {
    "rev": "string",
    "flows": [
      "value"
    ]
  },
  "msg": "string"
}
```

Success response fields

200 · application/json

Field	Type	Required	Description
code	integer	No	Response code
data	object	No	FlowDataResp. See the referenced schema in the Schemas section.
msg	string	No	Response message

Response status meanings

Status	Meaning	Media type
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /flow/get

Get a flow

Returns flow metadata for a numeric flow ID.

Request body

```
curl --request POST 'https://<tier0-host>/openapi/v1/flow/get' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"id":0}'
```

application/json

Field	Type	Required	Description
id	integer (int64)	Yes	Flow ID.

Responses

200 400 401 403 404 409 413 500

Success response example

200 · application/json

```
{
  "code": 0,
  "data": {
    "id": 0,
    "flowId": "string",
    "flowName": "string",
    "flowType": "string",
    "flowStatus": "string"
  },
  "msg": "string"
}
```


Success response fields

`200 · application/json`

Field	Type	Required	Description
`code`	`integer`	No	Response code
`data`	`object`	No	FlowInfo. See the referenced schema in the Schemas section.
`msg`	`string`	No	Response message

Response status meanings

Status	Meaning	Media type
`200`	A successful response.	`application/json`
`400`	Correct the request parameters or Content-Type before retrying.	`application/json`
`401`	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	`text/plain`
`403`	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	`text/plain`
`404`	Resource not found. Verify paths, IDs, and current resource state.	–
`409`	Resource state conflict. Verify paths, IDs, and current resource state.	–
`413`	Request too large. Reduce the request before retrying.	–
`500`	Service error. Investigate the service and confirm the outcome before retrying.	–

POST `/flow/list`

List flows

Lists flows, optionally filtered by a keyword or flow category.

Request body

```
curl --request POST 'https://<tier0-host>/openapi/v1/flow/list' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"keyword":"string","flowType":"string"}'
```

application/json

Field	Type	Required	Description
keyword	string	No	Keyword matched against flow names and descriptions.
flowType	string	No	Flow category filter: SourceFlow or EventFlow.

Responses

200 400 401 403 404 409 413 500

Success response example

200 · application/json


```
{
  "code": 200,
  "msg": "success",
  "data": {
    "list": [
      {
        "id": 101,
        "flowId": "flow-101",
        "flowName": "Line 1 collector",
        "flowType": "source",
        "flowStatus": "draft",
        "template": "[]",
        "description": "Collects line telemetry",
        "isFavorite": 2,
        "currentVersionId": 0,
        "currentVersionName": "",
        "currentVersionType": "",
        "createdTime": 1788134400000,
        "updatedTime": 1788134400000
      }
    ]
  }
}
```


Success response fields

200 · application/json

Field	Type	Required	Description
code	integer	No	Response code

Field	Type	Required	Description
`data`	`object`	No	FlowListResp. See the referenced schema in the Schemas section.
`msg`	`string`	No	Response message

Response status meanings

Status	Meaning	Media type
`200`	A successful response.	`application/json`
`400`	Correct the request parameters or Content-Type before retrying.	`application/json`
`401`	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	`text/plain`
`403`	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	`text/plain`
`404`	Resource not found. Verify paths, IDs, and current resource state.	–
`409`	Resource state conflict. Verify paths, IDs, and current resource state.	–
`413`	Request too large. Reduce the request before retrying.	–
`500`	Service error. Investigate the service and confirm the outcome before retrying.	–

POST `/flow/nodes`

List available flow node types

Returns the Node-RED node sets available for a flow category.

Request body

```
curl --request POST 'https://<tier0-host>/openapi/v1/flow/nodes' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"flowType":"string"}'
```

`application/json`

Field	Type	Required	Description
`flowType`	`string`	Yes	Flow category whose Node-RED node catalog should be returned.

Responses

`200` `400` `401` `403` `404` `409` `413` `500`

Success response example

`200 · application/json`

```
{
  "code": 0,
  "data": {
    "nodes": [
      "value"
    ]
  },
  "msg": "string"
}
```

Success response fields

`200 · application/json`

Field	Type	Required	Description
`code`	`integer`	No	Response code
`data`	`object`	No	FlowNodesResp. See the referenced schema in the Schemas section.
`msg`	`string`	No	Response message

Response status meanings

Status	Meaning	Media type
`200`	A successful response.	`application/json`
`400`	Correct the request parameters or Content-Type before retrying.	`application/json`
`401`	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	`text/plain`
`403`	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	`text/plain`
`404`	Resource not found. Verify paths, IDs, and current resource state.	–
`409`	Resource state conflict. Verify paths, IDs, and current resource state.	–
`413`	Request too large. Reduce the request before retrying.	–
`500`	Service error. Investigate the service and confirm the outcome before retrying.	–

POST `/flow/update`

Update a flow

Updates flow metadata and settings for a numeric flow ID.

Request body

```
curl --request POST 'https://<tier0-host>/openapi/v1/flow/update' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"id":0,"flowName":"string","description":"string","template":"string","isFavorite":0}'
```

`application/json`

Field	Type	Required	Description
`id`	`integer (int64)`	Yes	Flow ID to update.
`flowName`	`string`	No	Updated flow name.
`description`	`string`	No	Updated flow description.
`template`	`string`	No	Updated flow template JSON.
`isFavorite`	`integer (int64)`	No	Favorite state: 1 for favorite, 2 for not favorite.

Responses

`200` `400` `401` `403` `404` `409` `413` `500`

Success response example

`200 · application/json`

```
{
  "code": 0,
  "data": {
    "success": false
  },
  "msg": "string"
}
```

Success response fields

`200 · application/json`

Field	Type	Required	Description
`code`	`integer`	No	Response code
`data`	`object`	No	FlowEmptyResp. See the referenced schema in the Schemas section.
`msg`	`string`	No	Response message

Response status meanings

Status	Meaning	Media type
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

Platform and Projects

Read platform information and manage projects.

`POST` **`/info`**

Get service information

Returns the service name, version, reported capabilities, and configured MQTT broker.

Request body

```
curl --request POST 'https://<tier0-host>/openapi/v1/info' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{}'
```

application/json

This request body has no declared object fields in OpenAPI.

Responses

200 400 401 403 404 409 413 500

Response status meanings

Status	Meaning	Media type
200	A successful response.	–
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /launchpad/{projectName}/getMembers

List project members

Lists Launchpad project members. URL-encode projectName when it contains special characters.

Request parameters

Name	Location	Type	Required	Description
projectName	path	string	Yes	Project name in the Launchpad URL. URL-encode special characters.

Request body

```
curl --request POST 'https://<tier0-host>/openapi/v1/launchpad/{projectName}/getMembers' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"roleKey":"string","roles":
["string"],"updatedAtStart":"string","updatedAtEnd":"string","page":0}'
```

application/json

Field	Type	Required	Description
roleKey	string	No	Role key filter.
roles	array<string>	No	Role filters or the returned role list, depending on the model.
updatedAtStart	string	No	Inclusive update-time lower bound in RFC 3339 format.

Field	Type	Required	Description
updatedAtEnd	string	No	Inclusive update-time upper bound in RFC 3339 format.
page	integer (int64)	No	Page number, starting at 1.
size	integer (int64)	No	Number of members per page.

Responses

200 400 401 403 404 409 413 500

Success response example

200 · application/json

```
{
  "code": 0,
  "data": {
    "list": [
      "value"
    ],
    "total": 0,
    "page": 0,
    "size": 0
  },
  "msg": "string"
}
```

Success response fields

200 · application/json

Field	Type	Required	Description
code	integer	No	Response code
data	object	No	LaunchpadMembersResp. See the referenced schema in the Schemas section.
msg	string	No	Response message

Response status meanings

Status	Meaning	Media type
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain

Status	Meaning	Media type
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /platform/getMembers

List platform members

Lists platform members with optional keyword, role, status, and update-time filters.

Request body

```
curl --request POST 'https://<tier0-host>/openapi/v1/platform/getMembers' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"keyword":"string","roleKey":"string","roles":["string"],"statuses":
["string"],"updatedAtStart":"string"}'
```

application/json

Field	Type	Required	Description
keyword	string	No	Search keyword.
roleKey	string	No	Role key filter.
roles	array<string>	No	Role filters or the returned role list, depending on the model.
statuses	array<string>	No	Member status filters: active or disabled.
updatedAtStart	string	No	Inclusive update-time lower bound in RFC 3339 format.
updatedAtEnd	string	No	Inclusive update-time upper bound in RFC 3339 format.
page	integer (int64)	No	Page number, starting at 1.
size	integer (int64)	No	Number of members per page.

Responses

200 400 401 403 404 409 413 500

Success response example

200 · application/json

```
{
  "code": 0,
  "data": {
    "list": [
      "value"
    ],
    "total": 0,
    "page": 0,
    "size": 0
  },
  "msg": "string"
}
```


Success response fields

`200 · application/json`

Field	Type	Required	Description
`code`	`integer`	No	Response code
`data`	`object`	No	PlatformMembersResp. See the referenced schema in the Schemas section.
`msg`	`string`	No	Response message

Response status meanings

Status	Meaning	Media type
`200`	A successful response.	`application/json`
`400`	Correct the request parameters or Content-Type before retrying.	`application/json`
`401`	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	`text/plain`
`403`	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	`text/plain`
`404`	Resource not found. Verify paths, IDs, and current resource state.	–
`409`	Resource state conflict. Verify paths, IDs, and current resource state.	–
`413`	Request too large. Reduce the request before retrying.	–
`500`	Service error. Investigate the service and confirm the outcome before retrying.	–

MQTT Authentication

Create and remove MQTT credentials.

POST /mqtt-auth/create

Create an MQTT credential
Creates a normal MQTT credential. The returned password is available only once and must be stored securely immediately.

Request body

```
curl --request POST 'https://<tier0-host>/openapi/v1/mqtt-auth/create' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"name":"string","description":"string","clientIDRandomSuffixEnabled":false}'
```

`application/json`

Field	Type	Required	Description
`name`	`string`	Yes	MQTT credential name.
`description`	`string`	No	Description of the MQTT credential.
`clientIDRandomSuffixEnabled`	`boolean`	No	Whether to append a random suffix to the MQTT client ID.

Responses

`200` `400` `401` `403` `404` `409` `413` `500`

Success response example

`200 · application/json`

```
{
  "code": 0,
  "data": {
    "id": 0,
    "name": "string",
    "clientID": "string",
    "username": "string",
    "password": "string"
  },
  "msg": "string"
}
```

Success response fields

`200 · application/json`

Field	Type	Required	Description
`code`	`integer`	No	Response code

Field	Type	Required	Description
`data`	`object`	No	MqttAuthCreateResp. See the referenced schema in the Schemas section.
`msg`	`string`	No	Response message

Response status meanings

Status	Meaning	Media type
`200`	A successful response.	`application/json`
`400`	Correct the request parameters or Content-Type before retrying.	`application/json`
`401`	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	`text/plain`
`403`	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	`text/plain`
`404`	Resource not found. Verify paths, IDs, and current resource state.	–
`409`	Resource state conflict. Verify paths, IDs, and current resource state.	–
`413`	Request too large. Reduce the request before retrying.	–
`500`	Service error. Investigate the service and confirm the outcome before retrying.	–

POST `/mqtt-auth/delete`

Delete an MQTT credential

Idempotently deletes a normal MQTT credential. System credentials are not deleted.

Request body

```
curl --request POST 'https://<tier0-host>/openapi/v1/mqtt-auth/delete' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"id":0}'
```

`application/json`

Field	Type	Required	Description
`id`	`integer (int64)`	Yes	Resource ID.

Responses

`200` `400` `401` `403` `404` `409` `413` `500`

Success response example

`200 · application/json`

```
{
  "code": 0,
  "data": {
    "success": false
  },
  "msg": "string"
}
```

Success response fields

`200 · application/json`

Field	Type	Required	Description
`code`	`integer`	No	Response code
`data`	`object`	No	MqttAuthDeleteResp. See the referenced schema in the Schemas section.
`msg`	`string`	No	Response message

Response status meanings

Status	Meaning	Media type
`200`	A successful response.	`application/json`
`400`	Correct the request parameters or Content-Type before retrying.	`application/json`
`401`	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	`text/plain`
`403`	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	`text/plain`
`404`	Resource not found. Verify paths, IDs, and current resource state.	–
`409`	Resource state conflict. Verify paths, IDs, and current resource state.	–
`413`	Request too large. Reduce the request before retrying.	–
`500`	Service error. Investigate the service and confirm the outcome before retrying.	–

Schemas

API field names, examples, and source descriptions follow the bundled English OpenAPI specification.

AssetFileDeleteReq

Field	Type	Required	Description
`filePath`	`string`	Yes	Server-issued storage path returned by an upload API.

AssetFileDeleteResp

Field	Type	Required	Description
`deleted`	`boolean`	Yes	Whether the file was deleted.

AssetFileMultipartAbortReq

Field	Type	Required	Description
`fileKey`	`string`	Yes	Object-store key used for the multipart upload.
`uploadId`	`string`	Yes	Object-store multipart upload session ID.

AssetFileMultipartAbortResp

Field	Type	Required	Description
`aborted`	`integer (int32)`	Yes	Abort result: 1 = aborted (including idempotent replay success), 2 = not aborted.

AssetFileMultipartCompleteReq

Field	Type	Required	Description
`fileKey`	`string`	Yes	Object-store key used for the multipart upload.
`uploadId`	`string`	Yes	Object-store multipart upload session ID.
`parts`	`array<object>`	Yes	Uploaded part numbers and their ETags.

AssetFileMultipartCompleteResp

Field	Type	Required	Description
`filePath`	`string`	Yes	Server-issued storage path returned by an upload API.
`fileUrl`	`string`	No	File access URL.
`sizeBytes`	`integer (int64)`	Yes	File size in bytes.
`expiresAt`	`integer (int64)`	No	Expiration time, in Unix milliseconds.

AssetFileMultipartInitReq

Field	Type	Required	Description
`fileName`	`string`	Yes	Original file name.
`contentType`	`string`	No	File MIME type, for example application/octet-stream.
`size`	`integer (int64)`	Yes	Total file size in bytes, greater than 0 and at most 1 GiB.
`partSize`	`integer (int64)`	No	Preferred part size in bytes. The server may normalize it.
`business`	`string`	No	Business domain or feature associated with the file.
`useBy`	`string`	No	Optional usage category: user, workspace, or platform.
`visibility`	`string`	No	File visibility: private or public.
`appInstanceId`	`string`	No	Associated application instance ID.
`sessionId`	`string`	No	Caller session ID.
`bizId`	`string`	No	Associated business identifier.

AssetFileMultipartInitResp

Field	Type	Required	Description
`fileKey`	`string`	Yes	Object-store key used for the multipart upload.
`filePath`	`string`	Yes	Server-issued storage path returned by an upload API.
`uploadId`	`string`	Yes	Object-store multipart upload session ID.
`partSize`	`integer (int64)`	Yes	Part size in bytes; the server may normalize it.
`partCount`	`integer (int64)`	Yes	Total number of upload parts.
`expiresAt`	`integer (int64)`	Yes	Expiration time, in Unix milliseconds.

AssetFileMultipartPart

Field	Type	Required	Description
`partNumber`	`integer (int32)`	Yes	Part number, starting at 1.
`etag`	`string`	Yes	ETag returned after uploading this part.

AssetFileMultipartPartUrl

Field	Type	Required	Description
`partNumber`	`integer (int32)`	Yes	Part number, starting at 1.
`url`	`string`	Yes	Presigned PUT URL for this part.
`expiresAt`	`integer (int64)`	Yes	Expiration time, in Unix milliseconds.

AssetFileMultipartPartUrlsReq

Field	Type	Required	Description
`fileKey`	`string`	Yes	Object-store key used for the multipart upload.
`uploadId`	`string`	Yes	Object-store multipart upload session ID.
`partNumbers`	`array<integer (int32)>`	Yes	One-based part numbers to sign.

AssetFileMultipartPartUrlsResp

Field	Type	Required	Description
`partUrls`	`array<object>`	Yes	Presigned URLs for the requested parts.

AssetFileUploadReq

Field	Type	Required	Description
`fileName`	`string`	Yes	Original file name.
`contentType`	`string`	No	MIME content type.
`size`	`integer (int64)`	Yes	File size in bytes. Business validation requires a value greater than 0 and at most 1 GiB.
`business`	`string`	No	Business domain or feature associated with the file.
`useBy`	`string`	No	Optional usage category: user, workspace, or platform.
`visibility`	`string`	No	File visibility, such as private or public.
`appInstanceId`	`string`	No	Associated application instance ID.
`sessionId`	`string`	No	Caller session ID.

AssetFileUploadResp

Field	Type	Required	Description
`fileId`	`integer (int64)`	No	File ID.
`filePath`	`string`	Yes	Server-issued storage path returned by an upload API.
`fileUrl`	`string`	No	File access URL.
`uploadUrl`	`string`	Yes	Upload URL; the legacy Enterprise endpoint currently returns an empty string.
`expiresAt`	`integer (int64)`	No	Expiration time, in Unix milliseconds.

AssetFileUrlResp

Field	Type	Required	Description
`fileUrl`	`string`	Yes	File access URL.
`expiresAt`	`integer (int64)`	No	Expiration time, in Unix milliseconds.

AssetUploadReq

AssetUploadReq

Field	Type	Required	Description
`fileName`	`string`	Yes	Original file name.
`sizeBytes`	`integer (int64)`	Yes	File size in bytes.
`contentType`	`string`	No	File MIME type, for example application/octet-stream.
`size`	`integer (int64)`	No	File size in bytes.
`sha256`	`string`	No	Optional SHA-256 checksum of the file content.
`storageKey`	`string`	No	Optional object-store key. Omit it to let the server generate one.
`isPublic`	`boolean (boolean)`	No	Whether the file should be accessible through the public download endpoint.
`business`	`string`	No	Business domain or feature that owns the uploaded file.
`useBy`	`string`	No	Usage scenario or owner category for the uploaded file.

AssetUploadResp

AssetUploadResp

Field	Type	Required	Description
`uploadUrl`	`string`	Yes	Upload URL; the legacy Enterprise endpoint currently returns an empty string.
`filePath`	`string`	Yes	Server-issued storage path returned by an upload API.
`fileUrl`	`string`	No	File access URL.

AuthWhoamiResp

Field	Type	Required	Description
`userID`	`integer (int64)`	Yes	Numeric ID of the API key owner.
`userName`	`string`	Yes	User name.
`email`	`string`	Yes	Email address.
`workspaceID`	`integer (int64)`	Yes	Platform workspace ID.
`workspaceName`	`string`	Yes	Workspace name; may be empty in Enterprise.
`apiKeyName`	`string`	Yes	Name of the current API key.
`keyPrefix`	`string`	Yes	API key prefix.
`permissions`	`array<string>`	Yes	Effective permissions of the current identity.
`roles`	`array<string>`	Yes	Role filters or the returned role list, depending on the model.
`keyType`	`string`	Yes	API key ownership type, for example personal.

BrowseReq

BrowseReq

Field	Type	Required	Description
`path`	`string`	No	Root UNS path. Empty means the workspace root.
`max_depth`	`integer (int64)`	No	Maximum tree depth. Omit or use 0 for the server default.
`include_metadata`	`boolean (boolean)`	No	Whether returned nodes should include metadata.
`include_leaf_value`	`boolean (boolean)`	No	Whether returned leaf nodes should include their latest payload.

BrowseResp

BrowseResp

Field	Type	Required	Description
`tree`	`array<object>`	Yes	UNS node tree.

CreateResp

CreateResp

Field	Type	Required	Description
`success`	`boolean (boolean)`	Yes	Whether the operation succeeded.
`results`	`array<object>`	Yes	Per-item results. Check success and error for each item.

FlowCreateReq

Field	Type	Required	Description
`flowName`	`string`	Yes	Flow name, up to 63 Unicode characters.
`flowType`	`string`	Yes	Flow category: SourceFlow or EventFlow.
`description`	`string`	No	Flow description.
`template`	`string`	No	Optional flow template JSON.

FlowCreateResp

Field	Type	Required	Description
`id`	`integer (int64)`	Yes	Resource ID.
`brokerID`	`string`	Yes	ID of the MQTT broker configuration node injected when the flow is created.

FlowDataResp

Field	Type	Required	Description
`rev`	`string`	Yes	Node-RED canvas revision; may be an empty string.
`flows`	`array<object>`	Yes	Array of Node-RED canvas node objects.

FlowDeleteReq

Field	Type	Required	Description
ids	array<integer (int64)>	Yes	Flow IDs to delete.

FlowDeployReq

Field	Type	Required	Description
id	integer (int64)	Yes	Flow ID to deploy.
flowsJson	string	Yes	Node-RED canvas JSON to deploy.

FlowDeployResp

Field	Type	Required	Description
flowId	string	Yes	Flow identifier.

FlowEmptyResp

Field	Type	Required	Description
success	boolean	Yes	Whether the operation succeeded.

FlowGetReq

Field	Type	Required	Description
id	integer (int64)	Yes	Flow ID.

FlowInfo

Field	Type	Required	Description
id	integer (int64)	Yes	Resource ID.
flowId	string	Yes	Flow identifier.
flowName	string	Yes	Flow name.
flowType	string	Yes	Response flow type: source or event.
flowStatus	string	Yes	Current flow status, for example draft.
template	string	Yes	Flow template JSON, encoded as a string.
description	string	Yes	Description.

Field	Type	Required	Description
isFavorite	integer (int64)	Yes	Favorite state: 1 for favorite, 2 for not favorite.
currentVersionId	integer (int64)	Yes	Current version ID; 0 if unset.
currentVersionName	string	Yes	Current version name; empty if unset.
currentVersionType	string	Yes	Current version type; empty if unset.
createdTime	integer (int64)	Yes	Creation time, in Unix milliseconds.
updatedTime	integer (int64)	Yes	Last update time, in Unix milliseconds.

FlowListReq

Field	Type	Required	Description
keyword	string	No	Keyword matched against flow names and descriptions.
flowType	string	No	Flow category filter: SourceFlow or EventFlow.

FlowListResp

Field	Type	Required	Description
list	array<object>	Yes	Result list.

FlowNodesReq

Field	Type	Required	Description
flowType	string	Yes	Flow category whose Node-RED node catalog should be returned.

FlowNodesResp

Field	Type	Required	Description
nodes	array<object>	Yes	Available Node-RED node sets.

FlowUpdateReq

Field	Type	Required	Description
id	integer (int64)	Yes	Flow ID to update.
flowName	string	No	Updated flow name.
description	string	No	Updated flow description.

Field	Type	Required	Description
`template`	`string`	No	Updated flow template JSON.
`isFavorite`	`integer (int64)`	No	Favorite state: 1 for favorite, 2 for not favorite.

HistoryAggregation

HistoryAggregation

Field	Type	Required	Description
`interval`	`string`	Yes	Positive aggregation window, such as 1m, 5m, 1h, or 1d.
`function`	`string`	No	Aggregation function: avg, min, max, sum, count, first, or last.
`field`	`string`	No	First-level payload field to aggregate.
`fields`	`array<object>`	No	Per-field aggregation settings.

HistoryAggregationField

HistoryAggregationField

Field	Type	Required	Description
`name`	`string`	Yes	Payload field to aggregate.
`function`	`string`	No	Function for this field; defaults to the outer aggregation function.

HistoryQueryMeta

HistoryQueryMeta

Field	Type	Required	Description
`aggregated`	`boolean (boolean)`	Yes	Whether the result was aggregated.
`sampled`	`boolean (boolean)`	Yes	Whether the result was sampled.
`rawTotal`	`integer (int64)`	Yes	Total number of raw records.
`returnedTotal`	`integer (int64)`	Yes	Number of records returned in this response.
`requestedIntervalMs`	`integer (int64)`	No	Requested aggregation interval in milliseconds.
`actualIntervalMs`	`integer (int64)`	No	Actual aggregation interval in milliseconds.
`strategy`	`string`	No	Processing strategy used for the query.

HistoryReq

HistoryReq

Field	Type	Required	Description
topics	array<string>	Yes	UNS topic paths whose history should be queried.
start_time	string	Yes	Inclusive query start in RFC 3339 format.
end_time	string	Yes	Inclusive query end in RFC 3339 format.
aggregation	object	No	HistoryAggregation. See the referenced schema in the Schemas section.
page	integer (int64)	No	Page number, starting from 1.
size	integer (int64)	No	Page size; values above the server maximum are capped.

HistoryResp

HistoryResp

Field	Type	Required	Description
success	boolean (boolean)	Yes	Whether the operation succeeded.
results	array<object>	Yes	Per-item results. Check success and error for each item.
page	integer (int64)	Yes	Page number, starting at 1.
size	integer (int64)	Yes	File size in bytes.
total	integer (int64)	Yes	Total number of matching records.

InfoReq

```
{
  "type": "object",
  "properties": {}
}
```

LaunchpadGetMembersReq

Field	Type	Required	Description
roleKey	string	No	Role key filter.
roles	array<string>	No	Role filters or the returned role list, depending on the model.

Field	Type	Required	Description
updatedAtStart	string	No	Inclusive update-time lower bound in RFC 3339 format.
updatedAtEnd	string	No	Inclusive update-time upper bound in RFC 3339 format.
page	integer (int64)	No	Page number, starting at 1.
size	integer (int64)	No	Number of members per page.

LaunchpadMemberResp

Field	Type	Required	Description
memberId	string	Yes	Member identifier.
userId	string	Yes	User ID.
userName	string	No	User name.
email	string	No	Email address.
roles	array<object>	Yes	Role filters or the returned role list, depending on the model.
updatedAt	string	No	Last update time.

LaunchpadMemberRoleAppResp

Field	Type	Required	Description
appId	string	Yes	Application ID, represented as a string.
appName	string	Yes	Application name.

LaunchpadMemberRoleResp

Field	Type	Required	Description
roleId	string	Yes	Role ID, represented as a string.
roleKey	string	Yes	Role key filter.
roleName	string	No	Role display name.
description	string	No	Description.
memberCount	integer (int64)	Yes	Number of members assigned to the role.
apps	array<object>	No	Applications associated with the role.

LaunchpadMembersResp

Field	Type	Required	Description
list	array<object>	Yes	Result list.
total	integer (int64)	Yes	Total number of matching records.
page	integer (int64)	Yes	Page number, starting at 1.
size	integer (int64)	Yes	File size in bytes.

MqttAuthCreateReq

Field	Type	Required	Description
name	string	Yes	MQTT credential name.
description	string	No	Description of the MQTT credential.
clientIDRandomSuffixEnabled	boolean	No	Whether to append a random suffix to the MQTT client ID.

MqttAuthCreateResp

Field	Type	Required	Description
id	integer (int64)	Yes	Resource ID.
name	string	Yes	Name.
clientID	string	Yes	MQTT client ID.
username	string	Yes	MQTT user name.
password	string	Yes	Plaintext password. Save it now; it cannot be retrieved again.
clientIDRandomSuffixEnabled	boolean	Yes	Whether to append a random suffix to the MQTT client ID.

MqttAuthDeleteReq

Field	Type	Required	Description
id	integer (int64)	Yes	Resource ID.

MqttAuthDeleteResp

Field	Type	Required	Description
success	boolean	Yes	Whether the operation succeeded.

NamespaceNode

NamespaceNode

Field	Type	Required	Description
name	string	Yes	Node name, up to 63 Unicode characters. In a nested request, this is the name under its parent.
type	string	No	Node type: PATH or TOPIC.
children	array<object>	No	Child nodes to create under this node.
topicType	string	No	Topic data category: STATE, ACTION, or METRIC.
fields	array<object>	No	Payload schema fields for TOPIC nodes.
description	string	No	Node description, up to 512 Unicode characters.
displayName	string	No	Display name shown in clients.
extendProperties	object	No	Custom metadata key-value pairs.
alias	string	No	Optional topic alias.
enableHistory	boolean (boolean)	No	Whether history storage is enabled for this topic.

NodeCreateReq

NodeCreateReq

Field	Type	Required	Description
namespace	array<object>	Yes	UNS nodes to create. Nest children to create a subtree.

NodeDeleteReq

NodeDeleteReq

Field	Type	Required	Description
topics	array<string>	Yes	UNS topic or path node paths to delete.

Field	Type	Required	Description
hard_delete	boolean (boolean)	No	Permanently delete instead of moving nodes to the recycle state.

NodeRedNodeSet

Field	Type	Required	Description
id	string	Yes	Resource ID.
name	string	Yes	Name.
types	array<string>	Yes	Node-RED node types provided by the node set.
enabled	boolean	Yes	Whether the node set is enabled.
module	string	Yes	Node-RED module name.
version	string	Yes	Service or node module version.

NodeRestoreReq

NodeRestoreReq

Field	Type	Required	Description
path	string	Yes	Full UNS path of the node to restore. Use its path before deletion, not a node ID or HTTP URL.

NodeUpdateReq

NodeUpdateReq

Field	Type	Required	Description
path	string	Yes	Existing UNS topic or path node to update.
name	string	No	New node name.
description	string	No	Updated node description.
displayName	string	No	Updated display name.
extendProperties	object	No	Updated custom metadata key-value pairs.
fields	array<object>	No	Updated payload schema fields.
alias	string	No	Updated topic alias.
enableHistory	boolean (boolean)	No	Supply true or false to change history storage. Omit to keep the existing setting.

Field	Type	Required	Description
updateMask	array<string>	No	Compatibility field ignored by the current Enterprise implementation. It cannot be used to clear fields.

OpenapiCreateResult

OpenapiCreateResult

Field	Type	Required	Description
success	boolean (boolean)	Yes	Whether the operation succeeded.
topic	string	Yes	Full UNS topic path.
error	object	No	OpenapiItemError. See the referenced schema in the Schemas section.

OpenapiEmptyResp

OpenapiEmptyResp

Field	Type	Required	Description
success	boolean (boolean)	Yes	Whether the operation succeeded.

OpenapiHistoryItem

OpenapiHistoryItem

Field	Type	Required	Description
success	boolean (boolean)	Yes	Whether the operation succeeded.
topic	string	Yes	Full UNS topic path.
result	object	No	OpenapiHistoryResult. See the referenced schema in the Schemas section.
error	object	No	OpenapiItemError. See the referenced schema in the Schemas section.

OpenapiHistoryResult

OpenapiHistoryResult

Field	Type	Required	Description
values	array<object>	Yes	Historical VQT records.
meta	object	No	HistoryQueryMeta. See the referenced schema in the Schemas section.

OpenapiHistoryValue

OpenapiHistoryValue

Field	Type	Required	Description
value	object	No	Payload value.
quality	string	Yes	Data quality: Good, GoodNoData, Uncertain, or Bad.
timeStamp	integer (int64)	Yes	Unix timestamp in milliseconds. Writes use server time when omitted.

OpenapiItemError

OpenapiItemError

Field	Type	Required	Description
code	integer (int64)	Yes	Status code; in an item error, identifies the failure.
message	string	Yes	Item-level error message.

OpenapiNodeInfo

OpenapiNodeInfo

Field	Type	Required	Description
id	integer (int64)	Yes	Resource ID.
path	string	Yes	Full UNS node path.
name	string	Yes	Name.
type	string	Yes	Node type: PATH or TOPIC.
topicType	string	Yes	Topic category: STATE, ACTION, or METRIC.
description	string	No	Description.
displayName	string	No	Display name.
extendProperties	object	No	Custom node metadata as key-value pairs.
fields	array<object>	No	Payload field definitions.

Field	Type	Required	Description
children	array<object>	No	Child nodes under this node.
alias	string	No	Node alias.
enableHistory	boolean (boolean)	No	Whether history storage is enabled.
bindFlowID	integer (int64)	No	Bound Source Flow ID. 0 means no Source Flow is bound.
bindEventFlowID	integer (int64)	No	Bound Event Flow ID. 0 means no Event Flow is bound.
payload	object	No	OpenapiVQT. See the referenced schema in the Schemas section.

OpenapiReadItem

OpenapiReadItem

Field	Type	Required	Description
success	boolean (boolean)	Yes	Whether the operation succeeded.
topic	string	Yes	Full UNS topic path.
result	object	No	OpenapiVQT. See the referenced schema in the Schemas section.
error	object	No	OpenapiItemError. See the referenced schema in the Schemas section.
metadata	object	No	OpenapiNodeInfo. See the referenced schema in the Schemas section.

OpenapiVQT

OpenapiVQT

Field	Type	Required	Description
value	object	No	Payload value.
quality	string	Yes	Data quality: Good, GoodNoData, Uncertain, or Bad.
timeStamp	integer (int64)	Yes	Unix timestamp in milliseconds. Writes use server time when omitted.

OpenapiWriteResult

OpenapiWriteResult

Field	Type	Required	Description
success	boolean (boolean)	Yes	Whether the operation succeeded.
topic	string	Yes	Full UNS topic path.
error	object	No	OpenapiItemError. See the referenced schema in the Schemas section.

PlatformGetMembersReq

Field	Type	Required	Description
keyword	string	No	Search keyword.
roleKey	string	No	Role key filter.
roles	array<string>	No	Role filters or the returned role list, depending on the model.
statuses	array<string>	No	Member status filters: active or disabled.
updatedAtStart	string	No	Inclusive update-time lower bound in RFC 3339 format.
updatedAtEnd	string	No	Inclusive update-time upper bound in RFC 3339 format.
page	integer (int64)	No	Page number, starting at 1.
size	integer (int64)	No	Number of members per page.

PlatformMemberResp

Field	Type	Required	Description
userId	string	Yes	User ID.
userName	string	No	User name.
nickName	string	No	User nickname.
email	string	No	Email address.
status	string	Yes	Platform member status: active or disabled.
roles	array<object>	Yes	Role filters or the returned role list, depending on the model.
updatedAt	string	Yes	Last update time.

PlatformMemberRoleResp

Field	Type	Required	Description
`roleId`	`string`	Yes	Role ID, represented as a string.
`roleKey`	`string`	Yes	Role key filter.
`roleName`	`string`	No	Role display name.
`description`	`string`	No	Description.

PlatformMembersResp

Field	Type	Required	Description
`list`	`array<object>`	Yes	Result list.
`total`	`integer (int64)`	Yes	Total number of matching records.
`page`	`integer (int64)`	Yes	Page number, starting at 1.
`size`	`integer (int64)`	Yes	File size in bytes.

ReadReq

ReadReq

Field	Type	Required	Description
`topics`	`array<string>`	Yes	UNS topic paths to read.
`include_metadata`	`boolean (boolean)`	No	Whether each result should include UNS node metadata.
`include_leaf_value`	`boolean (boolean)`	No	Whether leaf nodes should include their latest payload.

ReadResp

ReadResp

Field	Type	Required	Description
`success`	`boolean (boolean)`	Yes	Whether the operation succeeded.
`results`	`array<object>`	Yes	Per-item results. Check success and error for each item.

SchemaField

SchemaField

Field	Type	Required	Description
`name`	`string`	Yes	Payload field name.
`type`	`string`	Yes	Payload field type: INTEGER, LONG, DOUBLE, FLOAT, STRING, BOOLEAN, or DATETIME.
`unit`	`string`	No	Optional engineering unit.

SearchReq

SearchReq

Field	Type	Required	Description
`keyword`	`string`	No	Keyword matched against UNS node names, aliases, and paths.
`topicType`	`string`	No	Topic data category filter: STATE, ACTION, or METRIC.
`path_prefix`	`string`	No	Only return nodes under this UNS path prefix.
`include_metadata`	`boolean (boolean)`	No	Whether returned nodes should include metadata.
`include_leaf_value`	`boolean (boolean)`	No	Whether returned leaf nodes should include their latest payload.
`page`	`integer (int64)`	No	Page number, starting from 1.
`size`	`integer (int64)`	No	Page size.

SearchResp

SearchResp

Field	Type	Required	Description
`objects`	`array<object>`	Yes	Matching UNS nodes.
`page`	`integer (int64)`	Yes	Page number, starting at 1.
`size`	`integer (int64)`	Yes	File size in bytes.
`total`	`integer (int64)`	Yes	Total number of matching records.

UnsAttachmentInfo

UnsAttachmentInfo

Field	Type	Required	Description
unsId	integer (int64)	Yes	Numeric UNS node ID.
fileName	string	No	Original file name.
filePath	string	Yes	Server-issued storage path returned by an upload API.
fileUrl	string	No	File access URL.
contentType	string	No	File MIME type, for example application/octet-stream.
size	integer (int64)	No	File size in bytes.
sha256	string	No	SHA-256 checksum of the file contents.
createdAt	integer (int64)	No	Creation time, in Unix milliseconds.
fileUrlExpiresAt	integer (int64)	No	File URL expiration time, in Unix milliseconds.

UnsAttachmentListReq

UnsAttachmentListReq

Field	Type	Required	Description
unsId	integer (int64)	No	Numeric UNS node ID.
topic	string	No	Full UNS topic path.
page	integer (int64)	No	Page number, starting at 1.
size	integer (int64)	No	Number of attachments per page.
includeFileUrl	boolean (boolean)	No	Whether to return temporary file URLs. Defaults to true.

UnsAttachmentListResp

UnsAttachmentListResp

Field	Type	Required	Description
unsId	integer (int64)	Yes	Numeric UNS node ID.
list	array<object>	Yes	Result list.
total	integer (int64)	Yes	Total number of matching records.
page	integer (int64)	Yes	Page number, starting at 1.
size	integer (int64)	Yes	File size in bytes.

UnsAttachmentUploadReq

UnsAttachmentUploadReq

Field	Type	Required	Description
`unsId`	`integer (int64)`	No	Numeric owner node ID. Supply unsId or topic; a valid unsId takes precedence.
`topic`	`string`	No	Full owner node path, used when a valid unsId is not supplied.
`fileName`	`string`	No	Original file name.
`sha256`	`string`	No	SHA-256 checksum of the file contents.
`file`	`string (binary)`	Yes	Binary file contents. The multipart field name must be file.

UnsAttachmentUploadResp

UnsAttachmentUploadResp

Field	Type	Required	Description
`unsId`	`integer (int64)`	Yes	Numeric UNS node ID.
`fileName`	`string`	No	Original file name.
`filePath`	`string`	Yes	Server-issued storage path returned by an upload API.
`fileUrl`	`string`	No	File access URL.
`contentType`	`string`	No	File MIME type, for example application/octet-stream.
`size`	`integer (int64)`	No	File size in bytes.
`sha256`	`string`	No	SHA-256 checksum of the file contents.
`createdAt`	`integer (int64)`	No	Creation time, in Unix milliseconds.
`fileUrlExpiresAt`	`integer (int64)`	No	File URL expiration time, in Unix milliseconds.

UnsBindFlowReq

UnsBindFlowReq

Field	Type	Required	Description
`unsId`	`integer (int64)`	Yes	UNS node ID to bind.
`flowId`	`integer (int64)`	Yes	Flow ID to bind.

UnsBindFlowResp

UnsBindFlowResp

Field	Type	Required	Description
`success`	`boolean (boolean)`	Yes	Whether the operation succeeded.
`unsId`	`integer (int64)`	Yes	Numeric UNS node ID.
`flowId`	`integer (int64)`	Yes	Flow identifier.

WriteItem

WriteItem

Field	Type	Required	Description
`topic`	`string`	Yes	UNS topic path to write.
`value`	`object`	No	Payload value to write.
`timeStamp`	`integer (int64)`	No	Optional Unix timestamp in milliseconds. Omit it to use server time.

WriteReq

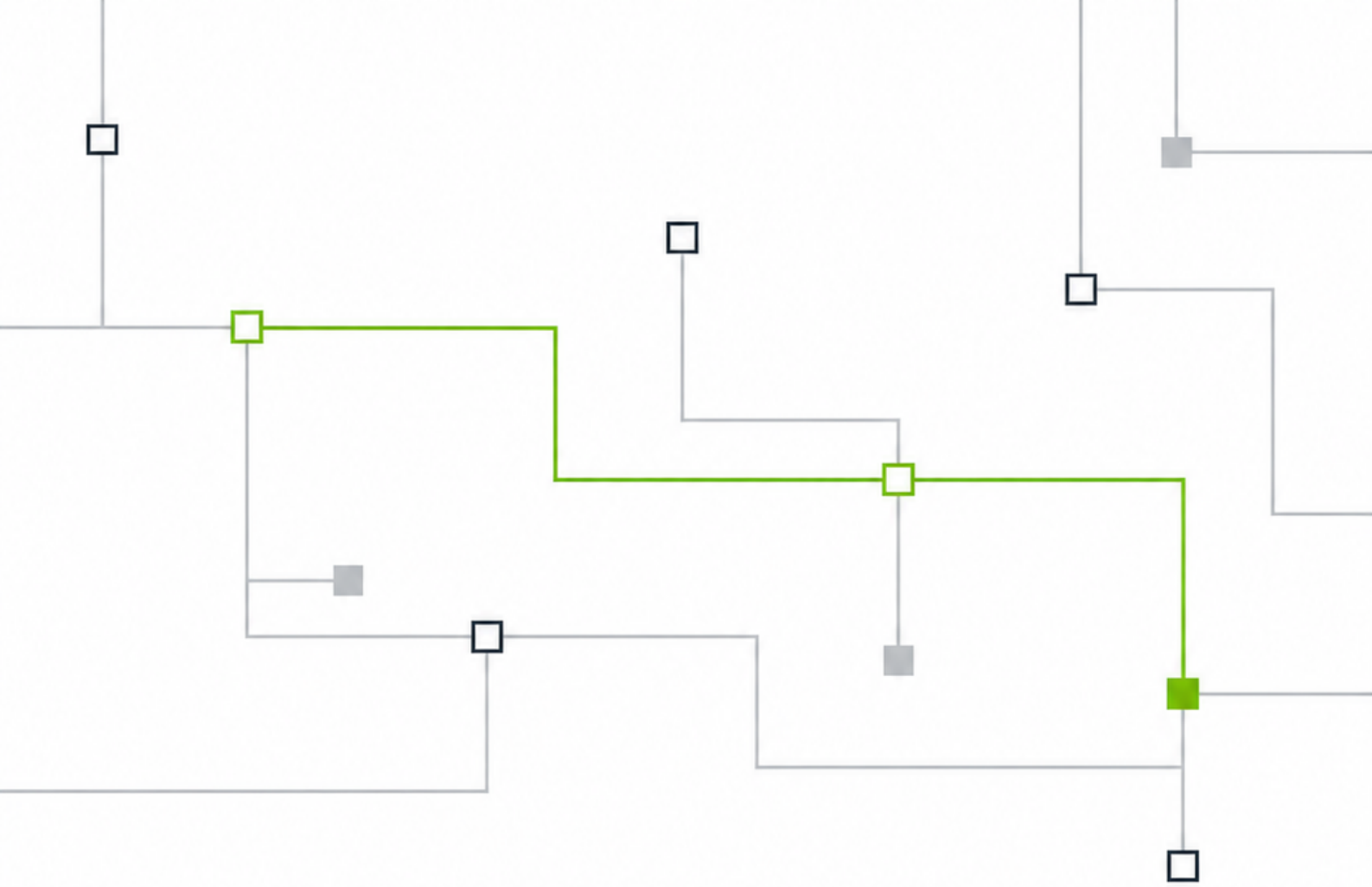
WriteReq

Field	Type	Required	Description
`writes`	`array<object>`	Yes	Write operations to apply.
`qos`	`integer (int64)`	No	MQTT QoS: 0, 1, or 2.
`retain`	`boolean (boolean)`	No	Whether the MQTT message should be retained.

WriteResp

WriteResp

Field	Type	Required	Description
`success`	`boolean (boolean)`	Yes	Whether the operation succeeded.
`results`	`array<object>`	Yes	Per-item results. Check success and error for each item.



TIER 0

tier0.app